



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Computación

Análisis del rendimiento y consumo de modelos de IA para detección de intrusiones

Performance and consumption analysis of AI models for intrusion detection

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, May 25, 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Mención en Computación

**Análisis del rendimiento y consumo de modelos de
IA para detección de intrusiones**

**Performance and consumption analysis of AI
models for intrusion detection**

Realizado por

Plácido Velasco Muñoz

Tutorizado por

Dr. Rodrigo Román Castro

Co-tutorizado por

Dr. Francisco José Jaime Rodríguez

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, MAY 25, 2026

Resumen

El proyecto se enfoca en el análisis, evaluación, selección de hiperparámetros y configuración de modelos de IA para comparar el rendimiento y eficacia computacional (CPU, latencia) de los distintos modelos aplicados a los sistemas de detección de intrusiones. Se busca determinar que modelos son más robustos, pero también eficientes desde el punto de vista computacional.

El proyecto se estructura en varias fases. Inicialmente, obtenemos y analizamos conjuntos de datos realistas proporcionados por expertos en ciberseguridad y altamente utilizados en tareas de investigación. Posteriormente, desarrollamos sistemas de detección de intrusiones, haciendo una búsqueda de los mejores hiperparámetros. Los modelos que usaremos son modelos ensembles de árboles, SVM, perceptrón multicapa y redes convolucionales 1-D.

Finalmente, miramos cómo se comportan estos modelos entrenados ante mecanismos de ataques de evasión como PGD o FGSM y compararlos para ver su degradación.

Palabras clave: sistemas de detección de intrusiones (IDS), ataques adversarios, Internet de las Cosas (IoT), eficiencia computacional, frontera de pareto, aprendizaje automático

Abstract

The project focuses on the analysis, evaluation, hyperparameter selection and configuration of AI models to compare the performance and computational efficiency (CPU, latency) of the various models applied to intrusion detection systems. The aim is to determine which models are not only more robust but also computationally efficient.

The project is structured in several phases. Initially, we obtain and analyse realistic datasets provided by cybersecurity experts and widely used in research tasks. Subsequently, we develop intrusion detection systems, searching for the optimal hyperparameters. The models we will use are ensemble models comprising decision trees, SVMs, multilayer perceptrons and 1-D convolutional networks.

Finally, we examine how these trained models perform against evasion attack mechanisms such as PGD or FGSM and compare them to assess their degradation.

Keywords: intrusion detection systems (IDS), cyberattacks, the Internet of Things (IoT), computational efficiency, pareto frontier, machine learning

Agradecimientos

Resumen	1
Abstract	3
Agradecimientos	5
1 Introducción	15
1.1 Motivación	15
1.2 Objetivos	15
1.3 Metodología de trabajo	16
1.4 Estructura del documento	16
2 Investigación Previa	17
2.1 Machine Learning	17
2.1.1 Random Forest (RF)	17
2.1.2 Extreme Gradient Boosting (XGBoost)	18
2.1.3 Light Gradient Boosting Machine (LightGBM)	19
2.1.4 CatBoost	20
2.1.5 Support Vector Machine (SVM)	21
2.2 Deep Learning	23
2.2.1 Multilayer Perceptron (MLP)	23
2.2.2 Convolutional Neural Networks (CNN)	24
3 Tecnologías utilizadas	27
3.1 Lenguaje de Programación	27
3.2 Librerías Usadas	27
3.3 Entorno de pruebas	27
3.4 Optuna	28
4 Obtención y Análisis de Datos	31
4.1 UNSW-NB15 DataSet	31
4.1.1 Motivación para el uso del dataset	31
4.1.2 Estructura del dataset	31
4.1.3 Distribución de ataques	32
4.2 ToN_IoT DataSet	33
4.2.1 Motivación para el uso del dataset	33
4.2.2 Estructura del dataset	33
4.2.3 Distribución de ataques	33
4.3 IoT-23 DataSet	33
4.3.1 Motivación para el uso del dataset	34
4.3.2 Estructura del dataset	34
4.3.3 Distribución de ataques	34

4.4	Relación entre los datasets	35
4.4.1	Justificación del enfoque en la fase de inferencia	35
4.4.2	Optimización de hiperparámetros estructurales	37
5	Análisis de Modelos y Búsqueda de Hiperparámetros	39
5.1	Modelos entrenados con UNSW-NB15	39
5.1.1	Random Forest	39
5.1.2	XGBoost	40
5.1.3	LightGBM	41
5.1.4	CatBoost	42
5.1.5	SVM	43
5.1.6	MLP	44
5.1.7	CNN-1D	45
5.1.8	Comparativa Modelos con Dataset UNSW-NB15	46
5.2	Modelos entrenados con IOT-23	50
5.2.1	Random Forest	50
5.2.2	XGBoost	51
5.2.3	LightGBM	52
5.2.4	CatBoost	53
5.2.5	SVM	54
5.2.6	MLP	55
5.2.7	CNN-1D	56
5.2.8	Comparativa Modelos con Dataset IOT-23	57
5.3	Modelos entrenados con ToN-IoT	60
5.3.1	Random Forest	60
5.3.2	XGBoost	61
5.3.3	LightGBM	62
5.3.4	CatBoost	63
5.3.5	SVM	65
5.3.6	MLP	66
5.3.7	CNN-1D	67
5.3.8	Comparativa Modelos con Dataset ToN-IoT	68
5.4	Modelos Evaluados en Local	71
5.4.1	Resultados UNSW-NB15	71
5.4.2	Resultados IOT-23	72
5.4.3	Resultados TON-IOT	72
6	Ataques de Evasión	73
6.1	Evaluación de robustez frente a ataques adversarios	73
6.1.1	Uso de Adversarial Robustness Toolbox	73
6.1.2	Análisis de Resultados en UNSW-NB15	75

6.1.3	Análisis de Resultados en IOT-23	79
6.1.4	Análisis de Resultados en TON-IOT	82
6.2	Análisis Límite de Perturbación	86
6.2.1	Resultados UNSW-NB15	86
6.2.2	Resultados IOT-23	87
6.2.3	Resultados TON-IOT	89
6.2.4	Conclusiones del análisis del límite de perturbación	90
7	Conclusiones y Líneas Futuras	93
7.1	Conclusiones	93
7.2	Líneas Futuras	93
	Bibliografía	95

2.1	Esquema representativo del funcionamiento de Random Forest.	18
2.2	Esquema representativo del funcionamiento de XGBoost.	19
2.3	Esquema representativo del funcionamiento de LightGBM.	20
2.4	Esquema representativo del funcionamiento de CatBoost.	21
2.5	Ejemplo de separación óptima de clases mediante SVM y margen máximo.	22
2.6	Proceso de retropropagación en una red neuronal MLP.	24
2.7	Esquema representativo de una arquitectura CNN 1D aplicada a datos tabulares.	25
3.1	Visualización de la optimización con Optuna	28
4.1	Logo de la University of New South Wales, entidad responsable de la publicación del dataset UNSW-NB15.	32
4.2	Logo del Stratosphere Laboratory de la Czech Technical University, entidad responsable de la publicación del dataset IoT-23.	35
5.1	Frontera de Pareto - Random Forest	39
5.2	Frontera de Pareto - XGBoost	40
5.3	Frontera de Pareto - LightGBM	41
5.4	Frontera de Pareto - CatBoost	42
5.5	Frontera de Pareto - SVM	44
5.6	Frontera de Pareto - MLP	45
5.7	Frontera de Pareto - CNN	46
5.8	Curva ROC - Modelos	48
5.9	MC - Random Forest	48
5.10	MC - XGBoost	48
5.11	MC - LightGBM	49
5.12	MC - CatBoost	49
5.13	MC - SVM	49
5.14	MC - MLP	49
5.15	MC - CNN	49
5.16	Frontera de Pareto - Random Forest (IoT-23)	50
5.17	Frontera de Pareto - XGBoost (IoT-23)	51
5.18	Frontera de Pareto - LightGBM (IoT-23)	52
5.19	Frontera de Pareto - CatBoost (IoT-23)	53
5.20	Frontera de Pareto - SVM (IoT-23)	54
5.21	Frontera de Pareto - MLP (IoT-23)	55
5.22	Frontera de Pareto - CNN-1D (IoT-23)	56

5.23	Curva ROC - Modelos (IOT-23)	58
5.24	MC - Random Forest (IOT-23)	58
5.25	MC - XGBoost (IOT-23)	58
5.26	MC - LightGBM (IOT-23)	59
5.27	MC - CatBoost (IOT-23)	59
5.28	MC - SVM (IOT-23)	59
5.29	MC - MLP (IOT-23)	59
5.30	MC - CNN (IOT-23)	59
5.31	Frontera de Pareto - Random Forest (ToN-IoT)	60
5.32	Frontera de Pareto - XGBoost (ToN-IoT)	61
5.33	Frontera de Pareto - LightGBM (ToN-IoT)	62
5.34	Frontera de Pareto - CatBoost (ToN-IoT)	64
5.35	Frontera de Pareto - SVM (ToN-IoT)	65
5.36	Frontera de Pareto - MLP (ToN-IoT)	66
5.37	Frontera de Pareto - CNN-1D (ToN-IoT)	67
5.38	Curva ROC - Modelos (ToN-IoT)	69
5.39	MC - Random Forest (ToN-IoT)	69
5.40	MC - XGBoost (ToN-IoT)	69
5.41	MC - LightGBM (ToN-IoT)	70
5.42	MC - CatBoost (ToN-IoT)	70
5.43	MC - SVM (ToN-IoT)	70
5.44	MC - MLP (ToN-IoT)	70
5.45	MC - CNN (ToN-IoT)	70
6.1	F1-PGD-UNSW-NB15	76
6.2	Accuracy-PGD-UNSW-NB15	76
6.3	Recall-PGD-UNSW-NB15	77
6.4	F1-FGSM-UNSW-NB15	78
6.5	Accuracy-FGSM-UNSW-NB15	78
6.6	Recall-FGSM-UNSW-NB15	78
6.7	F1-PGD-IOT-23	79
6.8	Accuracy-PGD-IOT-23	80
6.9	Recall-PGD-IOT-23	80
6.10	F1-FGSM-IOT-23	81
6.11	Accuracy-FGSM-IOT-23	81
6.12	Recall-FGSM-IOT-23	82
6.13	F1-PGD-TON-IOT	83
6.14	Accuracy-PGD-TON-IOT	83
6.15	Recall-PGD-TON-IOT	83

6.16	F1-FGSM-TON-IOT	84
6.17	Accuracy-FGSM-TON-IOT	85
6.18	Recall-FGSM-TON-IOT	85

2.1	Principales hiperparámetros de Random Forest y su significado.	18
2.2	Principales hiperparámetros de XGBoost y su significado.	19
2.3	Principales hiperparámetros de LightGBM y su significado.	20
2.4	Principales hiperparámetros de CatBoost y su significado.	21
2.5	Principales hiperparámetros de SVM y su significado.	22
2.6	Principales hiperparámetros de MLP y su significado.	23
2.7	Principales hiperparámetros de CNN 1D y su significado.	25
3.1	Librerías utilizadas durante el desarrollo experimental	27
3.2	Especificaciones del entorno experimental	28
4.1	Distribución de clases y tipos de ataque en el dataset UNSW-NB15	32
4.2	Distribución de clases y tipos de ataque en el dataset ToN_IoT	34
4.3	Distribución de clases y tipos de tráfico en el dataset IoT-23	35
4.4	Correspondencia principal entre características de CIC-IDS-2017 y UNSW-NB15 . .	36
4.5	Correspondencia principal entre características de ToN_IoT e IoT-23	36
5.1	Comparativa final de modelos Random Forest en el set de test	40
5.2	Comparativa final de modelos XGBoost en el set de test	41
5.3	Comparativa final de modelos LightGBM en el set de test	42
5.4	Comparativa final de modelos CatBoost en el set de test	43
5.5	Comparativa final de modelos SVM en el set de test	44
5.6	Comparativa final de modelos MLP en el set de test	45
5.7	Comparativa final de modelos CNN en el set de test	46
5.8	Definición de métricas de evaluación de rendimiento	47
5.9	Comparativa global de rendimiento de modelos - UNSW-NB15	47
5.10	Hiperparámetros ganadores de los modelos evaluados con UNSW-NB15	50
5.11	Comparativa final de modelos Random Forest en el set de test (IoT-23)	51
5.12	Comparativa final de modelos XGBoost en el set de test (IoT-23)	52
5.13	Comparativa final de modelos LightGBM en el set de test (IoT-23)	53
5.14	Comparativa final de modelos CatBoost en el set de test (IoT-23)	54
5.15	Comparativa final de modelos SVM en el set de test (IoT-23)	55
5.16	Comparativa final de modelos MLP en el set de test (IoT-23)	56
5.17	Comparativa final de modelos CNN-1D en el set de test (IoT-23)	57
5.18	Comparativa global de rendimiento de modelos - IoT-23	57
5.19	Hiperparámetros ganadores de los modelos evaluados con IoT-23	60
5.20	Comparativa final de modelos Random Forest en el set de test (ToN-IoT)	61

5.21	Comparativa final de modelos XGBoost en el set de test (ToN-IoT)	62
5.22	Comparativa final de modelos LightGBM en el set de test (ToN-IoT)	63
5.23	Comparativa final de modelos CatBoost en el set de test (ToN-IoT)	64
5.24	Comparativa final de modelos SVM en el set de test (ToN-IoT)	66
5.25	Comparativa final de modelos MLP en el set de test (ToN-IoT)	67
5.26	Comparativa final de modelos CNN-1D en el set de test (ToN-IoT)	68
5.27	Comparativa global de rendimiento de modelos - IOT-23	68
5.28	Hiperparámetros ganadores de los modelos evaluados con ToN-IoT	71
5.29	Tiempo medio de inferencia y throughput aproximado en máquina local para UNSW-NB15	71
5.30	Tiempo medio de inferencia y throughput aproximado en máquina local para IoT-23 .	72
5.31	Tiempo medio de inferencia y throughput aproximado en máquina local para TON-IoT	72
6.1	F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo MLP en UNSW-NB15	86
6.2	F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo CNN en UNSW-NB15	87
6.3	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP en UNSW-NB15	87
6.4	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN en UNSW-NB15	87
6.5	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	88
6.6	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	88
6.7	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	88
6.8	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	89
6.9	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	89
6.10	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	89
6.11	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP . .	90
6.12	F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN . .	90

1

Introducción

En esta sección se presenta la motivación para la elaboración de este proyecto, los objetivos a alcanzar, la metodología seguida y la estructura del documento.

1.1 Motivación

En el ámbito de la ciberseguridad, los sistemas de detección de intrusiones (*Intrusion Detection Systems*, IDS) desempeñan un papel fundamental en la identificación temprana de ataques y actividades maliciosas en redes informáticas. En los últimos años, el uso de técnicas de inteligencia artificial y aprendizaje automático (*Machine Learning*) ha permitido mejorar significativamente la precisión y la capacidad de adaptación de estos sistemas frente a amenazas cada vez más complejas y dinámicas.

Sin embargo, gran parte de la investigación existente se ha centrado principalmente en mejorar la tasa de detección o reducir los falsos positivos, sin considerar en la misma medida el impacto computacional que conllevan estos modelos. En entornos donde el procesamiento debe realizarse en tiempo real, como redes corporativas, sistemas embebidos, dispositivos IoT o infraestructuras críticas, el consumo de recursos, la memoria utilizada y la latencia de inferencia se convierten en factores determinantes para la viabilidad de la solución.

Por este motivo, resulta necesario analizar no solo la capacidad predictiva de los modelos, sino también su eficiencia y su comportamiento ante escenarios adversos. En este contexto, este Trabajo Fin de Grado plantea una evaluación experimental de distintos modelos de aprendizaje automático y aprendizaje profundo aplicados a la detección de intrusiones, considerando tanto su rendimiento en condiciones normales como su robustez frente a perturbaciones adversarias.

1.2 Objetivos

El objetivo principal de este Trabajo de Fin de Grado es evaluar de forma integral el rendimiento, la robustez y la viabilidad operativa de diferentes modelos de aprendizaje automático y profundo aplicados a Sistemas de Detección de Intrusiones (IDS). Se busca analizar el compromiso (trade-off) existente entre la capacidad predictiva de los algoritmos frente a amenazas camufladas y el coste computacional requerido para su ejecución en entornos con recursos limitados.

1.3 Metodología de trabajo

Para la elaboración de este TFG, se ha seguido una metodología estructurada siguiendo un enfoque iterativo y experimental. Dada la naturaleza exploratoria del proyecto, se ha seguido una metodología mista, combinando el modelo CRISP-DM con un enfoque experimental orientado al análisis de rendimiento. Esta metodología se divide en las siguientes fases:

- Estudio del estado del arte sobre aprendizaje automático y sistemas de detección de intrusiones.
- Obtención y análisis de los datasets, evaluando su viabilidad para el proyecto.
- Implementación, evaluación y búsqueda de hiperparámetros de modelos de Machine Learning y Deep Learning.
- Aplicación de ataques de evasión a los modelos ganadores
- Análisis y conclusiones de los resultados obtenidos

Además, se ha tenido un contacto continuo con los tutores, a través de mensajería y reuniones frecuentes para seguir el proceso continuo de avance del TFG y plantear las dudas correspondientes.

1.4 Estructura del documento

Esta memoria está estructurada en los siguientes capítulos:

- **Capítulo 1. Introducción:** se presenta una introducción del proyecto, a través de su motivación, los objetivos y la metodología de trabajo a seguir.
- **Capítulo 2. Investigación previa:** se tratan los conceptos básicos para entender el objetivo del proyecto. Además, se explica el contexto y estado actual de la detección de anomalías a través de aprendizaje automático.
- **Capítulo 3. Tecnologías utilizadas:** se describen las tecnologías utilizadas por el proyecto.
- **Capítulo 4. Obtención y Análisis de Datos:** se detallan los datasets elegidos con un análisis de ellos.
- **Capítulo 5. Análisis de Modelos y Búsqueda de Hiperparámetros:** se explica el proceso seguido para la búsqueda de los mejores hiperparámetros de cada modelo, junto con sus métricas y resultados.
- **Capítulo 6. Ataques de Evasión:** se tratan los modelos elegidos y se exponen ante ataques adversarios para ver la degradación de la eficacia de los modelos.
- **Capítulo 7. Conclusiones y líneas futuras:** se reúnen conclusiones obtenidas tras la realización del trabajo, además de caminos de interés a seguir para trabajos futuros.

2

Investigación Previa

2.1 Machine Learning

El *Machine Learning* (aprendizaje automático) es una rama de la inteligencia artificial que permite a los sistemas aprender y mejorar automáticamente a partir de la experiencia, sin ser programados explícitamente para cada tarea. Utiliza algoritmos que identifican patrones en grandes volúmenes de datos y toman decisiones o realizan predicciones basadas en esos patrones.

En el contexto de los Sistemas de Detección de Intrusos (IDS), el *Machine Learning* se emplea para analizar el tráfico de red y detectar comportamientos anómalos o maliciosos de manera más eficiente y precisa que los métodos tradicionales basados en firmas. Además, la relación con la *Green AI* surge porque el uso de técnicas de aprendizaje automático más eficientes puede reducir el consumo energético, promoviendo soluciones más sostenibles.

2.1.1 Random Forest (RF)

Random Forest es un modelo de aprendizaje supervisado que combina múltiples árboles de decisión para obtener una predicción más robusta que la de un único árbol. En lugar de entrenar todos los árboles con exactamente los mismos datos, cada uno se construye a partir de una muestra diferente del conjunto de entrenamiento, generada mediante *bootstrap sampling*. Además, en cada división se selecciona un subconjunto aleatorio de características. Esta combinación de aleatoriedad en los datos y en las variables es clave para que el modelo generalice mejor.

A diferencia de los métodos de *boosting*, como XGBoost, los árboles que componen un Random Forest no se entrenan de manera secuencial ni dependen unos de otros. Cada árbol aprende por su cuenta y aporta su propia "opinión", que posteriormente se integra con la del resto, normalmente mediante votación mayoritaria en clasificación. Esta independencia reduce la varianza del modelo y lo hace menos sensible al ruido o a pequeñas variaciones en los datos, algo especialmente útil en escenarios como la detección de intrusiones [3].

Tabla 2.1. Principales hiperparámetros de Random Forest y su significado.

Hiperparámetro	Significado
<code>n_estimators</code>	Número de árboles en el bosque
<code>max_depth</code>	Profundidad máxima de cada árbol
<code>min_samples_split</code>	Mínimo de muestras para dividir un nodo
<code>min_samples_leaf</code>	Mínimo de muestras en una hoja
<code>max_features</code>	Número de características consideradas en cada división
<code>bootstrap</code>	Si se usan muestras con reemplazo para entrenar cada árbol
<code>criterion</code>	Función para medir la calidad de una división (por ejemplo, <i>gini</i> o <i>entropy</i>)

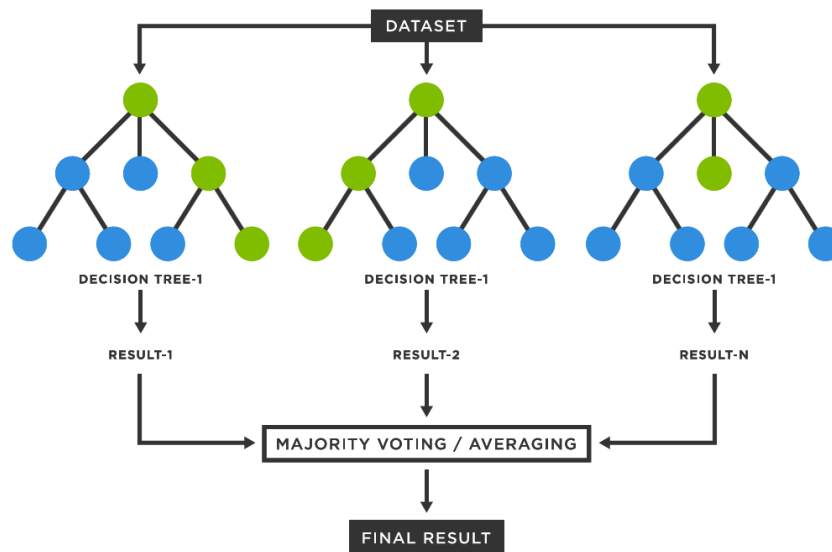


Figura 2.1. Esquema representativo del funcionamiento de Random Forest.

2.1.2 Extreme Gradient Boosting (XGBoost)

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa de forma altamente optimizada el principio de *gradient boosting*. A diferencia de otros métodos basados en ensamblados de árboles, XGBoost construye el modelo de manera secuencial, añadiendo nuevos árboles que se centran en corregir los errores cometidos por los árboles previamente entrenados. Este proceso se basa en la optimización iterativa de una función objetivo que combina el error de clasificación con términos explícitos de regularización, lo que permite controlar la complejidad del modelo y reducir el sobreajuste.

En cada iteración, XGBoost utiliza información de primer (dirección) y segundo (qué tan pronunciado es el plano) orden del gradiente de la función de pérdida para determinar la estructura óptima de los árboles, lo que le permite capturar relaciones no lineales complejas e interacciones entre características de forma eficiente. Estas propiedades lo convierten en una opción especialmente adecuada para

la detección de intrusiones en conjuntos de datos tabulares, donde los patrones maliciosos suelen manifestarse a través de combinaciones no triviales de múltiples variables.

En XGBoost, la predicción final del modelo no corresponde a la salida de un único árbol de decisión, sino que se obtiene como la combinación aditiva de las salidas de todos los árboles entrenados. Cada árbol aporta una contribución parcial a la predicción, y el resultado final se calcula sumando dichas contribuciones. El proceso de entrenamiento es secuencial: cada nuevo árbol se entrena para corregir los errores residuales cometidos por el conjunto de árboles previamente construidos. Como consecuencia, el último árbol no contiene una decisión completa por sí mismo, sino que únicamente modela aquellos patrones que aún no han sido correctamente capturados por los árboles anteriores, actuando como un ajuste fino del modelo global.

Tabla 2.2. Principales hiperparámetros de XGBoost y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles a construir
max_depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
subsample	Proporción de muestras usadas en cada iteración
colsample_bytree	Proporción de características usadas por árbol
gamma	Mínima reducción de pérdida para dividir un nodo
reg_alpha	Término de regularización L1
reg_lambda	Término de regularización L2
objective	Función objetivo a optimizar (por ejemplo, <i>binary:logistic</i>)

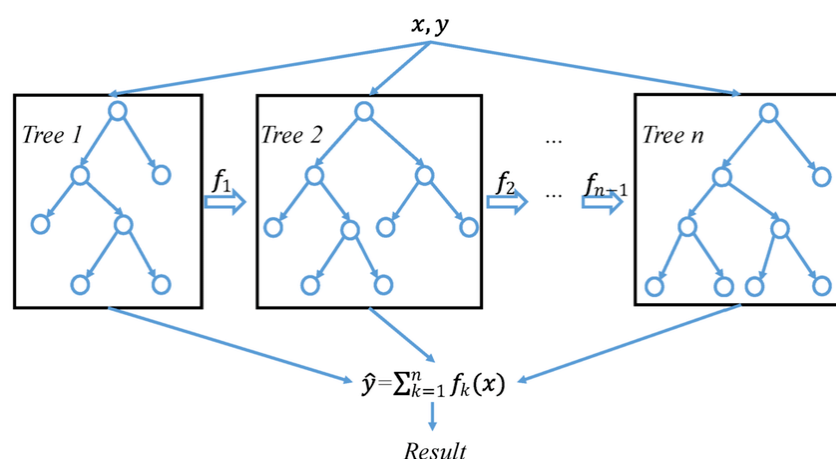


Figura 2.2. Esquema representativo del funcionamiento de XGBoost.

2.1.3 Light Gradient Boosting Machine (LightGBM)

Light Gradient Boosting Machine (LightGBM) es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*. A diferencia de

otros métodos de boosting, LightGBM utiliza un enfoque de crecimiento de árboles basado en hojas, lo que significa que en cada iteración se expande la hoja con el mayor error en lugar de expandir todos los nodos a un nivel determinado. Esta estrategia permite construir árboles más profundos y complejos sin aumentar significativamente el número de nodos, lo que mejora la capacidad de modelado y reduce el tiempo de entrenamiento [8].

Tabla 2.3. Principales hiperparámetros de LightGBM y su significado.

Hiperparámetro	Significado
n_estimators	Número de árboles a construir
num_leaves	Número máximo de hojas en cada árbol
max_depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
n_jobs	Número de threads para el entrenamiento paralelo
random_state	Semilla para reproducibilidad
verbosity	Nivel de detalle en los mensajes de salida

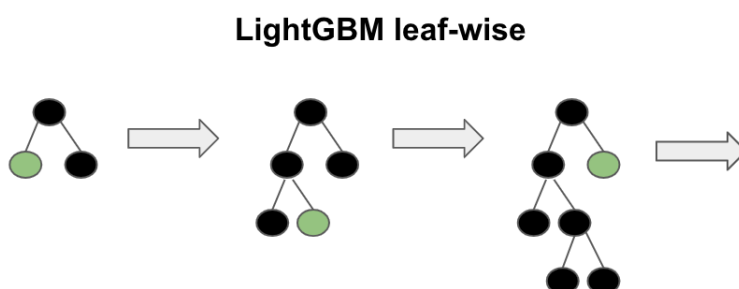


Figura 2.3. Esquema representativo del funcionamiento de LightGBM.

2.1.4 CatBoost

CatBoost es un algoritmo de aprendizaje supervisado basado en árboles de decisión que implementa una variante optimizada del *gradient boosting*, desarrollada por Yandex.

Una de las principales características de CatBoost es su capacidad para manejar de manera eficiente variables categóricas sin necesidad de realizar una codificación previa, lo que simplifica el proceso de preparación de datos y mejora el rendimiento del modelo.

Otro punto clave es que a diferencia de otros modelos como XGBoost o LightGBM, CatBoost crea árboles simétricos, lo que nos da una latencia de inferencia muy baja, ya que cada árbol tiene la

misma profundidad y se puede recorrer de manera más eficiente.

Tabla 2.4. Principales hiperparámetros de CatBoost y su significado.

Hiperparámetro	Significado
iterations	Número de árboles a construir
depth	Profundidad máxima de cada árbol
learning_rate	Tasa de aprendizaje (contribución de cada árbol)
cat_features	Índices de características categóricas
loss_function	Función de pérdida a optimizar
random_seed	Semilla para reproducibilidad

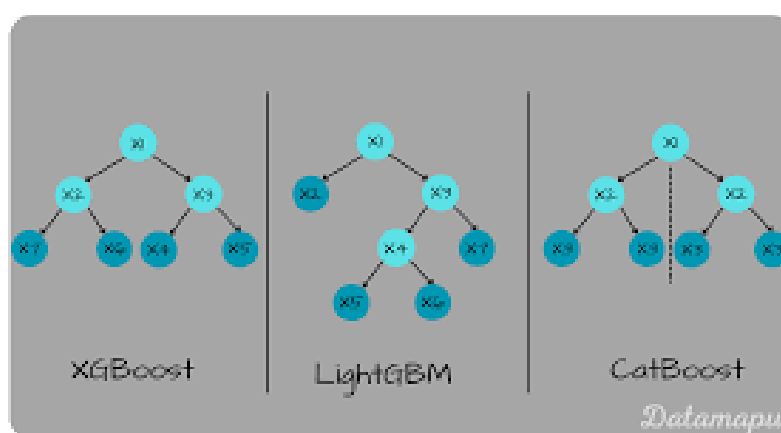


Figura 2.4. Esquema representativo del funcionamiento de CatBoost.

2.1.5 Support Vector Machine (SVM)

Support Vector Machines (SVM) es un algoritmo de aprendizaje supervisado cuyo objetivo principal es encontrar una frontera de decisión que separe de forma óptima las distintas clases en el espacio de características. En su formulación básica, SVM busca el hiperplano que maximiza el margen entre las muestras de distintas clases, utilizando únicamente un subconjunto de los datos denominado *vectores soporte*. Esta propiedad permite construir modelos robustos frente al ruido y con buena capacidad de generalización.

Una de las principales fortalezas de las SVM es el uso de *kernels*, que permiten proyectar los datos originales a espacios de mayor dimensionalidad sin realizar explícitamente dicha transformación. Gracias a este mecanismo, SVM puede modelar fronteras de decisión no lineales incluso cuando los datos no son separables linealmente en el espacio original. Entre los kernels más utilizados se encuentran:

- **Kernel lineal:** Define una frontera de decisión lineal en el espacio de características original. Es el kernel más simple y el más eficiente desde el punto de vista computacional, lo que lo hace

especialmente adecuado para datasets de gran tamaño y para escenarios donde se prioriza la eficiencia.

- **Kernel polinómico:** Permite modelar relaciones no lineales mediante funciones polinómicas de distinto grado. Aunque puede capturar interacciones más complejas entre las características, su coste computacional aumenta rápidamente con el grado del polinomio y el tamaño del dataset.
- **Kernel radial (RBF):** Proyecta los datos a un espacio de dimensión infinita, permitiendo separar patrones altamente no lineales. Si bien suele ofrecer un alto rendimiento predictivo, su entrenamiento e inferencia son significativamente más costosos, especialmente en conjuntos de datos de gran tamaño.

En el contexto de este trabajo, el uso de SVM se plantea principalmente mediante un **kernel lineal**, ya que ofrece un compromiso adecuado entre rendimiento y eficiencia computacional. Los kernels no lineales, como el RBF, resultan poco escalables para los volúmenes de datos considerados y presentan un coste energético elevado, lo que los hace menos compatibles con los principios de *Green AI*.

Tabla 2.5. Principales hiperparámetros de SVM y su significado.

Hiperparámetro	Significado
C	Parámetro de regularización que controla el equilibrio entre margen y error de clasificación
kernel	Tipo de función kernel utilizada (por ejemplo, <i>linear</i> , <i>poly</i> , <i>rbf</i>)
degree	Grado del polinomio (solo para kernel polinómico)
gamma	Coefficiente para kernels <i>rbf</i> , <i>poly</i> y <i>sigmoid</i>
coef0	Término independiente en kernels <i>poly</i> y <i>sigmoid</i>

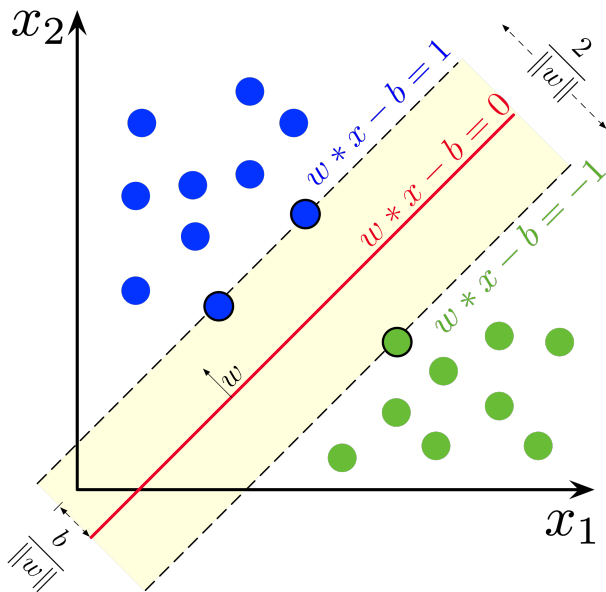


Figura 2.5. Ejemplo de separación óptima de clases mediante SVM y margen máximo.

2.2 Deep Learning

El *Deep Learning* (aprendizaje profundo) es una subárea del aprendizaje automático que se basa en redes neuronales artificiales con múltiples capas (redes neuronales profundas). Estas redes son capaces de aprender representaciones jerárquicas de los datos, lo que les permite capturar patrones complejos y realizar tareas como clasificación, reconocimiento de imágenes y procesamiento del lenguaje natural.

En el ámbito de los IDS, el *Deep Learning* se utiliza para mejorar la detección de intrusiones mediante la capacidad de aprender características complejas del tráfico de red. Sin embargo, es importante considerar que los modelos de aprendizaje profundo suelen requerir una gran cantidad de datos y recursos computacionales, lo que puede aumentar el consumo energético. Por lo tanto, es crucial buscar enfoques que optimicen el rendimiento sin comprometer la eficiencia energética.

2.2.1 Multilayer Perceptron (MLP)

El **Multi-Layer Perceptron** (MLP) es una red neuronal artificial de tipo *feed-forward* compuesta por una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa está formada por neuronas completamente conectadas, donde cada neurona realiza una combinación lineal de sus entradas seguida de una función de activación no lineal. Este tipo de arquitectura permite aproximar funciones no lineales complejas y es uno de los modelos más utilizados como punto de partida en problemas de aprendizaje profundo.

Desde el punto de vista computacional, su coste está directamente relacionado con el número de capas ocultas y el número de neuronas por capa. Arquitecturas profundas o excesivamente anchas incrementan de forma notable el tiempo de entrenamiento y el consumo de memoria, sin garantizar necesariamente mejoras en el rendimiento para datos tabulares. Por este motivo, en este trabajo se utilizan arquitecturas MLP, con un número reducido de capas y neuronas, priorizando un equilibrio adecuado entre capacidad de modelado y eficiencia computacional [10].

Tabla 2.6. Principales hiperparámetros de MLP y su significado.

Hiperparámetro	Significado
<code>hidden_layer_sizes</code>	Número y tamaño de las capas ocultas
<code>activation</code>	Función de activación utilizada en las neuronas (por ejemplo, <i>relu</i> , <i>tanh</i>)
<code>solver</code>	Algoritmo de optimización para el entrenamiento (por ejemplo, <i>adam</i> , <i>sgd</i>)
<code>alpha</code>	Parámetro de regularización L2
<code>learning_rate_init</code>	Tasa de aprendizaje inicial
<code>batch_size</code>	Tamaño del lote de muestras para cada actualización de los pesos
<code>max_iter</code>	Número máximo de iteraciones de entrenamiento
<code>early_stopping</code>	Si se detiene el entrenamiento anticipadamente al no mejorar el rendimiento

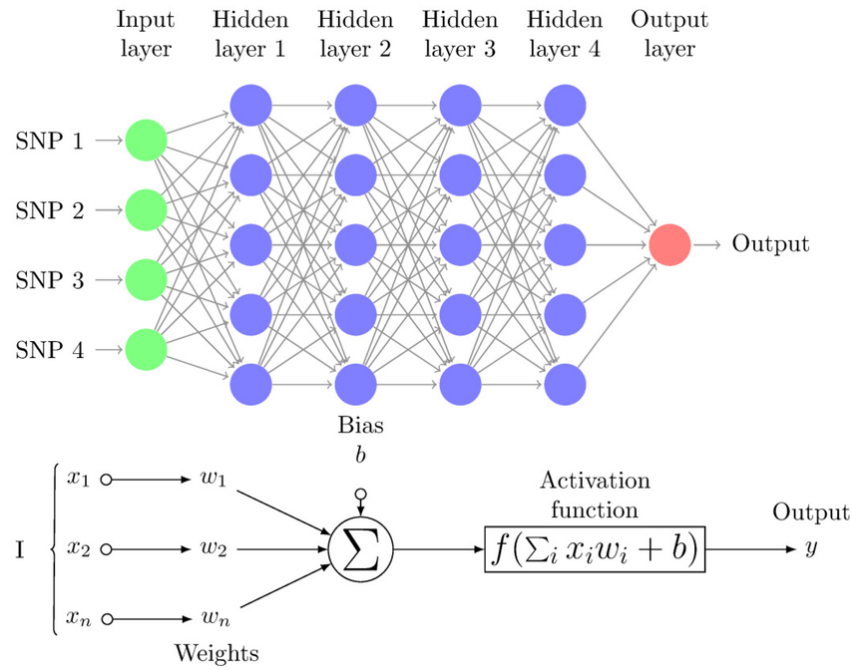


Figura 2.6. Proceso de retropropagación en una red neuronal MLP.

2.2.2 Convolutional Neural Networks (CNN)

Las **Convolutional Neural Networks** (CNN) son un tipo de red neuronal profunda diseñadas originalmente para el procesamiento de datos con estructura espacial, como imágenes o señales. Su principal característica es el uso de capas convolucionales, que aplican filtros locales sobre los datos de entrada para extraer patrones relevantes de forma jerárquica, reduciendo al mismo tiempo el número de parámetros respecto a redes completamente conectadas.

En el contexto de este trabajo, las CNN se emplean en su variante *1D*, donde las convoluciones se aplican sobre el vector de características de cada flujo de red, tratado como una señal unidimensional. Este enfoque permite capturar relaciones locales entre características adyacentes, modelando interacciones de corto alcance entre variables sin necesidad de introducir arquitecturas excesivamente complejas.

Desde el punto de vista computacional, el coste de una CNN depende principalmente del número de filtros, el tamaño del kernel y la profundidad de la red. Aunque las CNN suelen ser más eficientes que otras arquitecturas profundas al compartir pesos, un número elevado de filtros o capas puede incrementar significativamente el tiempo de entrenamiento y la latencia de inferencia.

Tabla 2.7. Principales hiperparámetros de CNN 1D y su significado.

Hiperparámetro	Significado
n_filters	Número de filtros en las capas convolucionales
kernel_size	Tamaño del kernel de convolución
dense_units	Número de neuronas en la capa densa final
time_steps	Número de pasos temporales considerados por la red
activation	Función de activación utilizada en las capas de la red
learning_rate	Tasa de aprendizaje del optimizador
batch_size	Tamaño del lote de muestras utilizado durante el entrenamiento
epochs	Número de épocas de entrenamiento

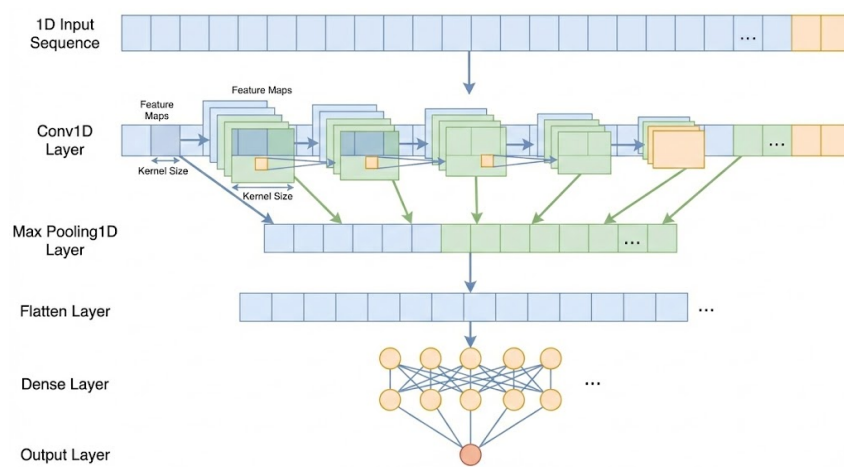


Figura 2.7. Esquema representativo de una arquitectura CNN 1D aplicada a datos tabulares.

En resumen, la selección de modelos de aprendizaje automático y profundo para la detección de intrusiones debe considerar no solo el rendimiento predictivo, sino también el coste computacional asociado.

3

Tecnologías utilizadas

3.1 Lenguaje de Programación

Para el desarrollo, experimentación y análisis de este trabajo, hemos usado **Python** como lenguaje de programación principal. En la actualidad, Python se ha consolidado como el lenguaje estándar de la comunidad científica para proyectos de ciberseguridad o Inteligencia Artificial. En cuanto al desarrollo, el proyecto se ha estructurado y ejecutado utilizando **Jupyter Notebook** para mantener un flujo de trabajo ágil y ordenado [9]. Por ejemplo, esto nos permite renderizar tablas de resultados en la misma línea del código y probar diferentes parámetros sin tener que ejecutar el código completo.

3.2 Librerías Usadas

Las librerías de código usadas en este trabajo han sido las siguientes:

Tabla 3.1. Librerías utilizadas durante el desarrollo experimental

Librería	Objetivo principal
NumPy	Operaciones numéricas, manejo de arrays y cálculo eficiente sobre matrices.
Polars	Procesamiento eficiente de datasets tabulares con gran volumen de registros.
Pandas	Manipulación de datos y creación de tablas sencillas de resultados.
Psutil	Monitorización de recursos del sistema y apoyo en la medición del rendimiento.
Matplotlib	Generación de gráficas y visualización de resultados experimentales.
Seaborn	Creación de visualizaciones estadísticas de forma más clara y legible.
TensorFlow/Keras	Implementación, entrenamiento y evaluación de modelos de deep learning.
Scikit-learn	Preprocesamiento, métricas de evaluación y entrenamiento de modelos clásicos.
XGBoost	Entrenamiento y evaluación del modelo basado en gradient boosting.
LightGBM	Entrenamiento eficiente de modelos de boosting sobre datos tabulares.
CatBoost	Entrenamiento de modelos de boosting con buen rendimiento en datos tabulares.
Adversarial Robustness Toolbox	Generación de ataques adversarios como FGSM y PGD.

3.3 Entorno de pruebas

Antes de analizar los resultados obtenidos, es importante describir el entorno de pruebas utilizado para la evaluación de los modelos. Todos los experimentos se han llevado a cabo en un entorno controlado con las siguientes características:

Componente	Especificación Técnica
Sistema Operativo	Debian GNU/Linux 12 (bookworm)
Procesador (CPU)	2x Intel® Xeon® Gold 5418Y (Arquitectura Dual-Socket)
Núcleos de CPU	48 físicos / 96 lógicos (Hilos)
Memoria RAM (Sistema)	512 GB (503 GiB disponibles)
Aceleradores Gráficos (GPU)	4x NVIDIA L40S
Memoria VRAM (Total)	192 GB (48 GB dedicados por cada GPU)
Plataforma de Cómputo	CUDA Version 13.0 (Driver 580.95.05)

Tabla 3.2. Especificaciones del entorno experimental

3.4 Optuna

Optuna es un framework de optimización de hiperparámetros de código abierto que permite a los desarrolladores y científicos de datos encontrar automáticamente los mejores hiperparámetros para sus modelos de aprendizaje automático [1]. Optuna utiliza técnicas de optimización bayesiana para explorar el espacio de hiperparámetros de manera eficiente, lo que puede mejorar significativamente el rendimiento de los modelos.

A diferencia de Grid Search, que realiza una búsqueda exhaustiva y costosa, o Random Search, que no aprende de iteraciones pasadas, Optuna utiliza un enfoque bayesiano basado en el Tree-structured Parzen Estimator (TPE).

El algoritmo TPE construye un modelo probabilístico de la función objetivo. En lugar de probar combinaciones al azar, predice qué regiones del espacio de búsqueda tienen más probabilidades de mejorar los resultados actuales.

Otra de las ventajas de este framework es que nos permite establecer una **optimización multiobjetivo**, lo que es especialmente útil cuando se desea optimizar varias métricas al mismo tiempo, como la precisión y eficiencia del modelo. [13]

Optuna identifica un conjunto de soluciones que se encuentran en la **frontera de Pareto**, lo que significa que no hay otra solución que sea mejor en todas las métricas. Esto permite a los usuarios elegir entre diferentes compromisos según sus necesidades específicas, por lo que no presentamos una única solución.



Figura 3.1. Visualización de la optimización con Optuna

El procedimiento general aplicado a cada modelo se resume en el Algoritmo 1. Aunque cada ar-

quitectura tiene un espacio de hiperparámetros específico, la estrategia de evaluación se mantiene constante: cada configuración se evalúa mediante validación cruzada estratificada y se optimiza simultáneamente el rendimiento predictivo y el coste de inferencia.

Algoritmo 1 Optimización multiobjetivo con validación cruzada estratificada

Entrada: Conjunto de entrenamiento completo $D_{train} = (X, y)$, modelo base M , espacio de hiperparámetros $\mathcal{H}$, número de ensayos N

Salida: Conjunto de configuraciones evaluadas y frontera de Pareto

- 1: Crear un estudio de Optuna con dos objetivos:
maximizar $F1$ y minimizar la latencia media de inferencia
 - 2: Definir una validación cruzada estratificada con $K = 3$ particiones
 - 3: **for** cada ensayo $t = 1, \dots, N$ **do**
 - 4: Seleccionar una configuración de hiperparámetros $h_t \in \mathcal{H}$
 - 5: Inicializar las listas $F1_scores$ y $Latencias$
 - 6: **for** cada partición $(D_{train}^{(k)}, D_{val}^{(k)})$ de la validación cruzada **do**
 - 7: Instanciar el modelo M_t con los hiperparámetros h_t
 - 8: Entrenar M_t sobre $D_{train}^{(k)}$
 - 9: Obtener las predicciones sobre $D_{val}^{(k)}$
 - 10: Calcular el $F1$ binario de la clase maliciosa
 - 11: Añadir el valor obtenido a $F1_scores$
 - 12: Seleccionar un subconjunto de validación para medir latencia
 - 13: Realizar una predicción inicial de calentamiento
 - 14: Medir varias repeticiones del tiempo de inferencia
 - 15: Calcular la latencia media por muestra en milisegundos
 - 16: Añadir la latencia obtenida a $Latencias$
 - 17: **end for**
 - 18: Calcular $F1_{medio}$ como la media de $F1_scores$
 - 19: Calcular $Latencia_{media}$ como la media de $Latencias$
 - 20: Devolver a Optuna el par $(F1_{medio}, Latencia_{media})$
 - 21: **end for**
 - 22: Obtener la frontera de Pareto a partir de todos los ensayos evaluados
-

4

Obtención y Análisis de Datos

Para cumplir con los objetivos de este Trabajo Fin de Grado, resulta imprescindible realizar una selección cuidadosa de los conjuntos de datos empleados en la fase experimental.

En particular, dado que este trabajo no se centra únicamente en la capacidad de detección, sino también en el rendimiento y el consumo de recursos computacionales, los conjuntos de datos seleccionados deben posibilitar el análisis de aspectos como la latencia de inferencia, el uso de CPU y memoria, y la escalabilidad de los modelos en distintos escenarios. El uso de datasets excesivamente simplificados o poco representativos podría conducir a conclusiones poco realistas, mientras que el empleo de conjuntos de datos desproporcionadamente grandes o complejos podría dificultar la reproducibilidad y el análisis experimental detallado.

Por este motivo, en este proyecto se han seleccionado datasets ampliamente utilizados en la literatura científica, que cubren diferentes entornos y tipologías de ataque, y que ofrecen un equilibrio adecuado entre realismo, diversidad y viabilidad experimental. Esta elección permite evaluar de forma comparativa el comportamiento de distintos modelos de aprendizaje automático y profundo.

4.1 UNSW-NB15 DataSet

4.1.1 Motivación para el uso del dataset

El conjunto de datos UNSW-NB15 se ha seleccionado como dataset principal en este trabajo con el objetivo de proporcionar un escenario más realista y actualizado para la evaluación de sistemas IDS [11]. Este dataset fue desarrollado por la *University of New South Wales* (UNSW).

4.1.2 Estructura del dataset

El dataset UNSW-NB15 está compuesto por registros de tráfico de red generados mediante la combinación de tráfico real y tráfico sintético, con el fin de simular de forma controlada distintos escenarios

de ataque en una red corporativa. El conjunto de datos se distribuye en dos archivos principales claramente diferenciados: uno destinado al entrenamiento de los modelos y otro reservado para su evaluación, lo que facilita la reproducibilidad experimental y minimiza el riesgo de *data leakage*.

Cada registro representa un flujo de red y se describe mediante un conjunto de características heterogéneas que incluyen métricas relacionadas con el volumen de tráfico, estadísticas temporales, información de protocolos y atributos derivados del comportamiento de la comunicación. En cuanto a las etiquetas, el dataset incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que especifica el tipo de ataque, lo que permite abordar tanto problemas de clasificación binaria como multiclase.

4.1.3 Distribución de ataques

Tabla 4.1. Distribución de clases y tipos de ataque en el dataset UNSW-NB15

Clase / Tipo de tráfico	Número de flujos
BENIGN	93 000
Generic	58 871
Exploits	44 525
Fuzzers	24 246
DoS	16 353
Reconnaissance	13 987
Analysis	2 677
Backdoor	2 329
Shellcode	1 511
Worms	174
Total	257 673

Como puede observarse, aunque el tráfico benigno constituye una parte relevante del dataset, una proporción significativa de los registros corresponde a tráfico malicioso, distribuido entre múltiples categorías de ataque. Destacan especialmente las clases *Generic*, *Exploits* y *Fuzzers*, mientras que otras, como *Worms* o *Shellcode*, presentan una frecuencia mucho menor. Esta distribución refleja un escenario realista y resulta adecuada para evaluar el comportamiento de los modelos tanto en términos de detección general de intrusiones como de clasificación específica de ataques.



Figura 4.1. Logo de la University of New South Wales, entidad responsable de la publicación del dataset UNSW-NB15.

4.2 ToN_IoT DataSet

4.2.1 Motivación para el uso del dataset

El conjunto de datos ToN_IoT (*Telemetry of Network IoT*) se ha incluido en este trabajo con el objetivo de evaluar el comportamiento de los modelos de detección de intrusiones en entornos IoT y *edge computing*, caracterizados por una elevada heterogeneidad, recursos computacionales limitados y requisitos estrictos de eficiencia. Este dataset fue desarrollado por la *University of New South Wales (UNSW)* con el propósito de cubrir las limitaciones de los conjuntos de datos tradicionales, que no representan adecuadamente el tráfico ni los patrones de ataque presentes en infraestructuras IoT reales [15].

4.2.2 Estructura del dataset

El dataset ToN_IoT se distribuye en múltiples subconjuntos que incluyen tráfico de red. En concreto, se ha empleado el archivo `Train_Test_Network_dataset.csv`, que proporciona los datos en formato CSV e incluye una partición predefinida de entrenamiento y prueba. Cada registro representa un flujo de red y se describe mediante un conjunto de características extraídas automáticamente, que abarcan métricas relacionadas con el volumen de tráfico, estadísticas temporales, información de protocolos y atributos derivados del comportamiento de la comunicación.

En cuanto al etiquetado, el dataset incorpora una etiqueta binaria que distingue entre tráfico benigno y tráfico malicioso, así como una etiqueta adicional que identifica el tipo de ataque. Esta estructura permite abordar tanto el problema de detección binaria de intrusiones como el análisis de los distintos tipos de tráfico malicioso presentes en entornos IoT.

4.2.3 Distribución de ataques

La distribución de clases del subconjunto de red de ToN_IoT presenta una proporción equilibrada entre tráfico benigno y varias categorías de ataques, lo que resulta especialmente interesante para evaluar el comportamiento de los modelos en un escenario IoT controlado. En la Tabla 4.2 se muestra el número total de flujos correspondiente a cada clase, considerando el conjunto completo de datos.

Como puede observarse, el dataset incluye múltiples tipos de ataques representativos de entornos IoT, tales como ataques de denegación de servicio, inyección, escaneo, ransomware y ataques de intermediario (*Man-in-the-Middle*).

4.3 IoT-23 DataSet

Tabla 4.2. Distribución de clases y tipos de ataque en el dataset ToN_IoT

Clase / Tipo de tráfico	Número de flujos
BENIGN	50 000
backdoor	20 000
ddos	20 000
dos	20 000
injection	20 000
password	20 000
ransomware	20 000
scanning	20 000
xss	20 000
mitm	1 043
Total	191 043

4.3.1 Motivación para el uso del dataset

El conjunto de datos **IoT-23** se ha incorporado en este trabajo como dataset complementario orientado a entornos IoT reales. IoT-23, desarrollado por el grupo *Stratosphere IPS* de la *Czech Technical University (CTU)*, es uno de los conjuntos de datos más representativos para el estudio de amenazas dirigidas específicamente a dispositivos IoT.

4.3.2 Estructura del dataset

IoT-23 se distribuye en forma de escenarios independientes, donde cada escenario corresponde a una captura concreta asociada a un tipo específico de malware o comportamiento malicioso. Estos escenarios se almacenan en directorios separados y presentan tamaños muy variables, desde unos pocos megabytes hasta decenas de gigabytes, lo que dificulta su uso directo en entornos experimentales con recursos limitados.

Con el fin de garantizar la viabilidad experimental y mantener la reproducibilidad de los experimentos, en este trabajo se ha seleccionado un subconjunto reducido de escenarios representativos. En concreto, se han utilizado los archivos `capture-1-1.labeled` y `capture-3-1.labeled`, que presentan un tamaño manejable y contienen tráfico IoT real con actividad maliciosa claramente identificable.

4.3.3 Distribución de ataques

La distribución de clases en los escenarios seleccionados de IoT-23 presenta un fuerte desbalanceo entre tráfico benigno y tráfico malicioso, así como una predominancia de determinados comportamientos de ataque característicos de dispositivos IoT comprometidos. En la Tabla 4.3 se muestra el número total de flujos correspondiente a cada clase y tipo de tráfico, considerando conjuntamente los archivos `capture-1-1.labeled` y `capture-3-1.labeled`.

Tabla 4.3. Distribución de clases y tipos de tráfico en el dataset IoT-23

Clase / Tipo de tráfico	Número de flujos
Malicious – PartOfAHorizontalPortScan	685 062
Benign	473 811
Malicious Attack	5 962
Malicious – C&C	16
Total	1 164 851

Como puede observarse, el tráfico malicioso asociado a escaneos horizontales de puertos domina el conjunto de datos.



Figura 4.2. Logo del Stratosphere Laboratory de la Czech Technical University, entidad responsable de la publicación del dataset IoT-23.

4.4 Relación entre los datasets

// Habría que obviar CIC-IDS-2017 ya que no se ha usado al final

A continuación se muestra una tabla simplificada que resume la correspondencia principal entre las características de CIC-IDS-2017 y UNSW-NB15, facilitando la comparación y el diseño de modelos comunes.

Ahora relacionamos los datasets ToN_IoT y IoT-23, destacando los features comunes.

4.4.1 Justificación del enfoque en la fase de inferencia

En el contexto de los Sistemas de Detección de Intrusiones (IDS), resulta fundamental distinguir entre la fase de entrenamiento del modelo y la fase de inferencia. Aunque el entrenamiento puede implicar un coste computacional elevado, este proceso se realiza de manera puntual o con una frecuencia reducida. Por el contrario, la fase de inferencia se **ejecuta de forma continua** una vez que el modelo está desplegado en un entorno real, analizando de manera permanente el tráfico de red entrante.

El coste asociado a la inferencia es el que determina la viabilidad práctica del sistema. Un modelo con excelente rendimiento predictivo pero con alta latencia o consumo excesivo de CPU y memoria puede resultar inviable para su despliegue en tiempo real. Por este motivo, el presente trabajo **centra**

Tabla 4.4. Correspondencia principal entre características de CIC-IDS-2017 y UNSW-NB15

Categoría	CIC-IDS-2017	UNSW-NB15
Duración flujo	Flow.Duration	dur
Intervalo paquetes origen	Fwd.IAT.Mean	sinpkt
Intervalo paquetes destino	Bwd.IAT.Mean	dinpkt
Paquetes origen	Total.Fwd.Packets	spkts
Paquetes destino	Total.Backward.Packets	dpkts
Bytes origen	Total.Length.of.Fwd.Packets	sbytes
Bytes destino	Total.Length.of.Bwd.Packets	dbytes
Tasa paquetes	Flow.Packets.s	rate
Tamaño medio paquetes origen	Fwd.Packet.Length.Mean	smean
Tamaño medio paquetes destino	Bwd.Packet.Length.Mean	dmean
SYN flag	SYN.Flag.Count	synack
ACK flag	ACK.Flag.Count	ackdat
Etiqueta	Label	label / attack_cat

Tabla 4.5. Correspondencia principal entre características de ToN_IoT e IoT-23

Categoría	ToN_IoT	IoT-23	Descripción
IP origen	src_ip	id.orig_h	Dirección IP origen
Puerto origen	src_port	id.orig_p	Puerto origen
IP destino	dst_ip	id.resp_h	Dirección IP destino
Puerto destino	dst_port	id.resp_p	Puerto destino
Protocolo	proto	proto	Protocolo de transporte
Duración	duration	duration	Duración de la conexión
Bytes origen	src_bytes	orig_bytes	Bytes enviados por el origen
Bytes destino	dst_bytes	resp_bytes	Bytes enviados por el destino
Paquetes origen	src_pkts	orig_pkts	Paquetes enviados por el origen
Paquetes destino	dst_pkts	resp_pkts	Paquetes enviados por el destino
Estado	conn_state	conn_state	Estado de la conexión
Etiqueta	label	label	Tráfico benigno o malicioso
Tipo ataque	type	detailed-label	Tipo de ataque

su análisis experimental en el consumo computacional durante la fase de inferencia.

4.4.2 Optimización de hiperparámetros estructurales

El ajuste de hiperparámetros se centra específicamente en aquellos que determinan la **estructura del modelo**. Los hiperparámetros estructurales influyen directamente en la complejidad computacional, el número de operaciones necesarias para generar una predicción y el tamaño final del modelo.

A diferencia de otros hiperparámetros relacionados con el proceso de entrenamiento (como la tasa de aprendizaje o el optimizador), los **hiperparámetros estructurales determinan la arquitectura final del modelo** y, por tanto, su coste en tiempo de ejecución una vez desplegado.

A continuación, se describen los hiperparámetros estructurales más relevantes para cada uno de los modelos considerados en este trabajo.

4.4.2.1 Random Forest

En el caso de Random Forest, el coste de inferencia depende principalmente de:

- **n_estimators**: número de árboles que componen el bosque. Un mayor número de árboles incrementa linealmente el número de recorridos necesarios para cada predicción.
- **max_depth**: profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones por predicción.

4.4.2.2 XGBoost

En XGBoost, la inferencia consiste en recorrer secuencialmente todos los árboles entrenados y sumar sus contribuciones. Los hiperparámetros estructurales más relevantes son:

- **n_estimators**: número total de árboles del modelo.
- **max_depth**: profundidad máxima de cada árbol.

4.4.2.3 LightGBM

En LightGBM, el coste de inferencia también está influenciado por:

- **n_estimators**: número de árboles en el modelo.
- **max_depth**: profundidad máxima de cada árbol.
- **num_leaves**: número máximo de hojas por árbol, que afecta la complejidad del modelo y el número de comparaciones necesarias para cada predicción.

4.4.2.4 CatBoost

En CatBoost, los hiperparámetros estructurales que afectan el coste de inferencia son:

- **iterations:** número de árboles a entrenar. Un mayor número de iteraciones incrementa linealmente el coste computacional en inferencia, ya que cada predicción requiere recorrer todos los árboles.
- **depth:** profundidad máxima de cada árbol. Árboles más profundos implican más comparaciones y operaciones por predicción.

4.4.2.5 Support Vector Machine (SVM)

Para SVM, el coste de inferencia depende principalmente del tipo de kernel utilizado. En este trabajo se prioriza el uso de:

- **kernel linear:** permite realizar predicciones mediante un producto escalar, lo que reduce significativamente el coste computacional.
- **C:** parámetro de regularización que puede influir en la complejidad del modelo, aunque su impacto en la inferencia es menor que el tipo de kernel.

4.4.2.6 Multilayer Perceptron (MLP)

En redes MLP, el coste de inferencia está directamente relacionado con el número total de pesos del modelo. Los hiperparámetros estructurales clave son:

- **hidden_layer_sizes:** número de capas ocultas y número de neuronas por capa.

4.4.2.7 Convolutional Neural Networks (CNN 1D)

En las CNN 1D utilizadas en este trabajo, los hiperparámetros estructurales más relevantes son:

- **n_filters:** número de filtros en las capas convolucionales.
- **kernel_size:** tamaño del kernel.
- **dense_units:** número de unidades en la capa densa.
- **time_steps:** número de pasos temporales procesados.

5

Análisis de Modelos y Búsqueda de Hiperparámetros

5.1 Modelos entrenados con UNSW-NB15

5.1.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

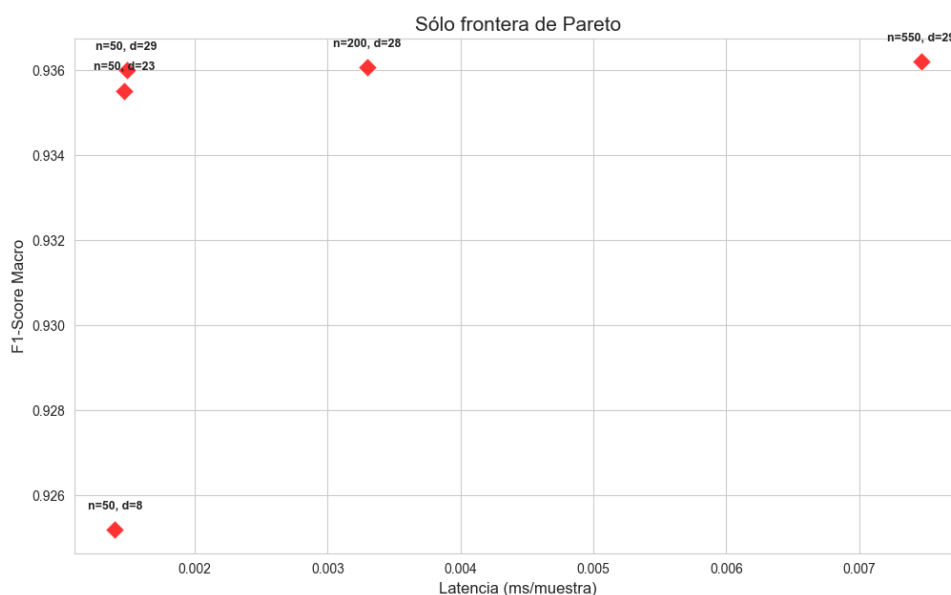


Figura 5.1. Frontera de Pareto - Random Forest

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 23)
2. (100, 29)
3. (200, 28)
4. (550, 29)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	23	0.885392	0.888622	0.001516
Candidato 2	50	29	0.873085	0.877411	0.001486
Candidato 3	200	28	0.876276	0.880338	0.003443
Candidato 4	550	29	0.87463	0.878881	0.007078

Tabla 5.1. Comparativa final de modelos Random Forest en el set de test

El **Candidato 1** no solo es el más preciso, sino también uno de los más rápidos, con una latencia de 0,0015 ms por muestra.

5.1.2 XGBoost

A continuación se muestran los resultados obtenidos por XGBoost:

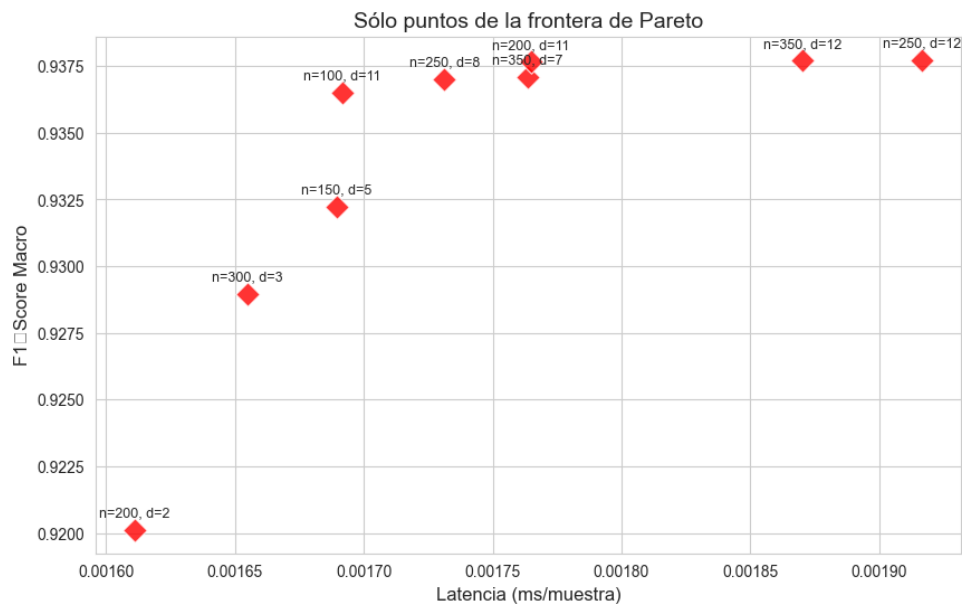


Figura 5.2. Frontera de Pareto - XGBoost

Hemos seleccionando las 3 combinaciones de la frontera de Pareto para testarlas:

1. (350, 12)
2. (200, 11)
3. (350, 7)

4. (250, 8)
5. (100, 11)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	12	0.858299	0.864378	0.000501
Candidato 2	200	11	0.857905	0.864111	0.000418
Candidato 3	350	7	0.856623	0.863152	0.000561
Candidato 4	250	8	0.856923	0.863322	0.000543
Candidato 5	100	11	0.857487	0.86388	0.000443

Tabla 5.2. Comparativa final de modelos XGBoost en el set de test

Nos quedamos con el **Candidato 2**, que ofrece un excelente equilibrio entre calidad y coste.

5.1.3 LightGBM

LightGBM ha sido entrenado con GPU, obteniendo la siguiente frontera de Pareto:

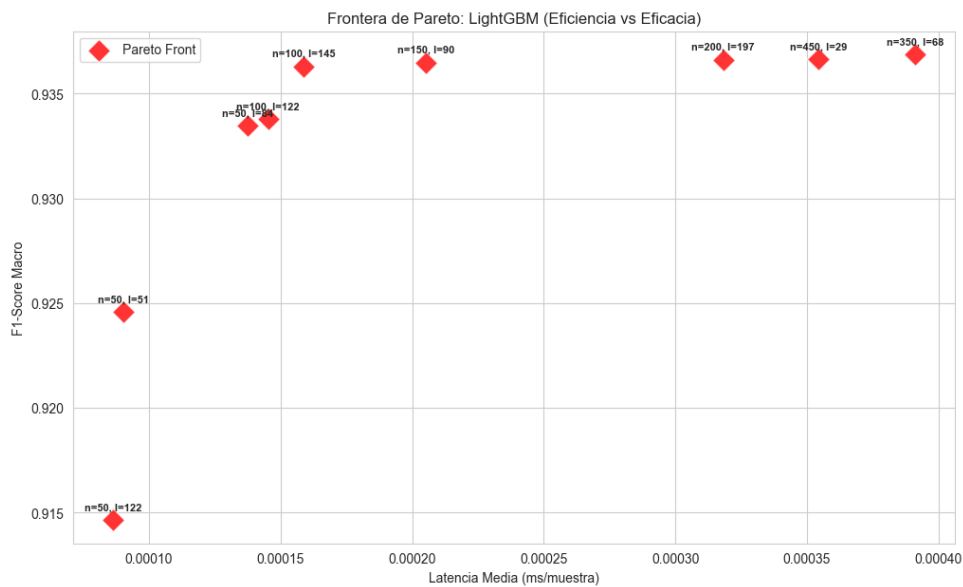


Figura 5.3. Frontera de Pareto - LightGBM

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evaluación en el set de test:

1. (350, 68, 9)
2. (450, 29, 12)
3. (150, 90, 12)
4. (100, 145, 12)

5. (100, 122, 7)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	350	68	9	0.861467	0.867354	0.000371
Candidato 2	450	29	12	0.860196	0.86631	0.00036
Candidato 3	150	90	12	0.860457	0.866552	0.0002
Candidato 4	100	145	12	0.859564	0.865775	0.000159
Candidato 5	100	122	7	0.852491	0.859654	0.000137

Tabla 5.3. Comparativa final de modelos LightGBM en el set de test

Si tuviéramos que quedarnos con un único modelo, nos quedamos con el **Candidato 4**.

5.1.4 CatBoost

La frontera de Pareto obtenida para CatBoost con el dataset UNSW-NB15 es la siguiente:

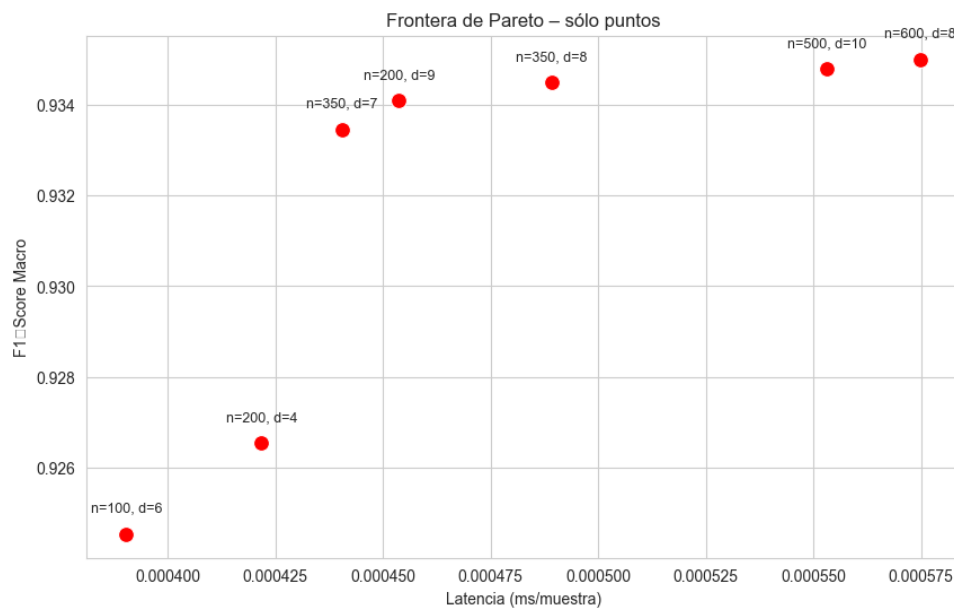


Figura 5.4. Frontera de Pareto – CatBoost

Entre los diferentes puntos de la frontera de Pareto, se han seleccionado los siguientes para su evalu-

acción en el set de test:

1. (200, 4)
2. (350, 7)
3. (200, 9)
4. (350, 8)
5. (500, 10)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	200	4	0.828623	0.838957	0.000298
Candidato 2	350	7	0.854301	0.861135	0.00044
Candidato 3	200	9	0.85276	0.859787	0.000477
Candidato 4	350	8	0.855648	0.862277	0.000489
Candidato 5	500	10	0.85702	0.863419	0.000375

Tabla 5.4. Comparativa final de modelos CatBoost en el set de test

El dato más revelador de la tabla es la latencia del **Candidato 5** (n=500, d=10). A pesar de ser el modelo más masivo y profundo de los evaluados, presenta una latencia de 0.000375 ms, siendo más rápido que modelos teóricamente más sencillos como el Candidato 4 (n=350, d=8, con 0.000489 ms).

5.1.5 SVM

La gráfica de la frontera de Pareto obtenida para SVM:

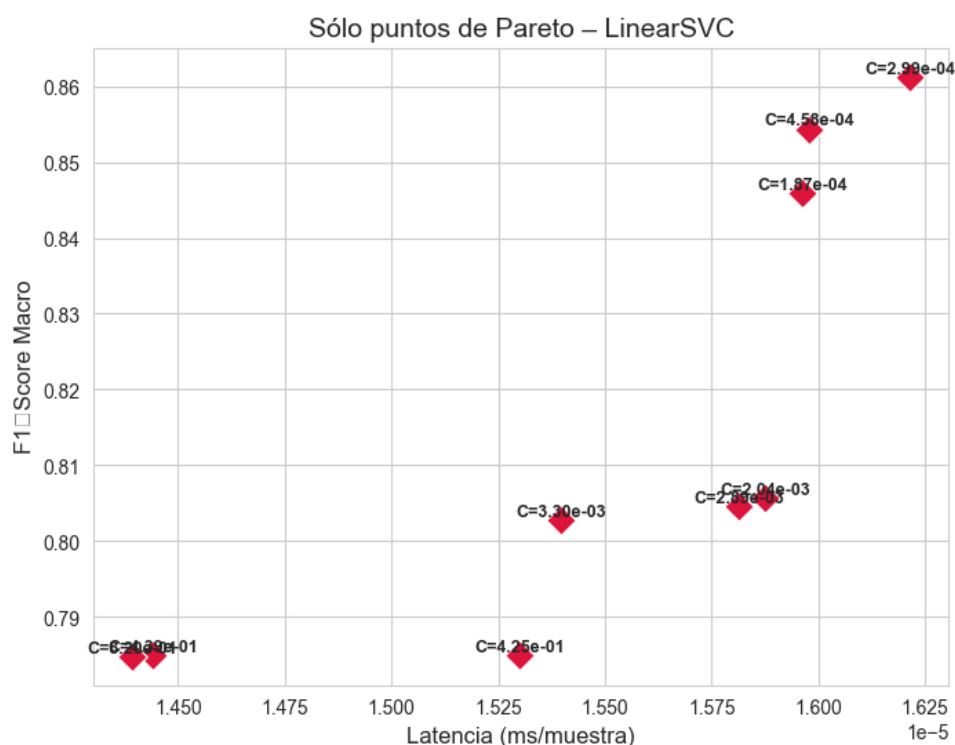


Figura 5.5. Frontera de Pareto – SVM

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	0.000299	0.708654	0.735461	0.000081
Candidato 2	0.000458	0.692147	0.722987	0.000093
Candidato 3	0.000187	0.709219	0.736105	0.000018
Candidato 4	0.002037	0.649204	0.692003	0.000019
Candidato 5	0.002889	0.648003	0.691104	0.000018

Tabla 5.5. Comparativa final de modelos SVM en el set de test

A diferencia de los modelos basados en árboles (RF, XGBoost, LightGBM) que superaban holgadamente el 85% de F1-Score, LinearSVC se estanca en un máximo de 0.7092 (Candidato 3). Esta drástica caída de rendimiento demuestra empíricamente que las intrusiones de red no son linealmente separables en este espacio de características.

Dentro de las limitaciones del enfoque lineal, el **Candidato 3** (C=0.000187) es el ganador absoluto e indiscutible.

5.1.6 MLP

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

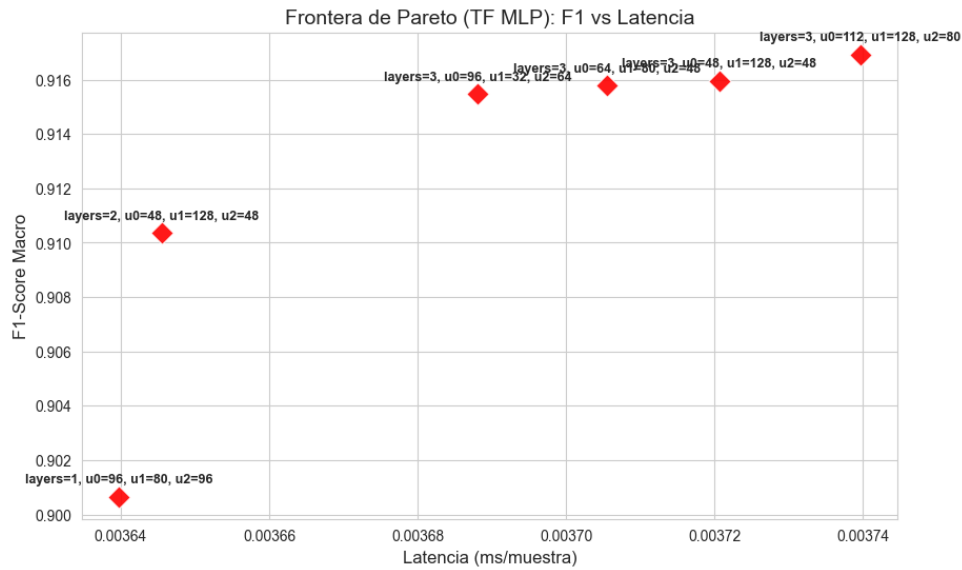


Figura 5.6. Frontera de Pareto - MLP

Ahora mostramos los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	96	80	96	0.777448	0.797017	0.00159
Candidato 2	3	48	128	48	0.758004	0.781397	0.001505
Candidato 3	3	64	80	48	0.782289	0.800515	0.001505
Candidato 4	3	96	32	64	0.786982	0.804147	0.001421

Tabla 5.6. Comparativa final de modelos MLP en el set de test

Nos quedamos con el **Candidato 4**, que ofrece el mejor rendimiento (F1-Score de 0.7869) y una latencia de inferencia de 0.001421 ms, lo que lo posiciona como un modelo competitivo dentro de los modelos no lineales, aunque aún por debajo de los basados en árboles.

5.1.7 CNN-1D

Aunque CNN se suele asociar principalmente a datos con estructura espacial (como imágenes), también se ha explorado su aplicación en la detección de intrusiones. Para su implementación, se descartó el uso de secuencias temporales (time steps) asociadas a arquitecturas como LSTM. Los conjuntos de datos empleados proporcionan información ya agregada en flujos de red independientes (formato tabular). Agrupar estas filas de forma secuencial introduciría dependencias temporales ficticias y violaría la asunción de independencia (i.i.d.) de los datos, invalidando la robustez de la evaluación mediante validación cruzada.

En su lugar, se implementó una arquitectura CNN-1D adaptando el vector de características de cada flujo mediante un redimensionamiento (reshape) a un tensor de la forma (muestras, características, 1).

Bajo este enfoque, la convolución actúa exclusivamente como un extractor automático de características latentes, descubriendo correlaciones locales no lineales entre las variables tabulares sin asumir una jerarquía temporal estricta. Esta es la gráfica de la frontera de Pareto obtenida para CNN:

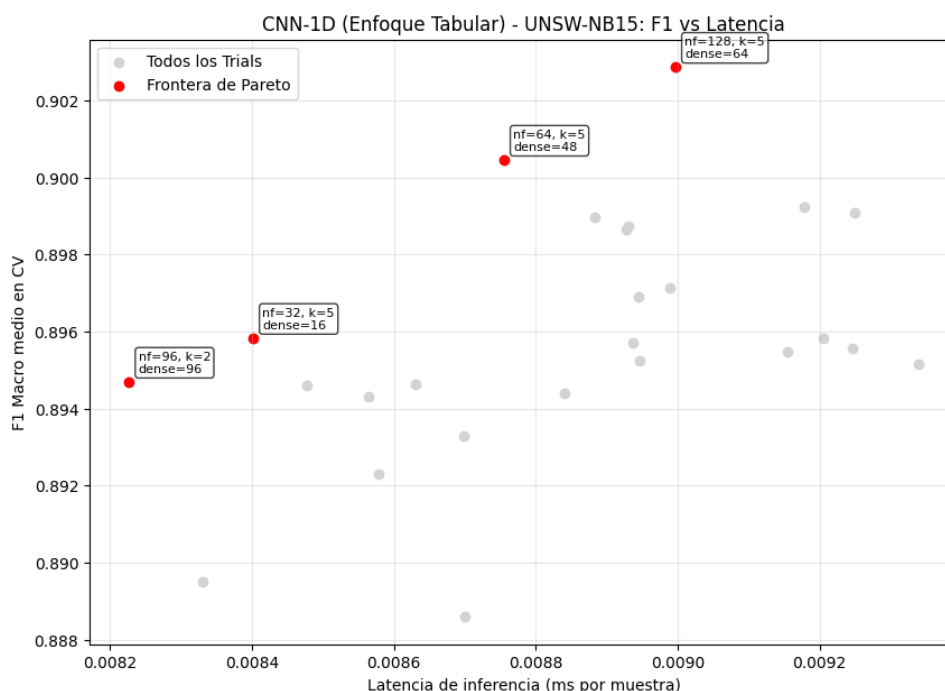


Figura 5.7. Frontera de Pareto - CNN

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	64	5	48	0.773052	0.793045	0.00888
Candidato 2	128	5	64	0.761881	0.784118	0.009398
Candidato 3	96	2	96	0.749724	0.77445	0.008625
Candidato 4	32	5	16	0.738305	0.76551	0.008725

Tabla 5.7. Comparativa final de modelos CNN en el set de test

El **Candidato 1** es el modelo ganador, con un F1-Score de 0.773052 y una latencia de inferencia de 0.00888 ms. Aunque su rendimiento es excelente, su latencia es significativamente mayor que la de los modelos basados en árboles, lo que podría limitar su aplicabilidad en entornos de producción donde la velocidad de detección es crítica.

5.1.8 Comparativa Modelos con Dataset UNSW-NB15

No solo hemos comparado los modelos a través de su F1-Score y latencia, sino que también hemos realizado una evaluación global de su rendimiento en términos de latencia, throughput, uso de CPU y RAM. Para calcular las métricas de rendimiento, hemos usado los siguientes conceptos:

Métrica	Significado Técnico	Metodología de Código
F1-Score	Métrica de eficacia predictiva. Es la media armónica entre Precisión y Sensibilidad (Recall). Representa la capacidad del modelo para detectar ataques reales minimizando las falsas alarmas, siendo ideal para tráfico desbalanceado.	Calculada mediante <code>scikit-learn (f1_score)</code> , cruzando las etiquetas reales del dataset de test frente a las predicciones generadas por el modelo.
Latencia (ms)	Tiempo de respuesta unitario. Representa los milisegundos que tarda el sistema en procesar y clasificar un único paquete desde entrada hasta veredicto.	Medida mediante <code>time.perf_counter()</code> al procesar el set de test por lotes, dividido entre el número total de paquetes procesados.
Throughput (paq/s)	Caudal de procesamiento. Representa el volumen de tráfico (paquetes por segundo) que el modelo puede soportar sin cuellos de botella.	Dividiendo el número total de paquetes del conjunto de test entre el tiempo físico total (Wall Time en segundos).
Pico RAM (MB)	Huella máxima de memoria volátil. Representa la cantidad de memoria física extra que el sistema operativo cedió al proceso durante la predicción.	Monitorizando el proceso con <code>psutil.memory_info().rss</code> , restando la memoria base al pico máximo alcanzado.

Tabla 5.8. Definición de métricas de evaluación de rendimiento

Tras hacer una fase de benchmarking con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00014	7.315991×10^6	0.09	0.9	0.7092
LightGBM	0.00064	1.551954×10^6	0.13	100.5	0.8592
XGBoost	0.00067	1.492419×10^6	0.0	101.5	0.8579
CatBoost	0.00156	641453.0	1.0	50.1	0.8570
Random Forest	0.00889	112423.0	9.04	1.6	0.8606
TensorFlow MLP	0.02443	40930.0	13.14	1.5	0.7862
CNN-1D	0.04931	20281.0	45.61	1.7	0.7665

Tabla 5.9. Comparativa global de rendimiento de modelos - UNSW-NB15

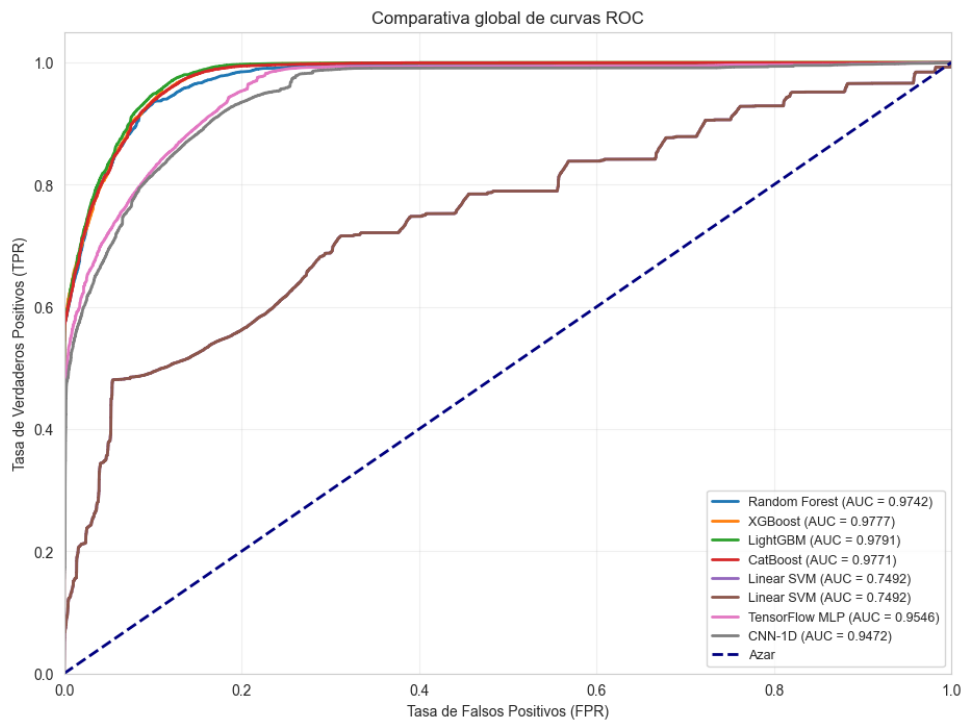


Figura 5.8. Curva ROC - Modelos

Las matrices de confusión de cada modelo se muestran a continuación:

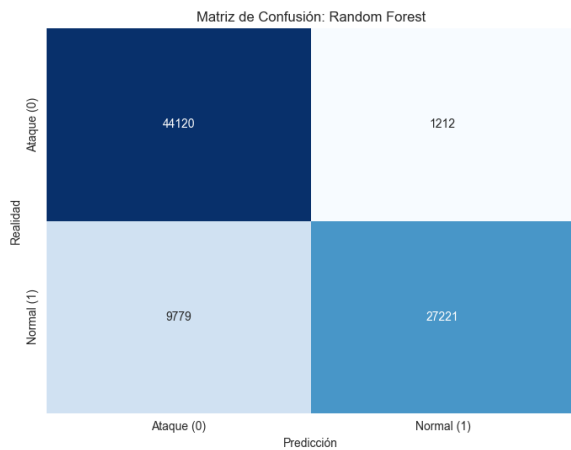


Figura 5.9. MC - Random Forest

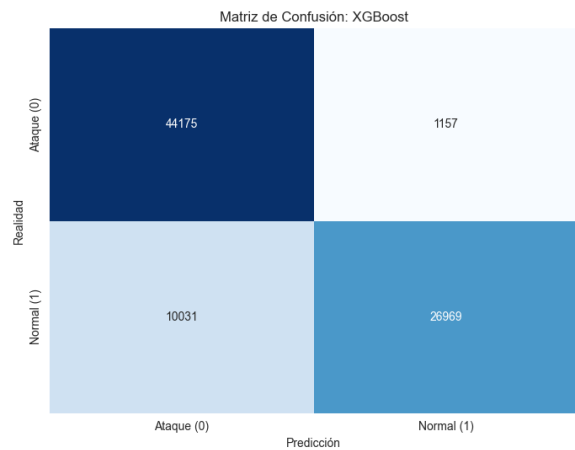


Figura 5.10. MC - XGBoost

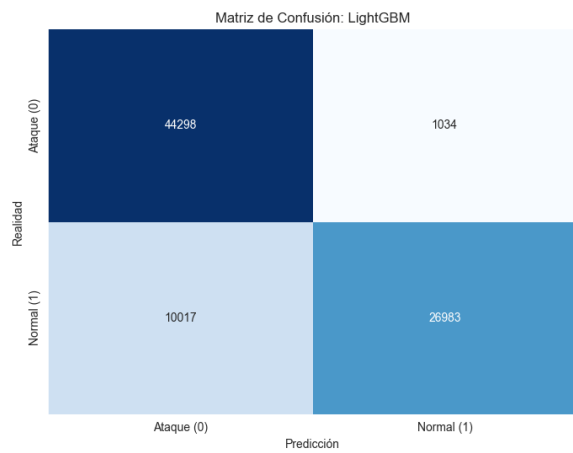


Figura 5.11. MC - LightGBM

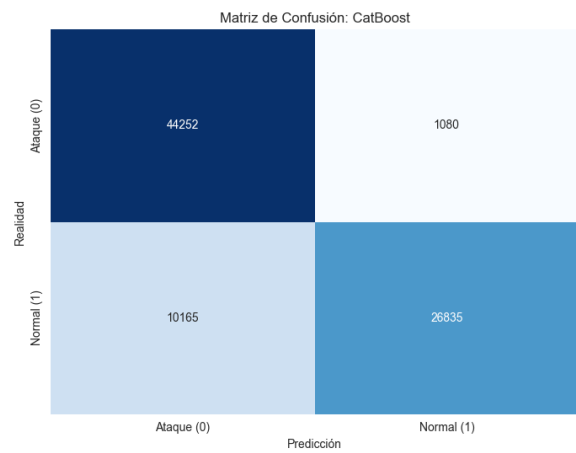


Figura 5.12. MC - CatBoost

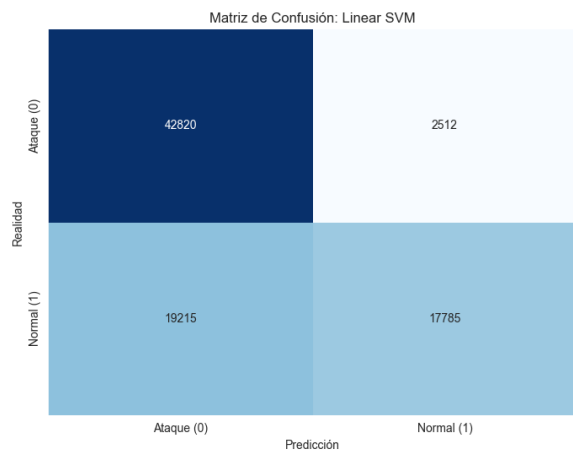


Figura 5.13. MC - SVM

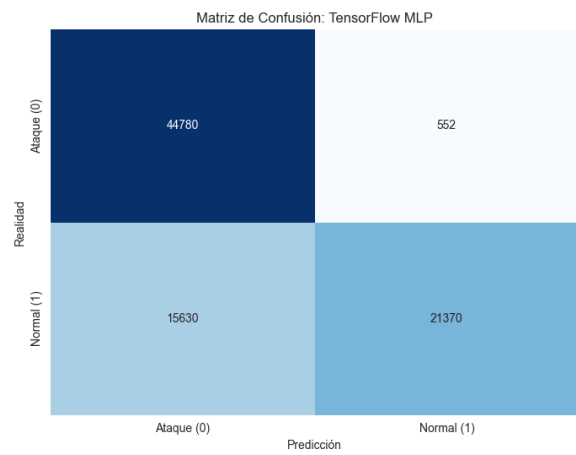


Figura 5.14. MC - MLP

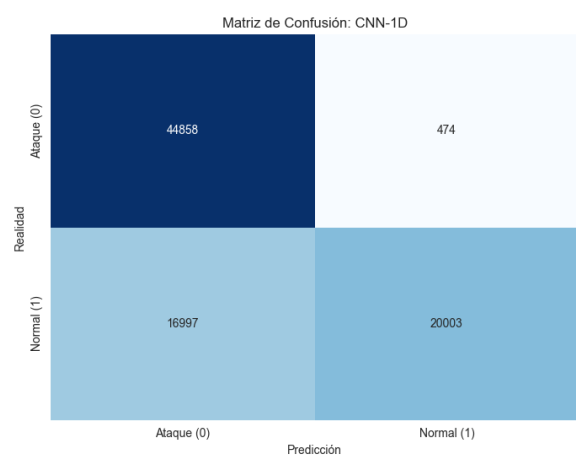


Figura 5.15. MC - CNN

Modelo	Hiperparámetros ganadores (UNSW-NB15)
Random Forest	n_estimators = 50, max_depth = 23
XGBoost	n_estimators = 200, max_depth = 11
LightGBM	n_estimators = 100, num_leaves = 145, max_depth = 12
CatBoost	n_estimators = 500, max_depth = 10
SVM	C = 0.000187
MLP	n_layers = 3, unidades = [96, 32, 64]
CNN-1D	n_filters = 64, kernel_size = 5, dense_units = 48

Tabla 5.10. Hiperparámetros ganadores de los modelos evaluados con UNSW-NB15

5.2 Modelos entrenados con IOT-23

Para completar el análisis y las comparaciones, es interesante ver cómo se comportan los modelos con un dataset adaptado a un entorno de Internet de las Cosas (IoT), que suele presentar características muy diferentes a los datasets tradicionales de tráfico de red. En este caso, se han entrenado los mismos modelos que el apartado anterior, pero usando este dataset.

5.2.1 Random Forest

Para el modelo de Random Forest, utilizando Optuna, se han obtenido los siguientes resultados como frontera de Pareto:

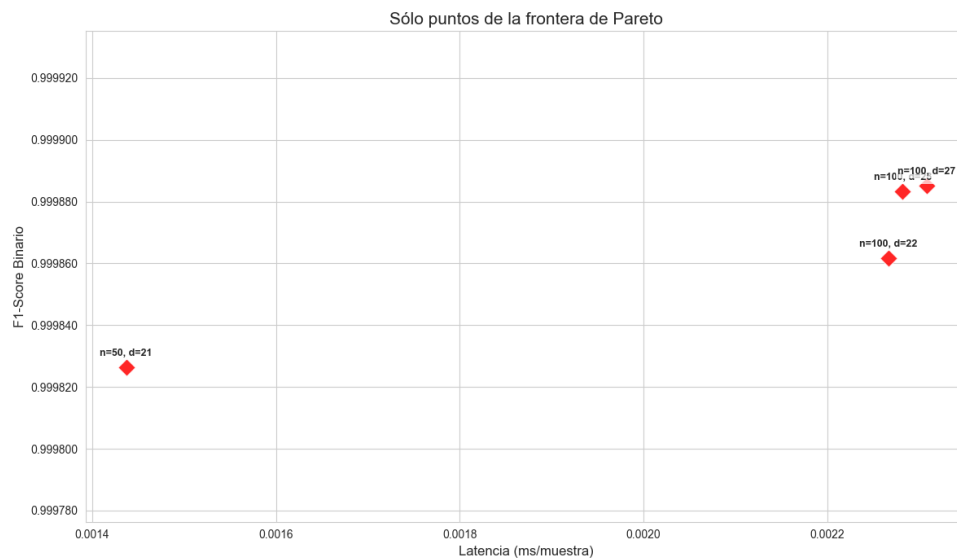


Figura 5.16. Frontera de Pareto – Random Forest (IOT-23)

Hemos visto cómo reaccionan en test los modelos con los siguientes hiperparámetros elegidos:

1. (50, 21)
2. (100, 22)
3. (100, 28)

4. (100, 27)

Los resultados son los siguientes:

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	21	0.999675	0.999614	0.001496
Candidato 2	100	22	0.999855	0.999828	0.002252
Candidato 3	100	28	0.999906	0.999888	0.002363
Candidato 4	100	27	0.999913	0.999897	0.00223

Tabla 5.11. Comparativa final de modelos Random Forest en el set de test (IOT-23)

En este caso, el **Candidato 1** es el modelo más eficiente, con un F1-Score de 0.999675 y una latencia de inferencia de 0.001496 ms, lo que lo posiciona como un modelo extremadamente preciso y rápido para este dataset específico de IoT.

5.2.2 XGBoost

Para XGBoost, la frontera de Pareto obtenida es la siguiente:

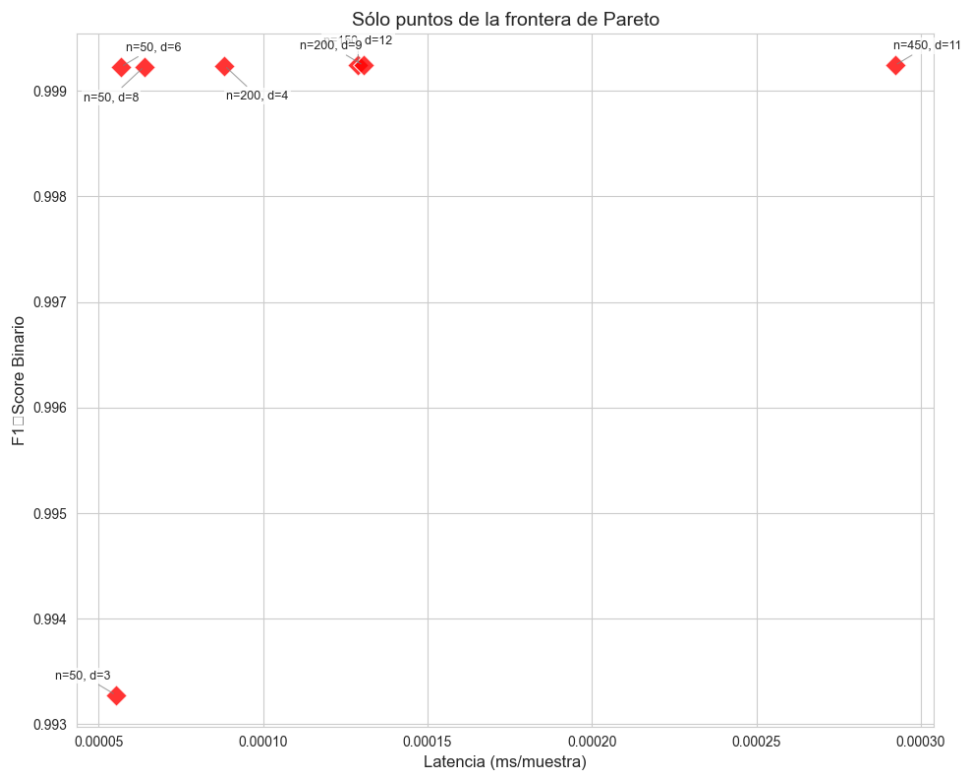


Figura 5.17. Frontera de Pareto - XGBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 8)

2. (50, 6)
3. (200, 4)
4. (200, 9)
5. (150, 12)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	8	0.999114	0.998948	0.000028
Candidato 2	50	6	0.999122	0.998957	0.000027
Candidato 3	200	4	0.999122	0.998957	0.000047
Candidato 4	200	9	0.999132	0.998970	0.000080
Candidato 5	150	12	0.999132	0.998970	0.000080

Tabla 5.12. Comparativa final de modelos XGBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999122 y una latencia de inferencia de 0.000027 ms, lo que lo posiciona como un modelo extremadamente preciso y rápido en este dataset de IoT, superando incluso a los modelos de Random Forest en términos de latencia.

5.2.3 LightGBM

La frontera de Pareto obtenida para LightGBM es la siguiente:

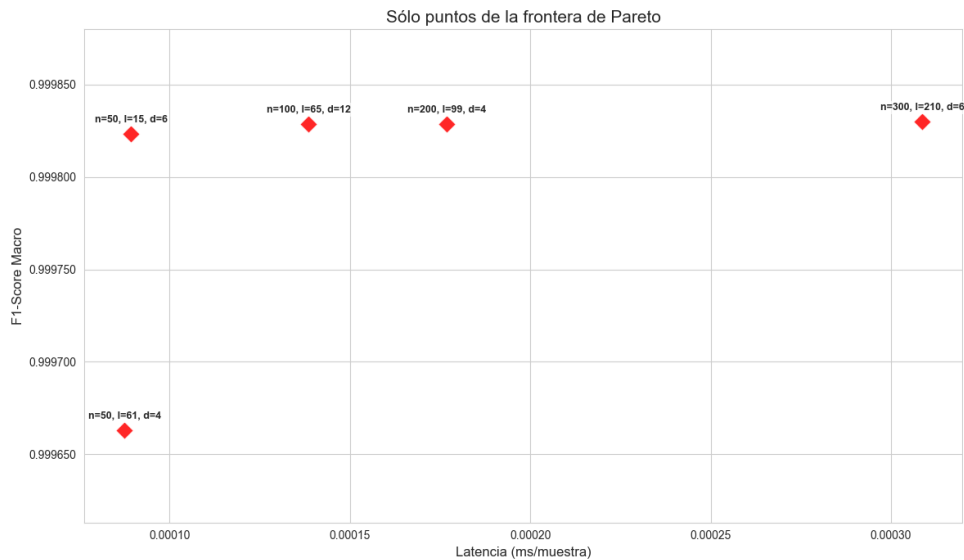


Figura 5.18. Frontera de Pareto - LightGBM (IOT-23)

Para este modelo, se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (50, 15, 6)
2. (100, 65, 12)

3. (200, 99, 4)
4. (50, 61, 4)
5. (300, 210, 6)

Los resultados obtenidos en el set de test son los siguientes:

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	15	6	0.999813	0.99982	0.000057
Candidato 2	100	65	12	0.999831	0.999837	0.000099
Candidato 3	200	99	4	0.999795	0.999803	0.000170
Candidato 4	50	61	4	0.999631	0.999644	0.000058
Candidato 5	300	210	6	0.881924	0.882891	0.000261

Tabla 5.13. Comparativa final de modelos LightGBM en el set de test (IOT-23)

En este caso, el **Candidato 1** es el mejor modelo, con un F1-Score de 0.999813 y una latencia de inferencia de 0.000057 ms, lo que lo posiciona como un modelo bastante bueno.

5.2.4 CatBoost

Este es el gráfico de la frontera de Pareto obtenida para CatBoost:

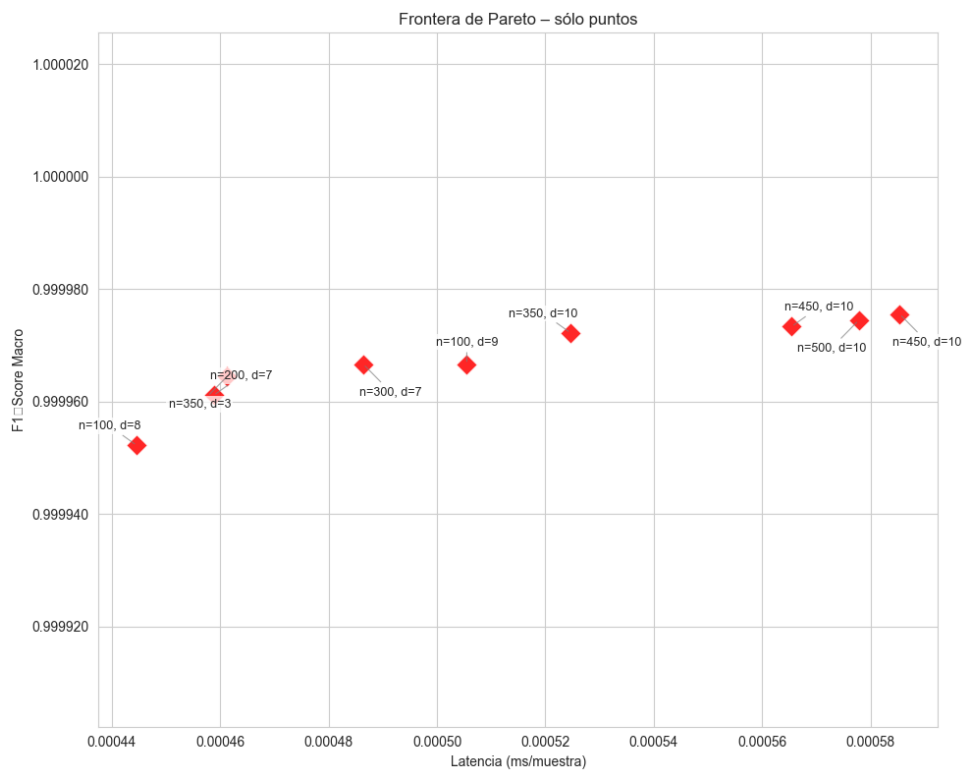


Figura 5.19. Frontera de Pareto - CatBoost (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (100, 8)
2. (350, 3)
3. (200, 7)
4. (300, 7)
5. (100, 9)
6. (350, 10)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	100	8	0.999947	0.999948	0.000242
Candidato 2	350	3	0.999960	0.999961	0.000256
Candidato 3	200	7	0.999951	0.999953	0.000263
Candidato 4	300	7	0.999951	0.999953	0.000250
Candidato 5	100	9	0.999951	0.999953	0.000259
Candidato 6	350	10	0.999951	0.999953	0.000269

Tabla 5.14. Comparativa final de modelos CatBoost en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.999960 y una latencia de inferencia de 0.000256 ms, lo que lo posiciona como un modelo extremadamente bueno.

5.2.5 SVM

La gráfica de la frontera de Pareto obtenida para SVM es la siguiente:

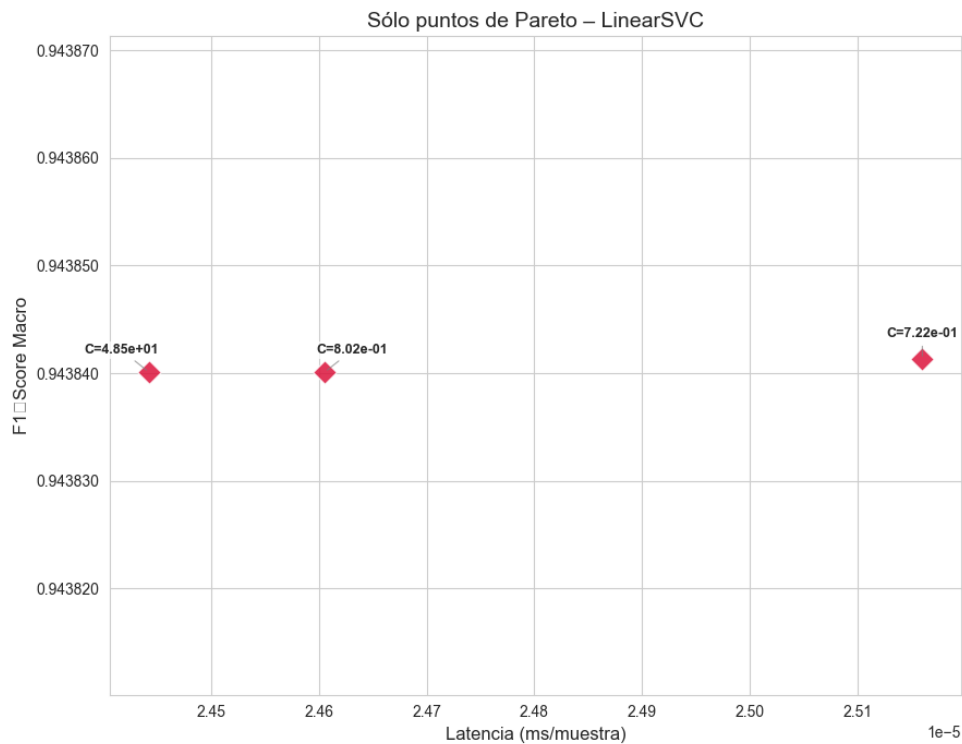


Figura 5.20. Frontera de Pareto - SVM (IOT-23)

Se han seleccionado los siguientes puntos de la frontera de Pareto para su evaluación en el set de test:

1. (48.498258)
2. (0.801714)
3. (0.721883)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	48.498258	0.943691	0.945963	0.000027
Candidato 2	0.801714	0.943695	0.945968	0.000025
Candidato 3	0.721883	0.943691	0.945963	0.000023

Tabla 5.15. Comparativa final de modelos SVM en el set de test (IOT-23)

En este caso, el **Candidato 2** es el modelo más eficiente, con un F1-Score de 0.943695 y una latencia de inferencia de 0.000025 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento es significativamente inferior al de los modelos basados en árboles.

5.2.6 MLP

La gráfica de la frontera de Pareto obtenida para MLP es la siguiente:

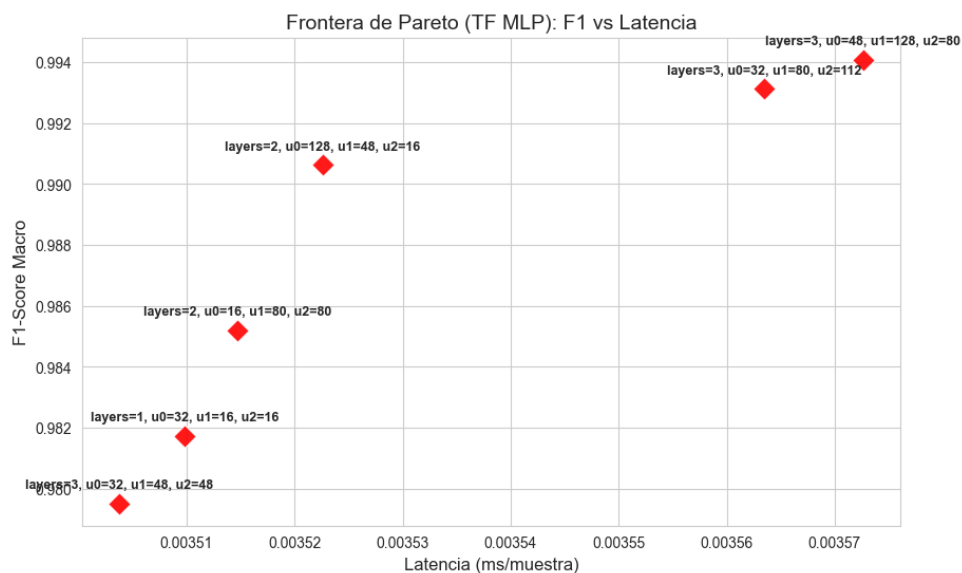


Figura 5.21. Frontera de Pareto - MLP (IOT-23)

La combinación de hiperparámetros seleccionada para su evaluación en el set de test es la siguiente:

1. (3, 32, 48, 48)
2. (1, 32, 0, 0)

3. (2, 16, 80, 0)
4. (2, 128, 48, 0)
5. (3, 32, 80, 112)
6. (3, 48, 128, 80)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	n_layers	n_units_l0	n_units_l1	n_units_l2	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	3	32	48	48	0.994819	0.995004	0.000717
Candidato 2	1	32	0	0	0.985158	0.985741	0.000621
Candidato 3	2	16	80	0	0.990610	0.990969	0.000719
Candidato 4	2	128	48	0	0.993502	0.993737	0.000772
Candidato 5	3	32	80	112	0.995042	0.995223	0.000919
Candidato 6	3	48	128	80	0.996544	0.996665	0.000906

Tabla 5.16. Comparativa final de modelos MLP en el set de test (IOT-23)

En este caso, nos quedamos con el **Candidato 1**, que ofrece un excelente rendimiento (F1-Score de 0.994819) y una latencia de inferencia de 0.000717 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

5.2.7 CNN-1D

La gráfica de la frontera de Pareto obtenida para CNN-1D es la siguiente:

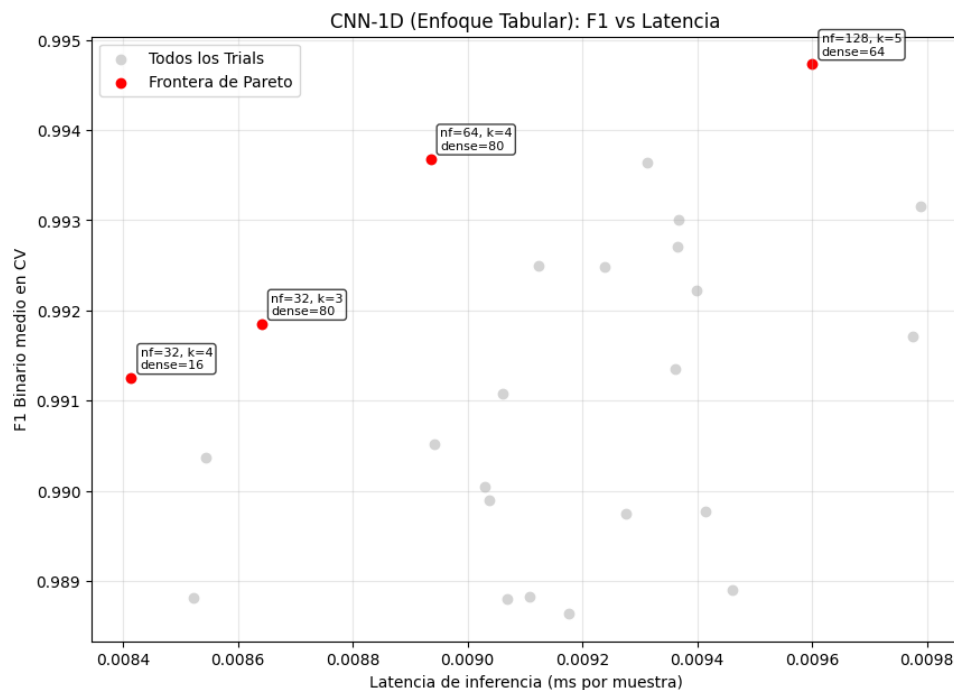


Figura 5.22. Frontera de Pareto - CNN-1D (IOT-23)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (32, 4, 16)
2. (32, 3, 80)
3. (64, 4, 80)
4. (128, 5, 64)

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	32	4	16	0.992844	0.991467	0.008293
Candidato 2	32	3	80	0.988798	0.986561	0.008442
Candidato 3	64	4	80	0.995359	0.99448	0.00892
Candidato 4	128	5	64	0.99583	0.995042	0.009345

Tabla 5.17. Comparativa final de modelos CNN-1D en el set de test (IOT-23)

En este caso, el **Candidato 3** es el modelo más eficiente, con un F1-Score de 0.995359 y una latencia de inferencia de 0.00892 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

5.2.8 Comparativa Modelos con Dataset IOT-23

Tras hacer una fase de benchmarking con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00018	5.684672×10^6	2.01	1.1	0.955
LightGBM	0.00047	2.144772×10^6	0.11	100.6	0.9998
XGBoost	0.00584	171225	0.31	96.2	0.9991
CatBoost	0.00102	982856	0.02	69.2	0.9999
Random Forest	0.00892	112103	13.39	1.4	0.9997
TensorFlow MLP	0.02377	42079	0.72	1.5	0.9956
CNN-1D	0.04959	20153	8.25	1.8	0.9952

Tabla 5.18. Comparativa global de rendimiento de modelos - IOT-23

La curva ROC de los modelos evaluados con el dataset IOT-23 es la siguiente:

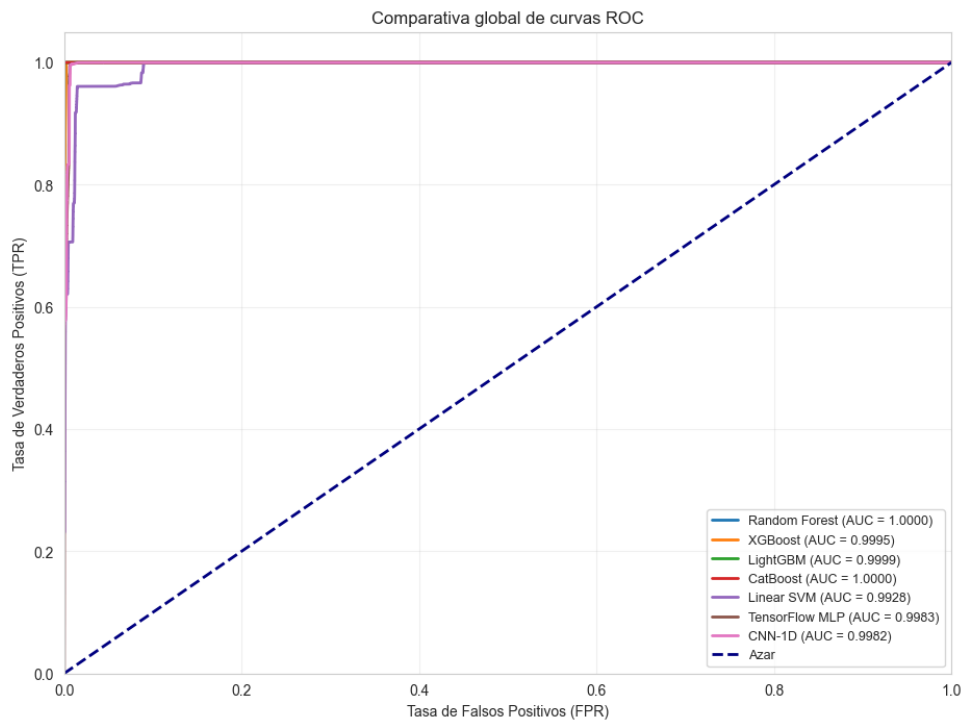


Figura 5.23. Curva ROC - Modelos (IOT-23)

Las matrices de confusión de cada modelo se muestran a continuación:

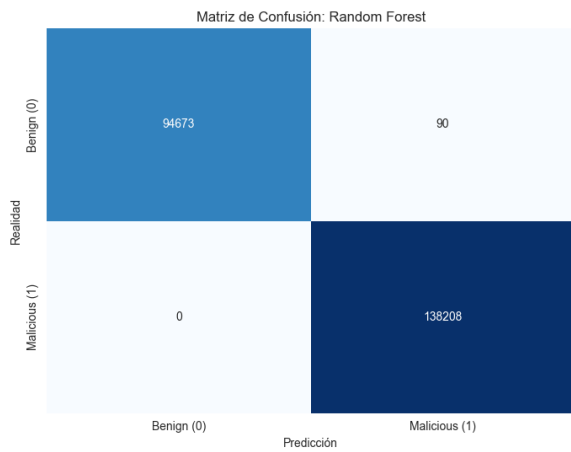


Figura 5.24. MC - Random Forest (IOT-23)

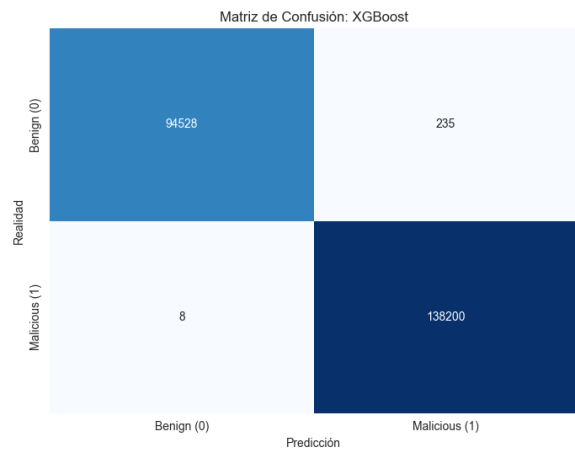


Figura 5.25. MC - XGBoost (IOT-23)

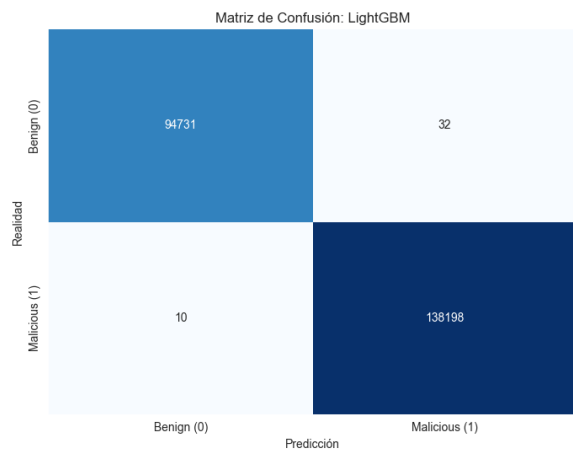


Figura 5.26. MC - LightGBM (IOT-23)

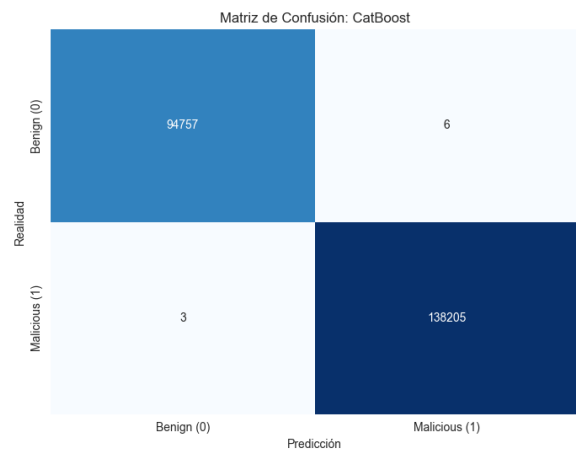


Figura 5.27. MC - CatBoost (IOT-23)

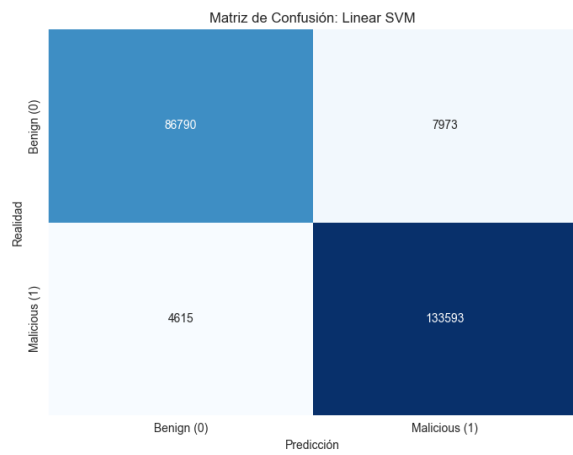


Figura 5.28. MC - SVM (IOT-23)

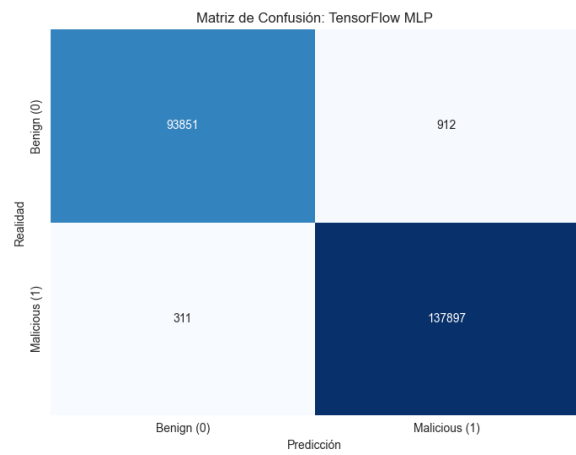


Figura 5.29. MC - MLP (IOT-23)

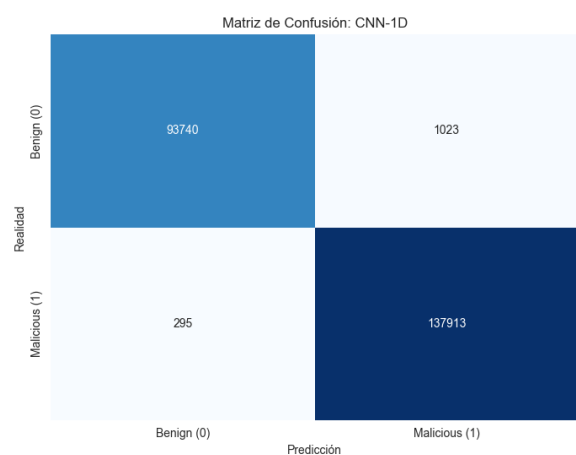


Figura 5.30. MC - CNN (IOT-23)

La tabla de los mejores hiperparámetros obtenidos para cada modelo con el dataset IOT-23 es la siguiente:

Modelo	Hiperparámetros ganadores (IOT-23)
Random Forest	n_estimators = 50, max_depth = 21
XGBoost	n_estimators = 50, max_depth = 8
LightGBM	n_estimators = 50, num_leaves = 15, max_depth = 6
CatBoost	n_estimators = 100, max_depth = 8
SVM	C = 0.801714
MLP	n_layers = 3, unidades = [32, 48, 48]
CNN-1D	n_filters = 64, kernel_size = 4, dense_units = 80

Tabla 5.19. Hiperparámetros ganadores de los modelos evaluados con IOT-23

5.3 Modelos entrenados con ToN-IoT

Una de las sospechas de los grandísimos resultados obtenidos con el dataset IOT-23 es que en este dataset la mayoría de muestras de ataque son de escaneos horizontales de puertos, lo que hace que el modelo pueda aprender a detectar ataques simplemente con la información de los puertos, sin necesidad de aprender patrones más complejos. Para comprobar esta hipótesis, se ha entrenado el mismo proceso de entrenamiento y benchmarking con el dataset ToN-IoT, que es un dataset de IoT con una mayor variedad de ataques y características.

5.3.1 Random Forest

La frontera de Pareto obtenida para Random Forest con el dataset ToN-IoT es la siguiente:

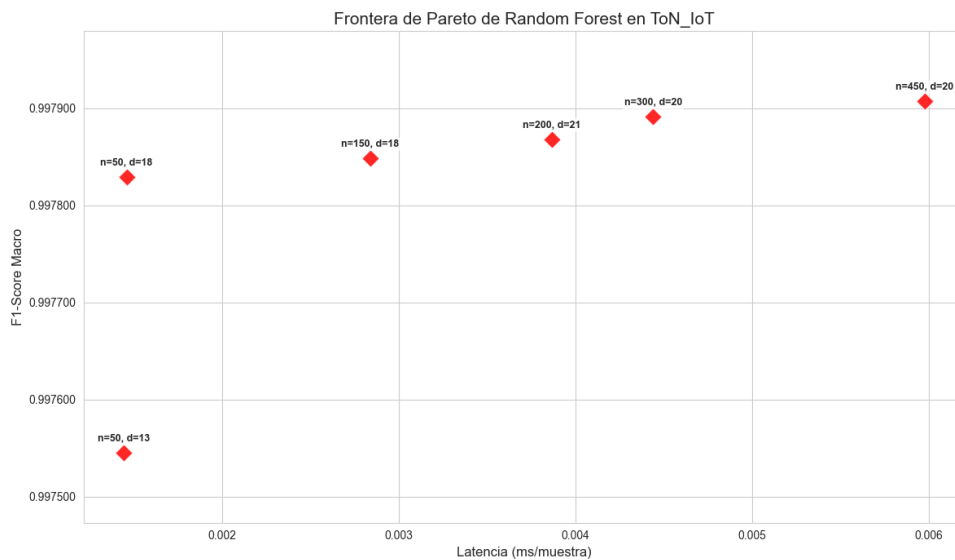


Figura 5.31. Frontera de Pareto – Random Forest (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 13)
2. (50, 18)

3. (150, 18)
4. (200, 21)
5. (300, 20)
6. (450, 20)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	13	0.995912	0.997039	0.001468
Candidato 2	50	18	0.996434	0.997418	0.001459
Candidato 3	150	18	0.996369	0.99737	0.002886
Candidato 4	200	21	0.996598	0.997536	0.004955
Candidato 5	300	20	0.996271	0.997299	0.004452
Candidato 6	450	20	0.996369	0.99737	0.00759

Tabla 5.20. Comparativa final de modelos Random Forest en el set de test (ToN-IoT)

El **Candidato 2** representa el punto de equilibrio perfecto (el óptimo de Pareto) entre capacidad de detección y coste computacional.

5.3.2 XGBoost

La frontera de Pareto obtenida para XGBoost con el dataset ToN-IoT es la siguiente:

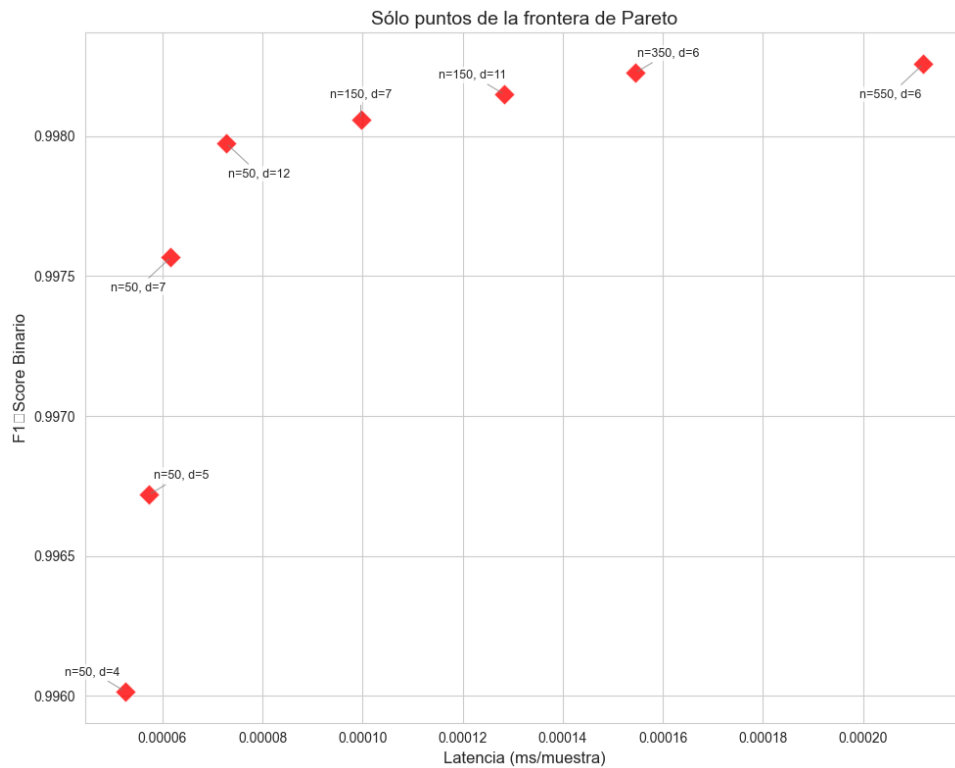


Figura 5.32. Frontera de Pareto - XGBoost (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 4)
2. (50, 5)
3. (50, 7)
4. (50, 12)
5. (150, 7)
6. (150, 11)
7. (350, 6)
8. (550, 6)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	4	0.995979	0.993864	0.00005
Candidato 2	50	5	0.99713	0.995617	0.000039
Candidato 3	50	7	0.997734	0.996541	0.000042
Candidato 4	50	12	0.998168	0.997204	0.000071
Candidato 5	150	7	0.998231	0.997299	0.000071
Candidato 6	150	11	0.998246	0.997323	0.000096
Candidato 7	350	6	0.998261	0.997347	0.000116
Candidato 8	550	6	0.99823	0.997299	0.000167

Tabla 5.21. Comparativa final de modelos XGBoost en el set de test (ToN-IoT)

Vamos a seleccionar al **Candidato 3** ya que representa un compromiso perfecto entre capacidad de detección y coste computacional, con un F1-Score de 0.997734 y una latencia de inferencia de 0.000042 ms.

5.3.3 LightGBM

La frontera de Pareto obtenida para LightGBM con el dataset ToN-IoT es la siguiente:

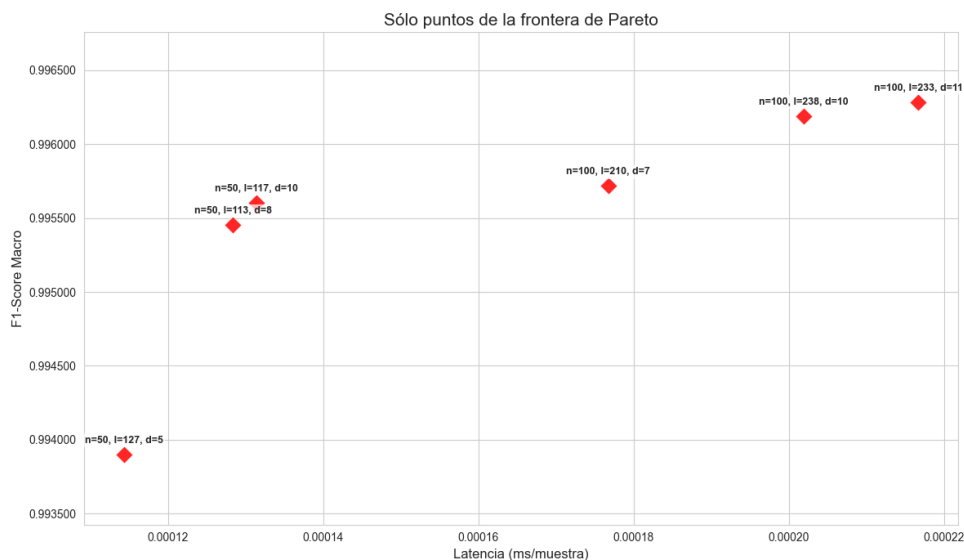


Figura 5.33. Frontera de Pareto - LightGBM (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 127, 5)
2. (50, 113, 8)
3. (50, 117, 10)
4. (100, 210, 7)
5. (100, 238, 10)
6. (100, 233, 11)

Perfil	n_estimators	num_leaves	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	127	5	0.99456	0.996067	0.00011
Candidato 2	50	113	8	0.996134	0.997204	0.000113
Candidato 3	50	117	10	0.996299	0.997323	0.000298
Candidato 4	100	210	7	0.996594	0.997536	0.000181
Candidato 5	100	238	10	0.996955	0.997797	0.000191
Candidato 6	100	233	11	0.997085	0.997891	0.000202

Tabla 5.22. Comparativa final de modelos LightGBM en el set de test (ToN-IoT)

Nos quedamos con el **Candidato 6** que ofrece un excelente rendimiento (F1-Score de 0.997085) y una latencia de inferencia de 0.000202 ms, lo que lo posiciona como un modelo muy bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

5.3.4 CatBoost

Para CatBoost, la frontera de Pareto obtenida con el dataset ToN-IoT es la siguiente:

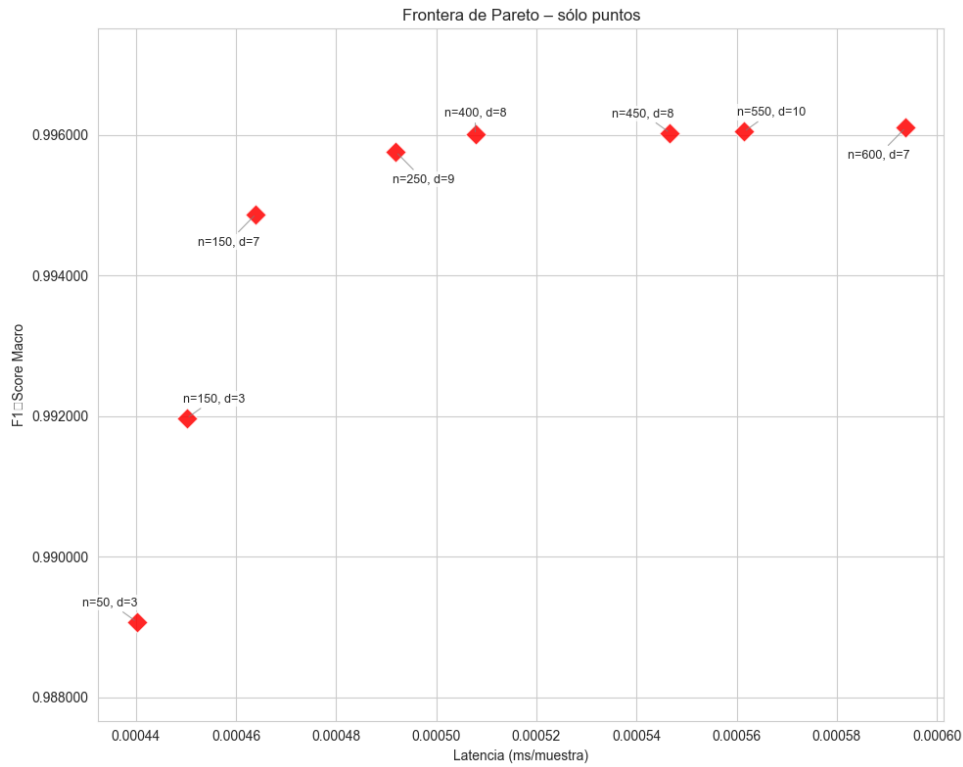


Figura 5.34. Frontera de Pareto – CatBoost (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (50, 3)
2. (150, 3)
3. (150, 7)
4. (250, 9)
5. (400, 8)
6. (450, 8)
7. (550, 10)
8. (600, 7)

Perfil	n_estimators	max_depth	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	50	3	0.98919	0.992205	0.000274
Candidato 2	150	3	0.992427	0.994527	0.00031
Candidato 3	150	7	0.995842	0.996991	0.00033
Candidato 4	250	9	0.996628	0.99756	0.000354
Candidato 5	400	8	0.996726	0.997631	0.000456
Candidato 6	450	8	0.996758	0.997655	0.00057
Candidato 7	550	10	0.996759	0.997655	0.000461
Candidato 8	600	7	0.996792	0.997678	0.000419

Tabla 5.23. Comparativa final de modelos CatBoost en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto (el óptimo de Pareto) entre capacidad de detección y coste computacional, con un F1-Score de 0.996628 y una latencia de inferencia de 0.000354 ms.

5.3.5 SVM

La frontera de Pareto obtenida para SVM con el dataset ToN-IoT es la siguiente:

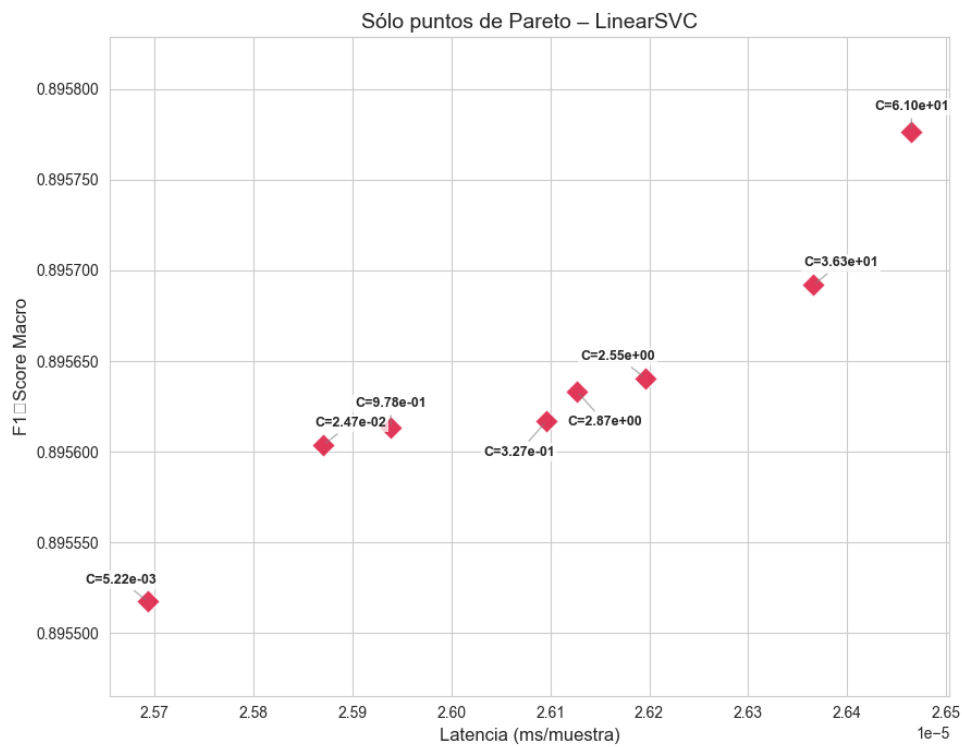


Figura 5.35. Frontera de Pareto – SVM (ToN-IoT)

Los candidatos seleccionados de la frontera de Pareto son:

1. (0.024697)
2. (0.977892)
3. (0.326653)
4. (2.870875)
5. (2.552291)
6. (36.274336)

Los resultados obtenidos en el set de test para los modelos seleccionados de la frontera de Pareto son los siguientes:

Perfil	C	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	0.024697	0.897624	0.926603	0.000027
Candidato 2	0.977892	0.897712	0.926674	0.000051
Candidato 3	0.326653	0.897712	0.926674	0.000029
Candidato 4	2.870875	0.897815	0.926745	0.000031
Candidato 5	2.552291	0.897712	0.926674	0.000047
Candidato 6	36.274336	0.897785	0.926722	0.000029

Tabla 5.24. Comparativa final de modelos SVM en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto (el óptimo de Pareto) entre capacidad de detección y coste computacional, con un F1-Score de 0.897815 y una latencia de inferencia de 0.000031 ms, lo que lo posiciona como un modelo bastante bueno para este dataset de IoT, aunque su rendimiento es significativamente inferior al de los modelos basados en árboles.

5.3.6 MLP

La frontera de Pareto obtenida para MLP con el dataset ToN-IoT es la siguiente:

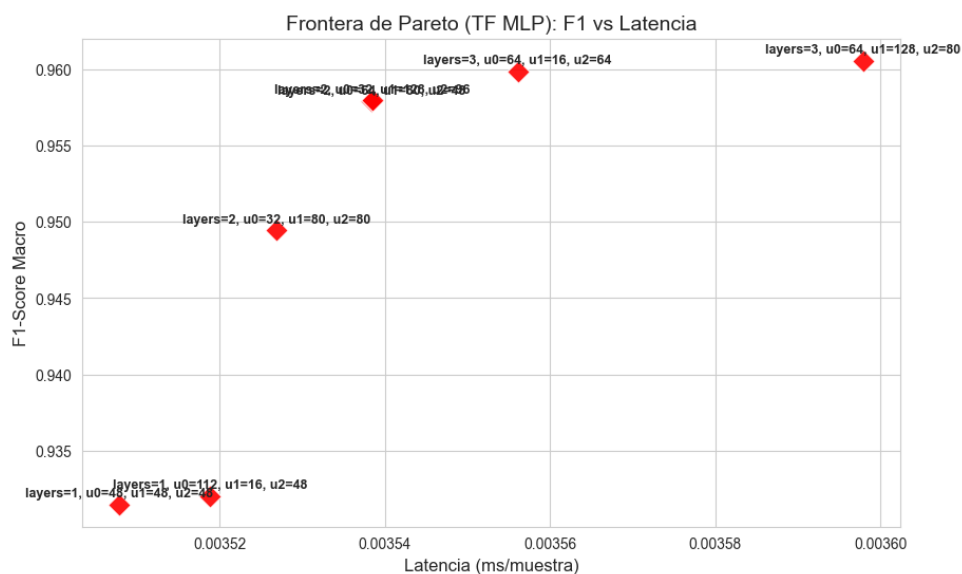


Figura 5.36. Frontera de Pareto - MLP (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (2, 32, 80, 0)
2. (2, 64, 80, 0)
3. (2, 32, 128, 0)
4. (3, 64, 16, 64)

Perfil	Capas	U. I0	U. I1	U. I2	F1	Accuracy	Lat. (ms)
Candidato 1	2	32	80	0	0.961973	0.9733	0.002257
Candidato 2	2	64	80	0	0.962048	0.973418	0.002328
Candidato 3	2	32	128	0	0.96124	0.972873	0.00237
Candidato 4	3	64	16	64	0.963034	0.97401	0.002268

Tabla 5.25. Comparativa final de modelos MLP en el set de test (ToN-IoT)

El **Candidato 4** representa el punto de equilibrio perfecto (el óptimo de Pareto) entre capacidad de detección y coste computacional, con un F1-Score de 0.963034 y una latencia de inferencia de 0.002268 ms, lo que lo posiciona como un modelo bastante bueno para este dataset de IoT, aunque su rendimiento sigue siendo inferior al de los modelos basados en árboles.

5.3.7 CNN-1D

La frontera de Pareto obtenida para CNN-1D con el dataset ToN-IoT es la siguiente:

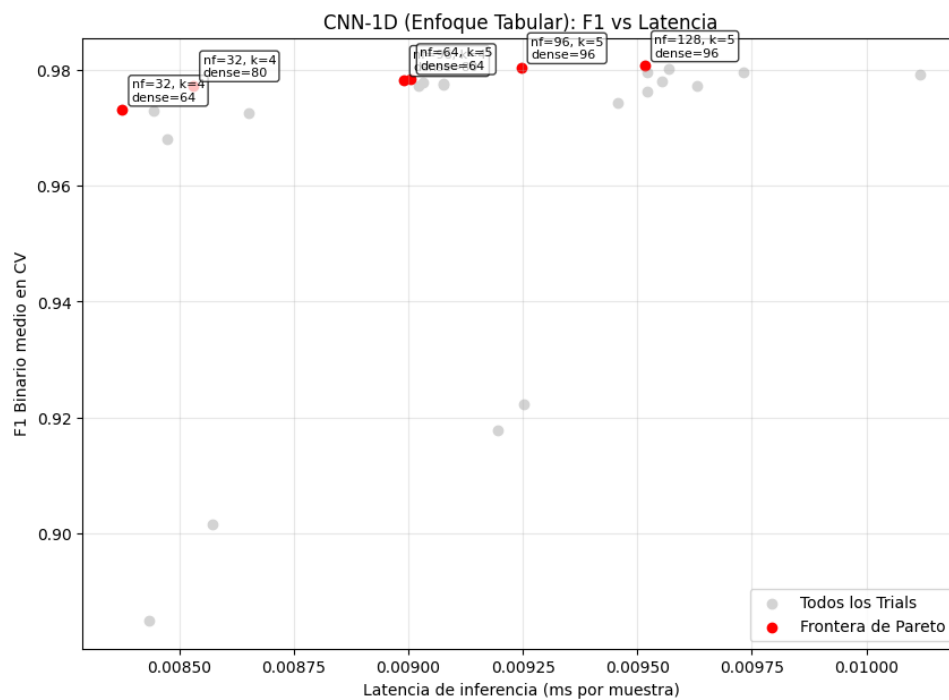


Figura 5.37. Frontera de Pareto - CNN-1D (ToN-IoT)

Los modelos seleccionados de la frontera de Pareto y sus resultados en el set de test son los siguientes:

1. (32, 4, 64)
2. (32, 4, 80)
3. (96, 4, 80)
4. (64, 5, 64)
5. (96, 5, 96)

6. (128, 5, 96)

Perfil	n_filters	kernel_size	dense_units	F1_Test	Accuracy_Test	Latencia (ms)
Candidato 1	32	4	64	0.977894	0.96586	0.008596
Candidato 2	32	4	80	0.974993	0.96124	0.009421
Candidato 3	96	4	80	0.978447	0.966737	0.009012
Candidato 4	64	5	64	0.981183	0.970836	0.009133
Candidato 5	96	5	96	0.981769	0.97176	0.008953
Candidato 6	128	5	96	0.981696	0.971641	0.009564

Tabla 5.26. Comparativa final de modelos CNN-1D en el set de test (ToN-IoT)

El ganador es el **Candidato 5** que ofrece el rendimiento más alto (F1-Score de 0.981769) y una latencia de inferencia de 0.008953 ms.

5.3.8 Comparativa Modelos con Dataset ToN-IoT

Tras hacer una fase de benchmarking con los mejores candidatos de los diferentes modelos, hemos obtenido los siguientes resultados:

Modelo	Latencia (ms)	Throughput (paq/s)	Incremento RAM (MB)	CPU Efectiva (%)	F1-Score
Linear SVM	0.00019	5.185815×10^6	0.02	1.3	0.9522
LightGBM	0.0003	3.333129×10^6	0.0	105.3	0.9977
XGBoost	0.00065	1.536528×10^6	0.2	101.8	0.9986
CatBoost	0.00129	773537.0	0.03	57.3	0.9985
Random Forest	0.00886	112889.0	8.04	1.5	0.9983
TensorFlow MLP	0.02633	37985.0	1.92	1.5	0.9829
CNN-1D	0.0531	18831.0	89.74	2.4	0.9826

Tabla 5.27. Comparativa global de rendimiento de modelos - IOT-23

La curva ROC de los modelos evaluados con el dataset ToN-IoT es la siguiente:

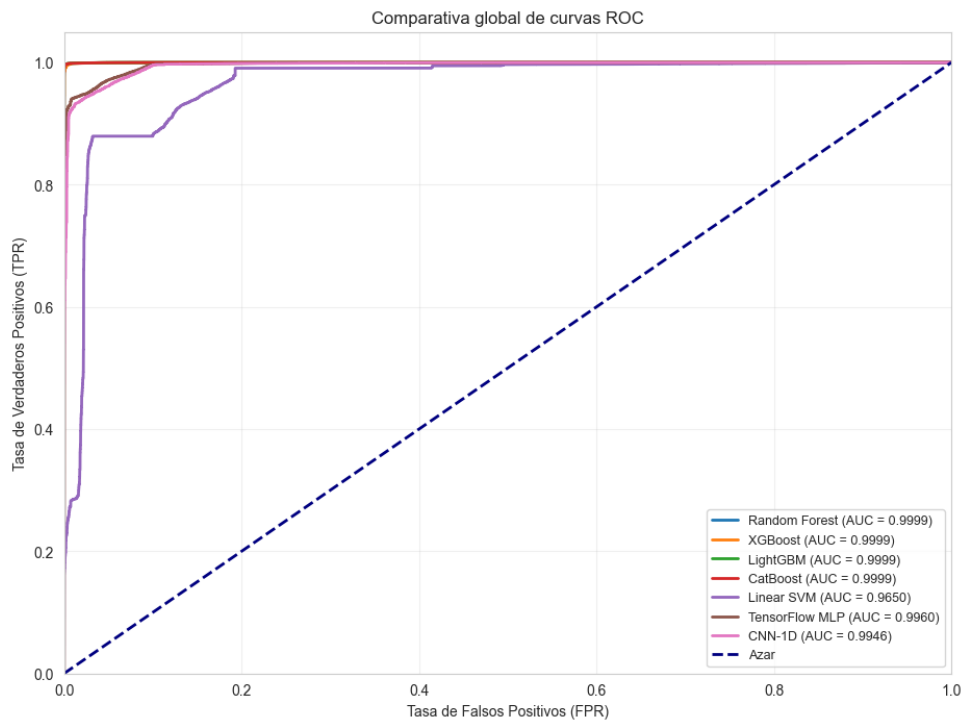


Figura 5.38. Curva ROC - Modelos (ToN-IoT)

Las matrices de confusión de cada modelo se muestran a continuación:

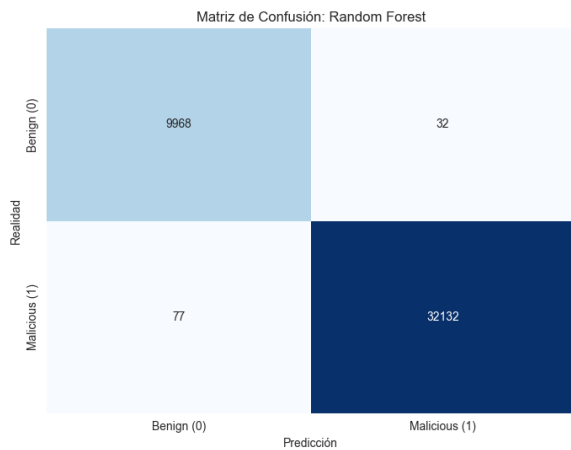


Figura 5.39. MC - Random Forest (ToN-IoT)

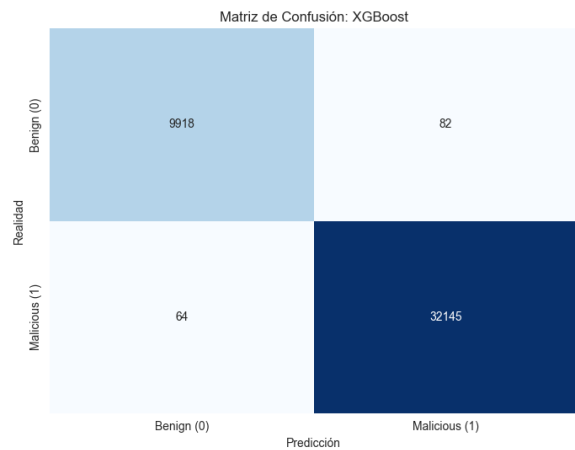


Figura 5.40. MC - XGBoost (ToN-IoT)

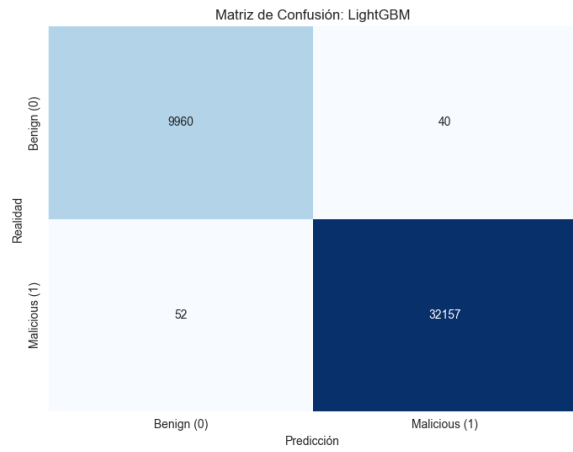


Figura 5.41. MC - LightGBM (ToN-IoT)

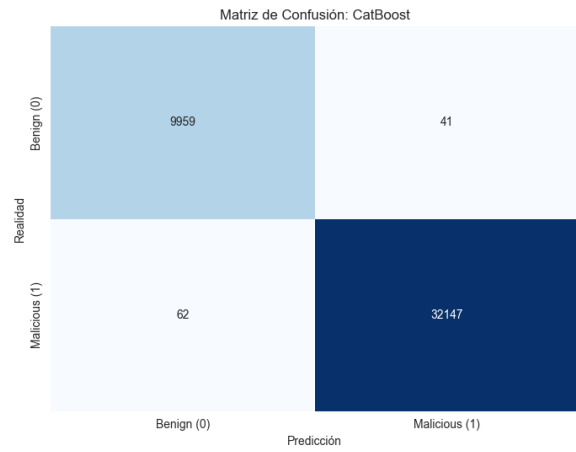


Figura 5.42. MC - CatBoost (ToN-IoT)

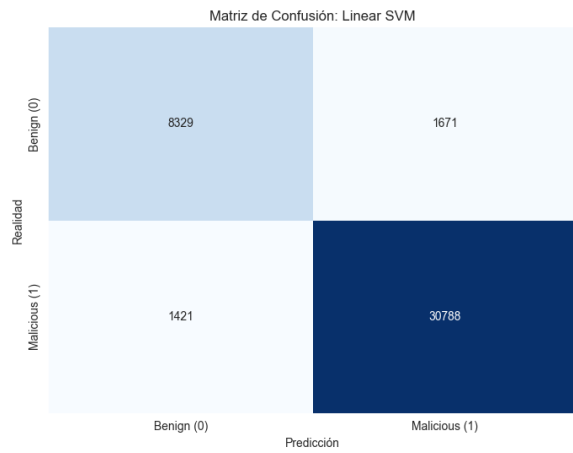


Figura 5.43. MC - SVM (ToN-IoT)

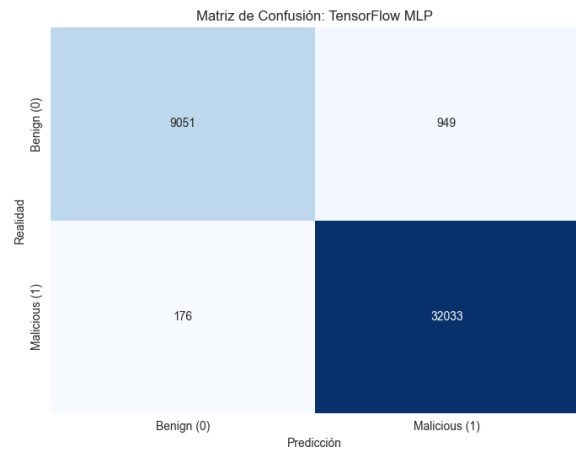


Figura 5.44. MC - MLP (ToN-IoT)

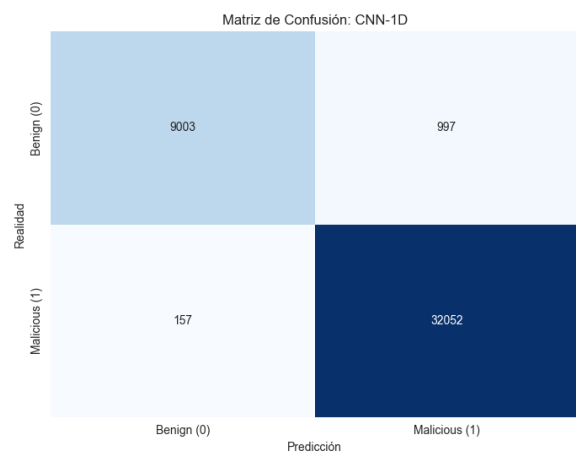


Figura 5.45. MC - CNN (ToN-IoT)

La tabla de los mejores hiperparámetros obtenidos para cada modelo con el dataset ToN-IoT es la siguiente:

Modelo	Hiperparámetros ganadores (ToN-IoT)
Random Forest	n_estimators = 50, max_depth = 18
XGBoost	n_estimators = 50, max_depth = 7
LightGBM	n_estimators = 100, num_leaves = 233, max_depth = 11
CatBoost	n_estimators = 250, max_depth = 9
SVM	C = 2.870875
MLP	n_layers = 3, unidades = [64, 16, 64]
CNN-1D	n_filters = 96, kernel_size = 5, dense_units = 96

Tabla 5.28. Hiperparámetros ganadores de los modelos evaluados con ToN-IoT

5.4 Modelos Evaluados en Local

Para terminar la comparativa entre los modelos es interesante ver el tiempo que tardarían en la fase de inferencia en un entorno más realista, es decir, cargar los modelos en nuestro propio ordenador (no usando un super servidor). Hemos cargado los modelos ganadores entrenados en mi ordenador y hemos calculado el tiempo que tardaría en procesar todos los registros y hacer la predicción.

5.4.1 Resultados UNSW-NB15

Tabla 5.29. Tiempo medio de inferencia y throughput aproximado en máquina local para UNSW-NB15

Modelo	Tiempo medio (s)	Muestras/s aprox.
SVM	0.0155	16600905
CatBoost	0.1572	1639079
LightGBM	0.2044	1260820
RF	0.2130	1209890
MLP	0.2240	1150412
XGBoost	0.2313	1113803
CNN	1.1434	225353

5.4.2 Resultados IOT-23

Tabla 5.30. Tiempo medio de inferencia y throughput aproximado en máquina local para IoT-23

Modelo	Tiempo medio (s)	Muestras/s aprox.
XGBoost	0.2160	5391713
SVM	0.2188	5323551
LightGBM	0.2962	3932579
CatBoost	0.3018	3859140
MLP	0.5940	1961042
RF	1.0131	1149778
CNN	6.2271	187062

5.4.3 Resultados TON-IOT

Tabla 5.31. Tiempo medio de inferencia y throughput aproximado en máquina local para TON-IoT

Modelo	Tiempo medio (s)	Muestras/s aprox.
SVM	0.0413	5113359
XGBoost	0.0810	2605794
CatBoost	0.0912	2313078
RF	0.1506	1401017
LightGBM	0.1676	1259440
MLP	0.2519	837747
CNN	2.1399	98625

En conclusión, los resultados demuestran que la viabilidad de un IDS no solo se mide por su robustez métrica, sino por su eficiencia temporal. Los modelos evaluados exhiben una fase de inferencia altamente competitiva en hardware convencional, lo que valida su aplicabilidad práctica fuera de entornos de supercomputación.

La selección del clasificador óptimo responderá, por tanto, a un compromiso de ingeniería (trade-off) entre los recursos disponibles y los requerimientos de seguridad de cada escenario. Arquitecturas profundas como la CNN ofrecen una tolerancia y resiliencia sobresalientes frente a ataques de evasión adversarios, pero penalizan severamente la latencia del sistema. Por el contrario, los ensambles basados en árboles de decisión se posicionan como las soluciones más equilibradas. La elección final quedará condicionada a si el entorno **prioriza la robustez o la disponibilidad en tiempo real**.

6

Ataques de Evasión

6.1 Evaluación de robustez frente a ataques adversarios

Hasta este punto del trabajo, los modelos de detección de intrusiones desarrollados han mostrado un rendimiento muy elevado sobre los conjuntos de prueba originales. En particular, las arquitecturas profundas, como MLP y CNN-1D, han alcanzado valores de F1-score superiores al 96%, mientras que los modelos basados en ensambles de árboles de decisión, como Random Forest, XGBoost, LightGBM y CatBoost, han obtenido resultados cercanos al 99%. Estos resultados indican que los modelos son capaces de aprender patrones relevantes en los datasets empleados y ofrecen una alta capacidad de clasificación en condiciones normales de evaluación.

Sin embargo, en el ámbito de la ciberseguridad, un buen rendimiento sobre datos estáticos no garantiza por sí solo que un sistema sea robusto en escenarios reales. Los atacantes pueden conocer o inferir el funcionamiento de los sistemas de detección y adaptar su comportamiento para evadirlos. En este contexto surge el **aprendizaje automático adversario**, cuyo objetivo es estudiar cómo pequeñas modificaciones en las entradas pueden provocar errores de clasificación en modelos de inteligencia artificial.

En el caso de los sistemas de detección de intrusiones en red, los ataques de evasión se producen durante la fase de inferencia. En este tipo de ataques, una muestra maliciosa se modifica de forma controlada para intentar que el modelo la clasifique como tráfico benigno.

Para analizar esta problemática, en esta fase del proyecto se evaluará la **robustez adversaria de los modelos finales seleccionados en las etapas anteriores**. Es importante destacar que no se realizará una nueva optimización de hiperparámetros, sino que se partirá de las **configuraciones ya elegidas para cada modelo**. De este modo, la evaluación adversaria se plantea como una prueba adicional de resistencia sobre los modelos previamente considerados como mejores bajo condiciones limpias.

6.1.1 Uso de Adversarial Robustness Toolbox

La herramienta principal empleada para esta evaluación será **Adversarial Robustness Toolbox** (ART), una librería orientada al análisis de seguridad y robustez de modelos de aprendizaje automático. ART

proporciona implementaciones estandarizadas de ataques adversarios utilizados habitualmente en la literatura, como FGSM, BIM o PGD, y permite integrarlos con modelos desarrollados en distintos entornos de aprendizaje automático y aprendizaje profundo.

La evaluación de robustez de los distintos clasificadores NIDS propuestos en este trabajo exige adaptar la estrategia de ataque a la naturaleza matemática subyacente de cada algoritmo. Por este motivo, se ha diseñado una metodología en dos fases, **basada en ataques directos y en la transferencia de vulnerabilidades**.

6.1.1.1 Ataques Empleados

En esta evaluación se han empleado dos ataques de evasión ampliamente reconocidos en la literatura de aprendizaje automático adversario:

- **Fast Gradient Sign Method (FGSM):** Este método genera perturbaciones adversarias a partir del gradiente de la función de pérdida con respecto a las características de entrada. La perturbación se calcula como el signo del gradiente multiplicado por un factor de escala ϵ , que controla la magnitud de la alteración. FGSM es un ataque rápido y eficiente, aunque su efectividad puede ser limitada frente a modelos robustos o con mecanismos de defensa [5].
- **Projected Gradient Descent (PGD):** PGD es una extensión iterativa de FGSM que aplica múltiples pasos de perturbación, proyectando cada iteración dentro de un espacio permitido definido por ϵ . Este método es más potente que FGSM, ya que permite explorar un espacio más amplio de perturbaciones y puede superar defensas basadas en la limitación de la magnitud de las alteraciones [7].

6.1.1.2 Ataques de Caja Blanca basados en Gradiente (MLP y CNN-1D)

Los algoritmos de evasión empleados en este estudio, como *Projected Gradient Descent* (PGD) o *Fast Gradient Sign Method* (FGSM), requieren de un requisito matemático indispensable: **el modelo objetivo debe ser continuo y diferenciable**.

En arquitecturas de aprendizaje profundo como el Perceptrón Multicapa (MLP) y las Redes Neuronales Convolucionales (CNN-1D), el atacante emplea la retropropagación (*backpropagation*) para calcular el gradiente de la función de pérdida con respecto a las características de entrada. Esto permite identificar la dirección matemática exacta para alterar el tráfico de red original e inducir a una clasificación errónea (falsos negativos).

6.1.1.3 Transferencia Adversaria para Modelos Discretos (Ensamblados y SVM)

A diferencia de las redes neuronales, los modelos que fundamentan sus decisiones en particiones del espacio paramétrico, como los ensambles de árboles de decisión (Random Forest, XGBoost, LightGBM,

CatBoost) o las Máquinas de Vectores de Soporte (SVM), presentan fronteras lógicas abruptas que inutilizan la aplicación directa del cálculo de gradientes.

Para solventar esta limitación e incorporar estos algoritmos a la evaluación de seguridad, se explota la técnica de **Transferencia Adversaria** (*Adversarial Transferability*) [4]. Esta estrategia asume que las perturbaciones diseñadas para engañar a un modelo específico suelen evadir con éxito clasificadores con arquitecturas completamente distintas. Es decir, el veneno que fabricas para matar a un modelo específico, suele ser letal también para otros modelos que no tienen nada que ver con el primero. Por ejemplo, un ataque diseñado para engañar a un MLP podría transferirse con éxito a un Random Forest o a una SVM, incluso sin acceso directo a sus parámetros internos.

El procedimiento aplicado es el siguiente:

1. **Generación (White-box):** Los modelos diferenciables (MLP y CNN-1D) actúan como modelos sustitutos (*surrogate models*) para la generación algorítmica del nuevo conjunto de tráfico de red evadido.
2. **Evaluación Cruzada (Black-box):** El *Dataset Adversario* se introduce directamente en la fase de inferencia de los clasificadores basados en árboles y SVM.
3. **Cuantificación del Impacto:** Se mide la degradación de las métricas de rendimiento frente al estado basal.

6.1.2 Análisis de Resultados en UNSW-NB15

En esta sección se presentarán los resultados obtenidos tras la aplicación de los ataques de evasión descritos en la sección anterior sobre los modelos ganadores de la sección anterior y con su dataset correspondiente.

El procedimiento que se siguió fue entrenar cada modelo nuevamente con la configuración de hiperparámetros seleccionada en la fase anterior con su dataset correspondiente. Posteriormente, se aplicaron los ataques de evasión para generar un nuevo conjunto de datos adversarios sobre los modelos MLP y CNN-1D (transferencia adversaria).

Es fundamental destacar que estos ataques se han configurado con un límite de perturbación extremadamente restrictivo ($\epsilon = 0.05$). Esto implica que el algoritmo de evasión solo ha modificado, como máximo, un 5% el valor original de las características numéricas continuas, garantizando que el tráfico malicioso generado sea prácticamente indistinguible del tráfico original a nivel de red.

Finalmente, los datos adversarios generados se introdujeron en la fase de inferencia de los distintos clasificadores para extraer sus respectivas métricas de rendimiento. El análisis se fundamentó en métricas estándar como el Accuracy, el F1-Score y el Recall, priorizando en todo momento la **evaluación sobre la clase maliciosa**. Esta decisión metodológica se basa en que el objetivo crítico del estudio es medir la degradación en la capacidad de detección de ataques del sistema.

6.1.2.1 Resultados PGD

Presentamos los resultados de la aplicación del ataque PGD sobre los modelos entrenados con el dataset UNSW-NB15. En los gráficos de barras podemos ver la métrica que corresponde a cada modelo, tanto en su estado original (barra azul), tras la aplicación del ataque con el dataset alterado con MLP (barra naranja) y tras la aplicación del ataque con el dataset alterado con CNN-1D (barra verde).

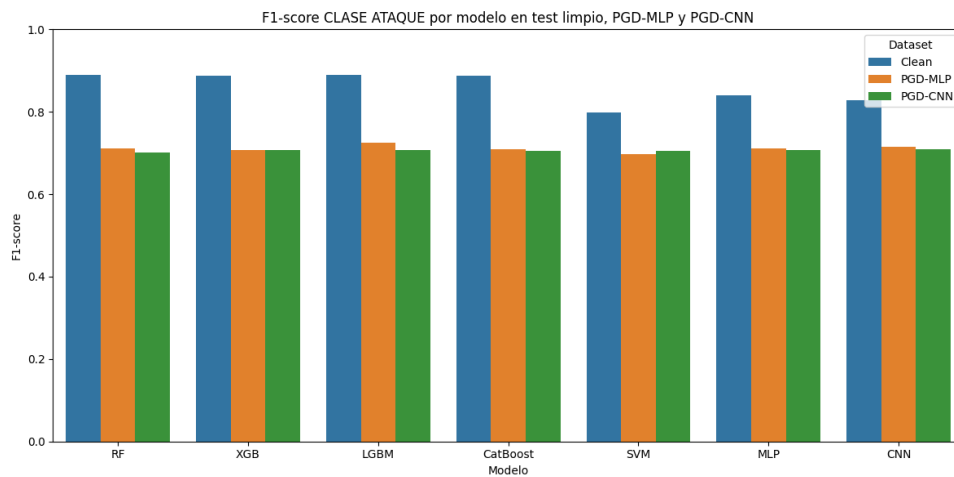


Figura 6.1. F1-PGD-UNSW-NB15

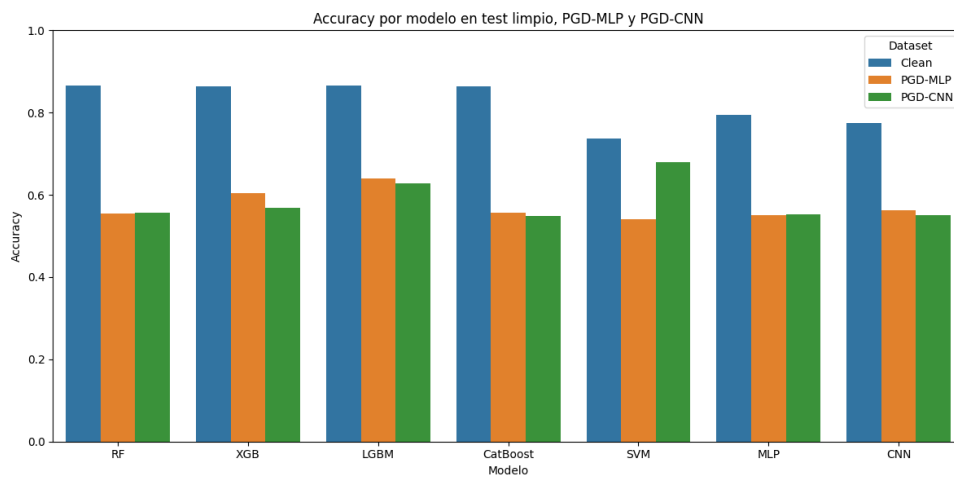


Figura 6.2. Accuracy-PGD-UNSW-NB15

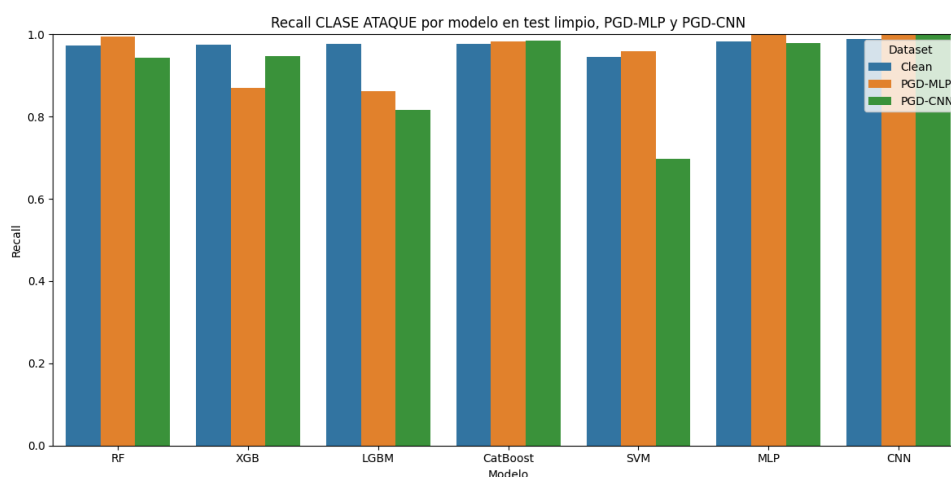


Figura 6.3. Recall-PGD-UNSW-NB15

Podemos concluir que, incluso con un límite de perturbación moderado de $\epsilon = 0.05$, el ataque PGD consigue degradar el rendimiento de los modelos respecto al test limpio. Esta caída se observa especialmente en la **accuracy** y en el **F1-score de la clase ataque**, mientras que el *recall* se mantiene elevado en varios modelos.

Este comportamiento indica que los modelos siguen detectando una proporción alta de ataques, pero pierden capacidad para clasificar correctamente el conjunto completo de muestras. En otras palabras, la perturbación afecta al equilibrio general del clasificador, aunque no siempre reduzca de forma drástica la detección de ataques.

Como el *recall* de la clase ataque sigue siendo alto en muchos casos, pero el F1-score disminuye, es probable que la precisión de dicha clase se vea reducida. Esto sugiere que algunos modelos tienden a clasificar un mayor número de muestras como ataque, aumentando así los falsos positivos.

Por último, las perturbaciones generadas sobre los modelos MLP y CNN presentan un impacto similar en términos de F1-score, lo que evidencia nuestra hipótesis sobre la transferencia adversaria de los ataques hacia modelos clásicos como RF, XGB, LGBM, CatBoost y SVM.

6.1.2.2 Resultados FGSM

Estos son los resultados de la aplicación del ataque FGSM:

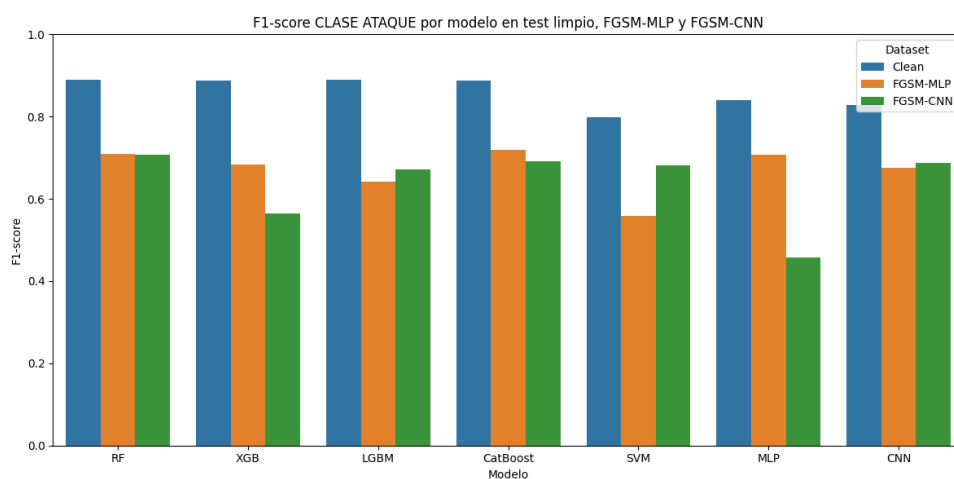


Figura 6.4. F1-FGSM-UNSW-NB15

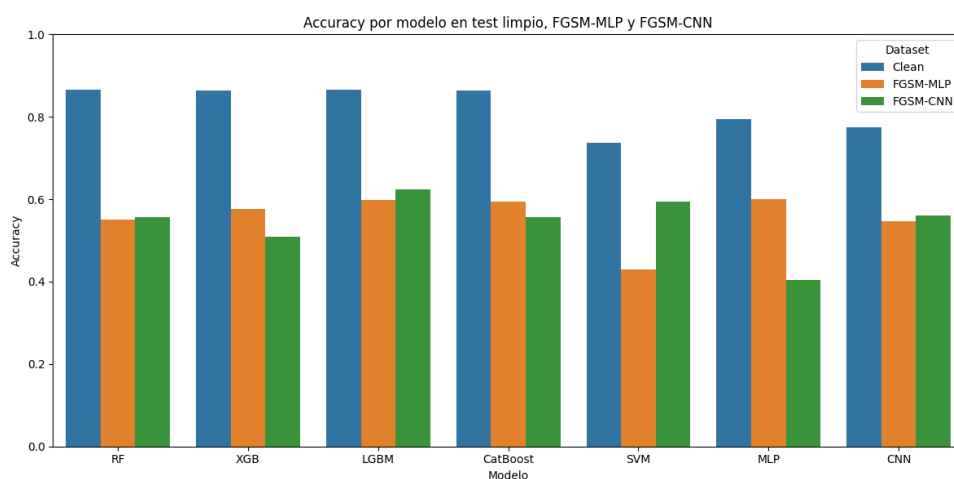


Figura 6.5. Accuracy-FGSM-UNSW-NB15

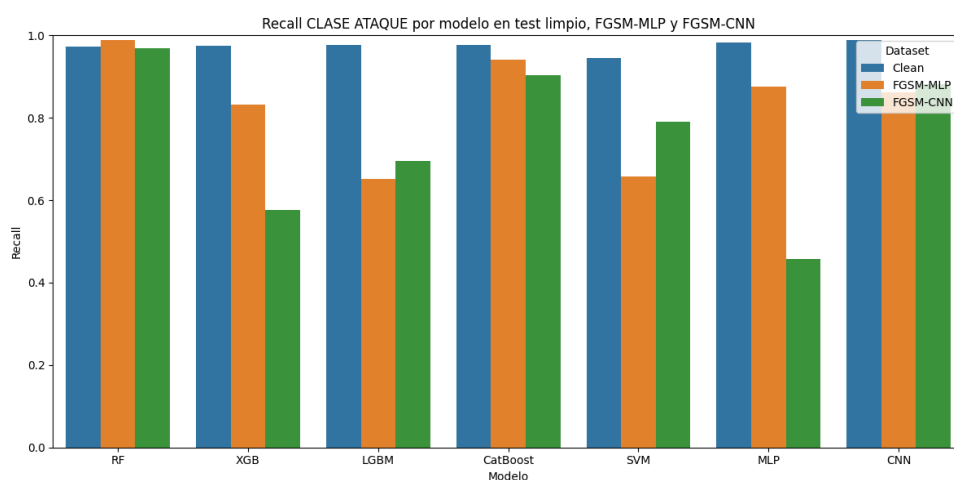


Figura 6.6. Recall-FGSM-UNSW-NB15

En el caso del ataque FGSM también se observa una degradación clara del rendimiento respecto al

test limpio. Esta caída aparece tanto en la **accuracy** como en el **F1-score de la clase ataque**, aunque el impacto varía según el modelo evaluado y según si la perturbación se genera a partir del MLP o de la CNN.

En términos de *accuracy*, todos los modelos reducen su rendimiento respecto al escenario limpio. Esta caída indica que FGSM afecta a la capacidad global de clasificación, provocando un mayor número de errores sobre el conjunto de test. En algunos modelos, como MLP frente a FGSM-CNN o SVM frente a FGSM-MLP, la reducción es especialmente notable.

Respecto al F1-score de la clase ataque, se aprecia que los modelos mantienen valores moderados en varios casos, pero siempre por debajo del rendimiento limpio. Esto indica que el ataque afecta al equilibrio entre precisión y recall de la clase ataque. En particular, algunos modelos como XGB, SVM o MLP muestran una mayor sensibilidad dependiendo del modelo utilizado para generar la perturbación.

El *recall* de la clase ataque muestra un comportamiento más desigual. Algunos modelos, como RF, CatBoost y CNN, mantienen una alta capacidad de detección de ataques incluso bajo perturbación FGSM. Sin embargo, otros modelos presentan caídas importantes, especialmente XGB, LGBM, SVM y MLP en determinados escenarios.

6.1.3 Análisis de Resultados en IOT-23

6.1.3.1 Resultados PGD

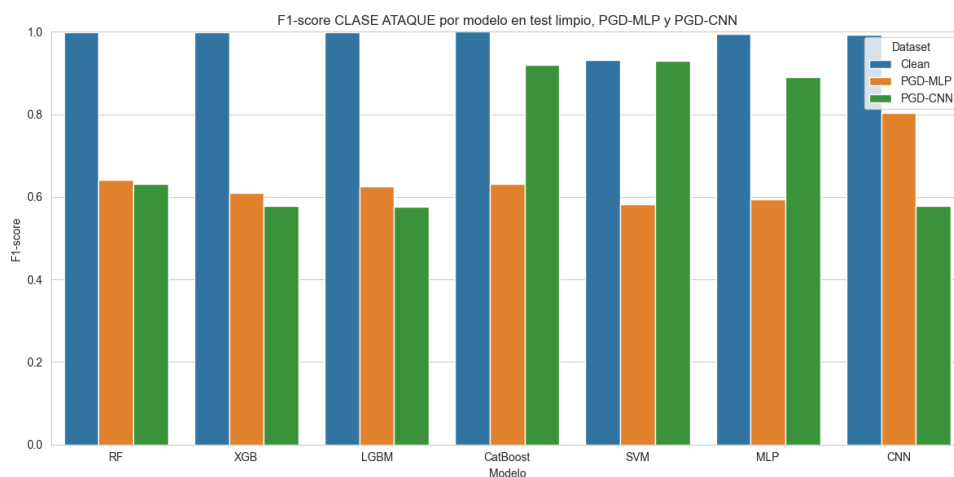


Figura 6.7. F1-PGD-IOT-23

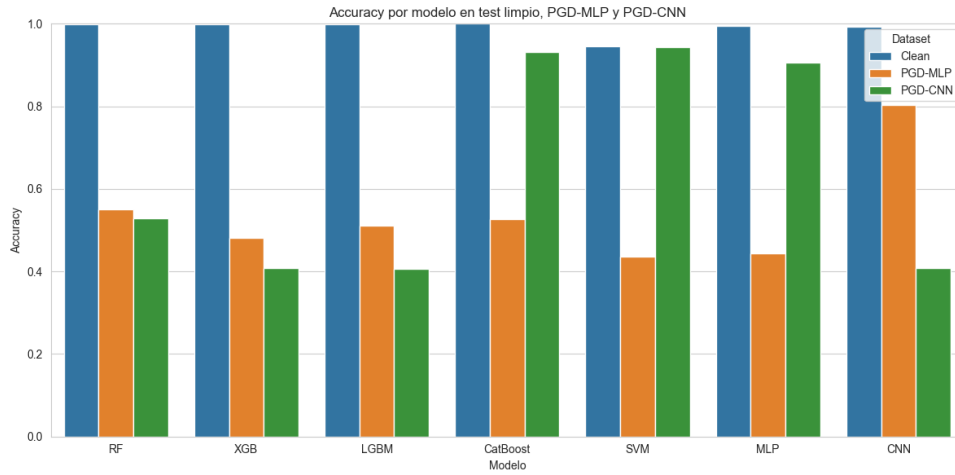


Figura 6.8. Accuracy-PGD-IOT-23

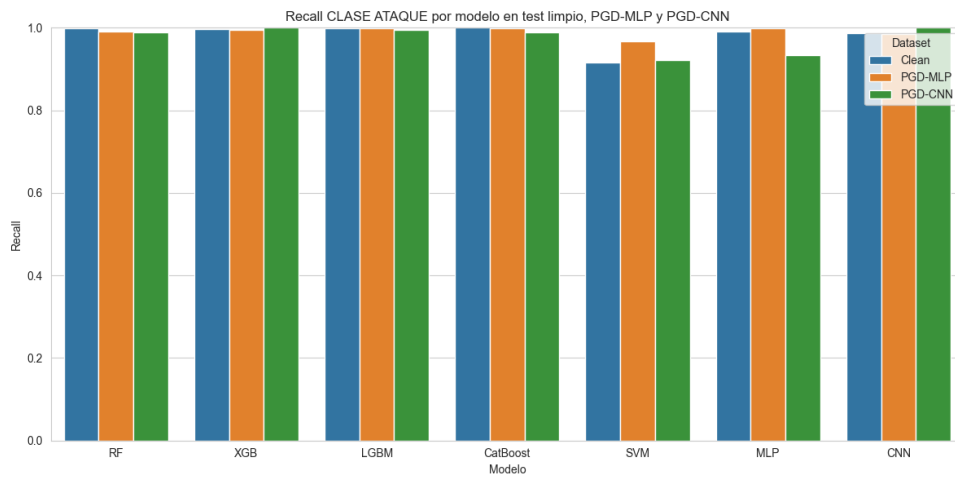


Figura 6.9. Recall-PGD-IOT-23

En el dataset IoT-23, para el ataque PGD con límite de perturbación $\epsilon = 0.05$, se observa una degradación notable del rendimiento respecto al test limpio, especialmente en términos de **accuracy** y **F1-score de la clase ataque**. Aunque los modelos presentan un rendimiento casi perfecto en el escenario limpio, las perturbaciones adversarias reducen de forma clara la capacidad global de clasificación.

Sin embargo, el *recall* de la clase ataque se mantiene muy elevado en prácticamente todos los modelos, incluso tras aplicar las perturbaciones PGD generadas sobre MLP y CNN. Esto indica que los modelos siguen detectando la mayoría de las muestras maliciosas, por lo que el ataque no provoca principalmente una pérdida de detección de ataques.

La diferencia entre el alto *recall* y la caída observada en accuracy y F1-score sugiere que el principal efecto del ataque es un aumento de falsos positivos. Es decir, los modelos tienden a clasificar más muestras normales como ataques, manteniendo una alta detección de tráfico malicioso, pero reduciendo su capacidad para distinguir correctamente entre tráfico benigno y malicioso.

Además, se observa que las perturbaciones generadas sobre MLP y CNN presentan capacidad de transferencia hacia modelos clásicos, aunque con diferencias entre clasificadores. En particular, PGD-MLP provoca una caída importante en modelos como RF, XGB, LGBM, CatBoost, SVM y MLP, mientras que PGD-CNN afecta de forma muy acusada a XGB, LGBM y CNN, pero tiene menor impacto sobre CatBoost, SVM y MLP.

6.1.3.2 Resultados FGSM

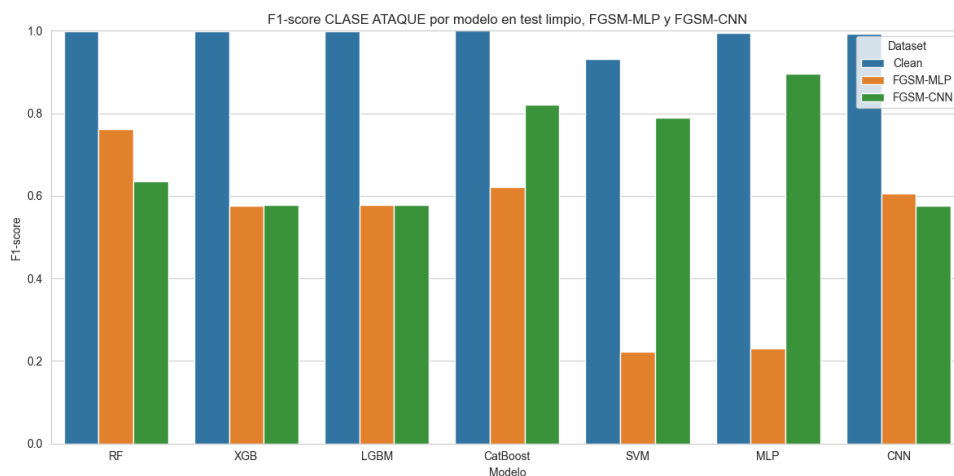


Figura 6.10. F1-FGSM-IOT-23

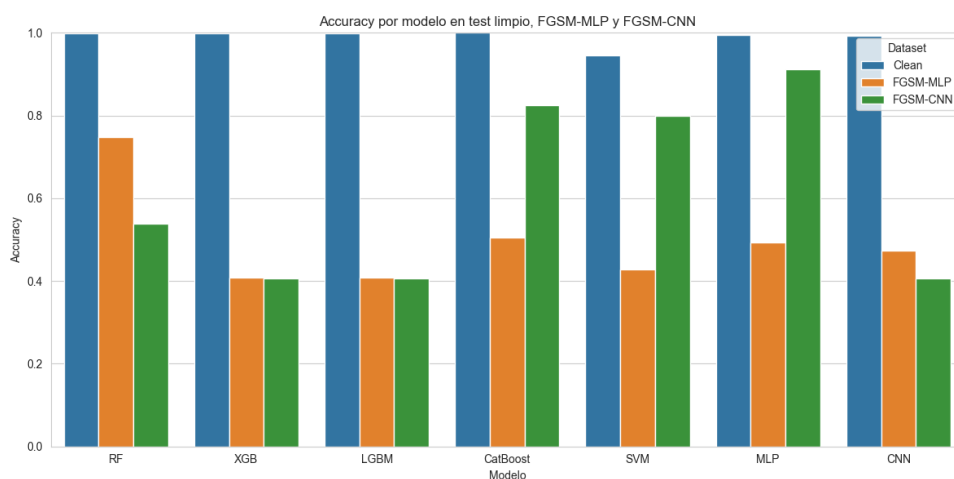


Figura 6.11. Accuracy-FGSM-IOT-23

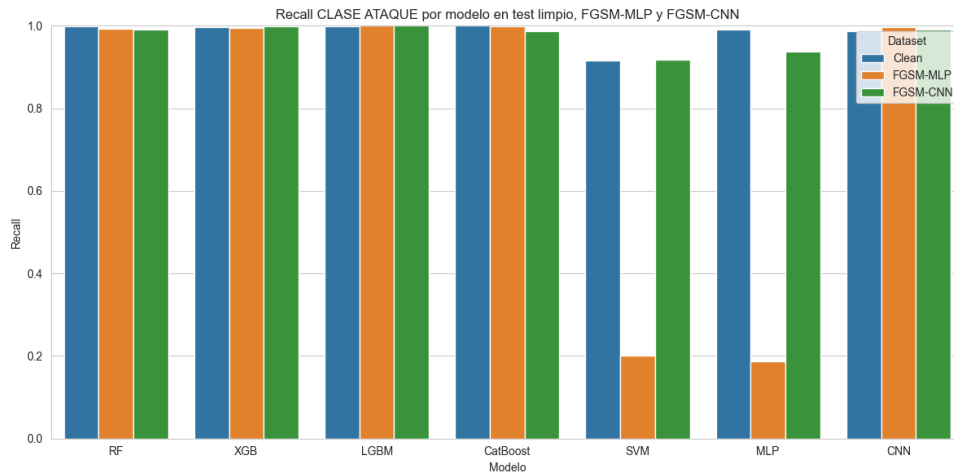


Figura 6.12. Recall-FGSM-IOT-23

Aquí observamos una degradación notable del rendimiento respecto al test limpio. Esta caída se aprecia principalmente en la **accuracy** y en el **F1-score de la clase ataque**.

Respecto al F1-score de la clase ataque, se observa una reducción clara frente al escenario limpio. El caso más llamativo se produce en SVM y MLP bajo FGSM-MLP, donde el F1-score cae de forma muy pronunciada. En cambio, bajo FGSM-CNN, modelos como CatBoost, SVM y MLP mantienen valores considerablemente más altos, lo que evidencia que el efecto del ataque depende tanto del modelo atacado como del modelo usado para generar la perturbación.

El *recall* de la clase ataque se mantiene prácticamente perfecto en la mayoría de modelos, especialmente en RF, XGB, LGBM, CatBoost y CNN. Sin embargo, aparecen caídas muy fuertes en SVM y MLP cuando las perturbaciones se generan sobre MLP. Esto indica que, en estos casos concretos, FGSM no solo incrementa los falsos positivos, sino que también provoca que una parte importante de los ataques deje de ser detectada correctamente.

6.1.4 Análisis de Resultados en TON-IOT

6.1.4.1 Resultados PGD

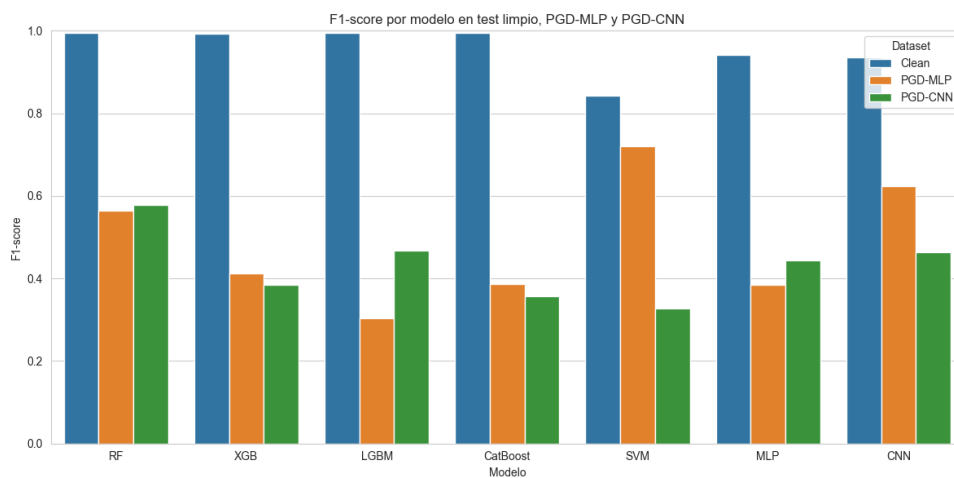


Figura 6.13. F1-PGD-TON-IOT

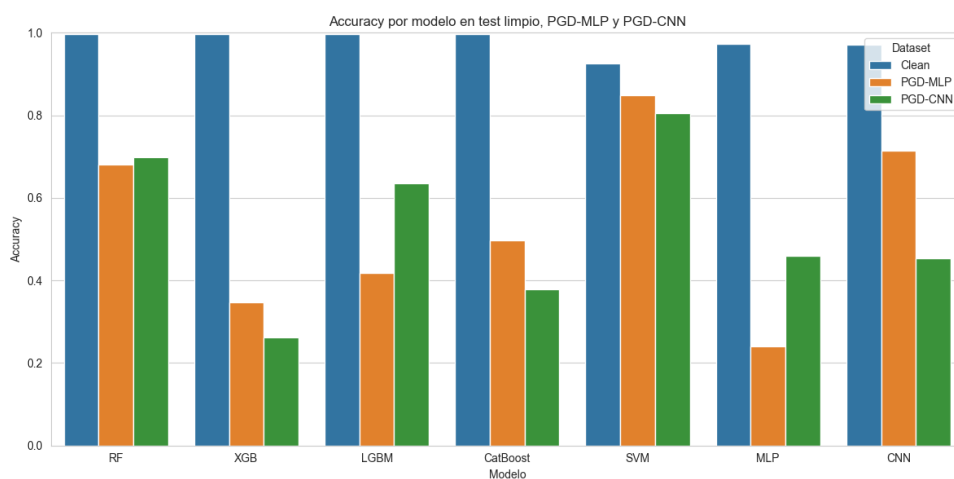


Figura 6.14. Accuracy-PGD-TON-IOT

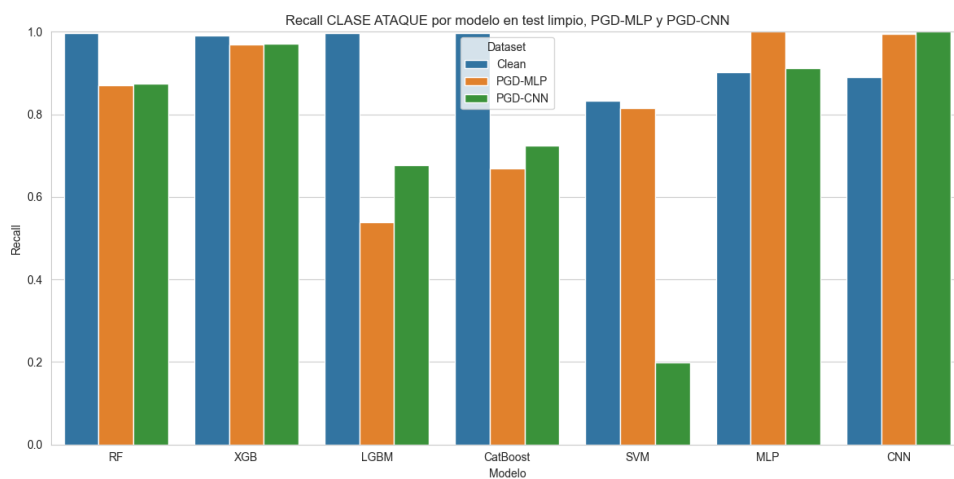


Figura 6.15. Recall-PGD-TON-IOT

En el dataset TON-IoT, para el ataque PGD, se observa una degradación significativa del rendimiento respecto al test limpio, donde la mayoría de modelos presentan valores muy elevados de accuracy y F1-score, cercanos a 1. Sin embargo, tras aplicar las perturbaciones PGD, ambos indicadores descienden de forma clara en la mayoría de clasificadores.

En términos de **accuracy**, los modelos más afectados son XGB, LGBM, CatBoost y MLP.

Respecto al **F1-score**, la degradación es todavía más evidente. Modelos como LGBM, CatBoost y MLP sufren caídas pronunciadas, lo que indica que el ataque afecta de forma importante al equilibrio entre precisión y recall de la clase ataque.

El *recall* de la clase ataque presenta un comportamiento desigual. En algunos modelos, como XGB, MLP y CNN, se mantiene elevado bajo determinadas perturbaciones, lo que indica que siguen detectando gran parte de los ataques. Sin embargo, en otros casos se producen caídas importantes, especialmente en LGBM con PGD-MLP, CatBoost con PGD-MLP y SVM con PGD-CNN.

En conjunto, los resultados muestran que PGD con $\epsilon = 0.05$ tiene un impacto relevante sobre TON-IoT. A diferencia de otros datasets donde el recall se mantenía alto en casi todos los modelos, aquí el ataque también reduce la capacidad de detección de ataques en determinados clasificadores. Esto sugiere que TON-IoT presenta una mayor sensibilidad frente a perturbaciones adversarias, especialmente en modelos como LGBM, CatBoost, MLP y SVM dependiendo del modelo generador de la perturbación.

6.1.4.2 Resultados FGSM

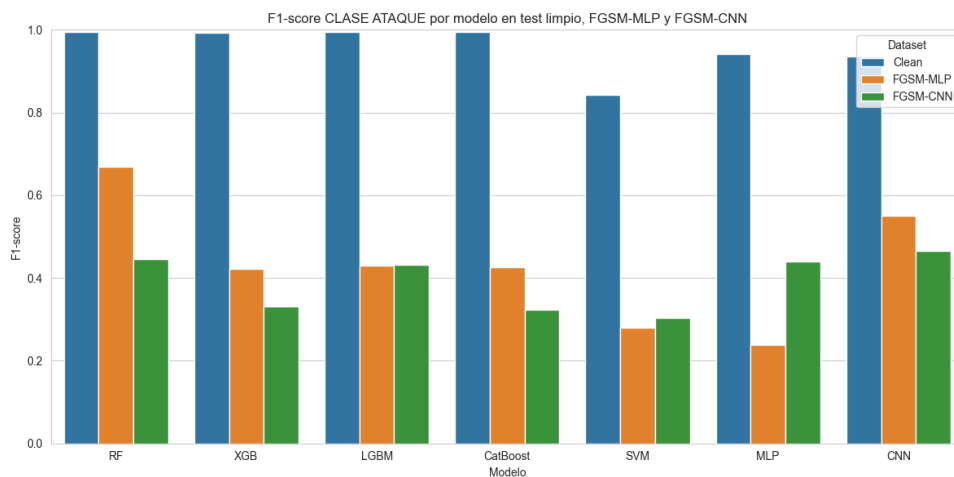


Figura 6.16. F1-FGSM-TON-IOT

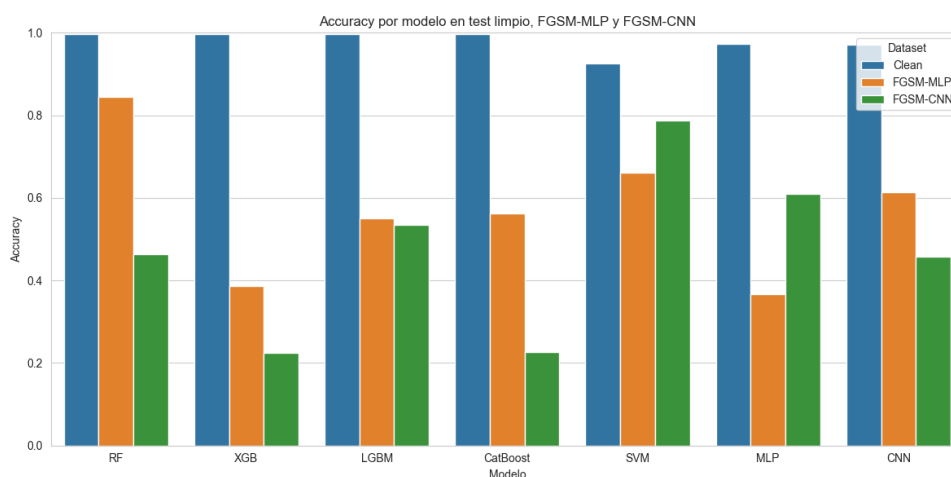


Figura 6.17. Accuracy-FGSM-TON-IOT

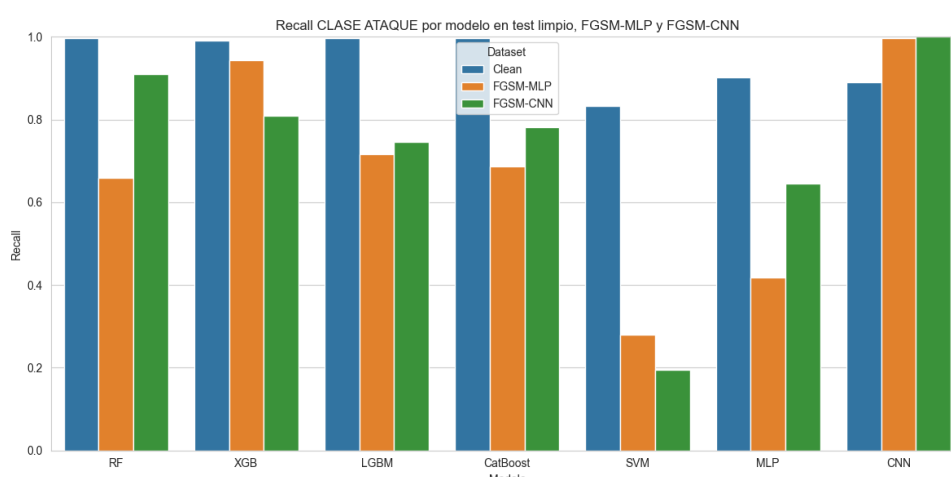


Figura 6.18. Recall-FGSM-TON-IOT

Finalmente, para el ataque FGSM, en el escenario limpio, la mayoría de modelos presentan valores muy elevados de accuracy y F1-score, cercanos a 1. Sin embargo, tras aplicar las perturbaciones FGSM, ambos indicadores descienden de forma clara en prácticamente todos los clasificadores.

En términos de **accuracy**, la caída es especialmente acusada en XGB, CatBoost y CNN cuando las perturbaciones se generan sobre CNN. También se observa una degradación importante con FGSM-MLP, especialmente en XGB, LGBM, CatBoost y MLP.

Respecto al **F1-score de la clase ataque**, la degradación es muy notable en todos los modelos.

El *recall* de la clase ataque presenta un comportamiento desigual. Algunos modelos, como CNN, mantienen un recall muy elevado e incluso cercano a 1 bajo perturbación, lo que indica que siguen detectando la mayoría de ataques. Sin embargo, otros modelos sufren caídas importantes, como SVM y MLP.

En resumen, hemos visto como un límite de perturbación de ataque pequeño, como 0.05, puede generar

caídas bastante grandes en las métricas analizadas, siempre dependiendo del modelo y del dataset perturbado que se aplique.

6.2 Análisis Límite de Perturbación

Los datasets perturbados con PGD y FGSM se generaron con un límite de perturbación de $\epsilon = 0.05$. Sin embargo, puede ser interesante analizar cómo varía la robustez de los modelos a medida que se incrementa el límite de perturbación. Para ello, se ha realizado un análisis adicional sobre los datasets probando diferentes valores de ϵ : 0.01, 0.03, 0.05, 0.075, 0.1, 0.2 y 0.5, viendo su F1-Score asociado a la clase de ataque. De esta forma, se puede observar la relación entre la magnitud de la perturbación y la degradación del F1-Score de los modelos.

La idea es probar estos diferentes valores de límites de perturbación y ver a partir de qué valor los modelos bajan su F1-Score por debajo de un umbral, en este caso va a ser 0.5, que es un valor bastante bajo y que indicaría que el modelo ya no es capaz de detectar correctamente la clase maliciosa.

6.2.1 Resultados UNSW-NB15

6.2.1.1 FGSM

Tabla 6.1. F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo MLP en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7290	0.7208	0.6652	0.7136	0.6677	0.7111	0.7294
0.030	0.7071	0.7202	0.6420	0.7091	0.6219	0.6851	0.6848
0.050	0.7082	0.6841	0.6411	0.7191	0.5590	0.7071	0.6765
0.075	0.7094	0.6774	0.6250	0.7020	0.4503	0.7080	0.6710
0.100	0.7125	0.6840	0.6015	0.6889	0.4097	0.7090	0.6680
0.200	0.7062	0.6608	0.5387	0.7181	0.3791	0.7097	0.6759
0.500	0.7112	0.6626	0.5239	0.7383	0.3811	0.7085	0.6995

Tabla 6.2. F1-score de la clase ataque ante perturbaciones PGD generadas sobre el modelo CNN en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7255	0.7287	0.6826	0.6985	0.7431	0.7321	0.7156
0.030	0.7013	0.6789	0.6688	0.7076	0.7201	0.4889	0.7034
0.050	0.7067	0.5639	0.6706	0.6913	0.6822	0.4581	0.6882
0.075	0.7178	0.6979	0.6710	0.6299	0.6418	0.4983	0.6846
0.100	0.7194	0.6727	0.6745	0.6496	0.6396	0.5328	0.6847
0.200	0.7159	0.6564	0.6409	0.7293	0.6441	0.5298	0.6841
0.500	0.6806	0.6410	0.6032	0.7404	0.6424	0.5378	0.6847

6.2.1.2 PGD

Tabla 6.3. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7210	0.6873	0.7124	0.6579	0.7046	0.7165	0.7772
0.030	0.7196	0.7019	0.6780	0.6922	0.7054	0.7110	0.7162
0.050	0.7113	0.7075	0.7251	0.7095	0.6965	0.7103	0.7157
0.075	0.7093	0.7073	0.6508	0.6874	0.7105	0.7102	0.7103
0.100	0.7084	0.7000	0.6101	0.7091	0.7145	0.7102	0.7100
0.200	0.7089	0.6983	0.7188	0.7037	0.7102	0.7102	0.7083
0.500	0.7098	0.7087	0.4985	0.7102	0.7102	0.7102	0.7137

Tabla 6.4. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN en UNSW-NB15

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.8892	0.8876	0.8892	0.8873	0.7976	0.8401	0.8285
0.010	0.7247	0.7199	0.7669	0.6825	0.7444	0.7384	0.7156
0.030	0.6778	0.7015	0.7192	0.6880	0.6099	0.7157	0.7150
0.050	0.7013	0.7071	0.7073	0.7060	0.7053	0.7068	0.7102
0.075	0.7133	0.7124	0.7191	0.7003	0.6629	0.7126	0.7102
0.100	0.7160	0.6833	0.7201	0.7110	0.7142	0.6717	0.7102
0.200	0.7215	0.7111	0.7302	0.7036	0.6758	0.6134	0.7107
0.500	0.6781	0.7310	0.7148	0.6915	0.6761	0.6159	0.6918

6.2.2 Resultados IOT-23

6.2.2.1 FGSM

Tabla 6.5. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6736	0.6092	0.5789	0.5784	0.9437	0.8052	0.9484
0.030	0.7356	0.5764	0.5790	0.5834	0.7617	0.5243	0.7623
0.050	0.7620	0.5771	0.5786	0.6219	0.2217	0.2314	0.6059
0.075	0.7619	0.5766	0.5786	0.6182	0.2203	0.1808	0.5887
0.100	0.7793	0.5768	0.5789	0.6119	0.2197	0.1786	0.6225
0.200	0.5790	0.6088	0.6097	0.5916	0.2155	0.1782	0.5888
0.500	0.5781	0.6085	0.6096	0.5916	0.2155	0.1782	0.5829

Tabla 6.6. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6318	0.6104	0.5769	0.5786	0.8853	0.8944	0.9346
0.030	0.6167	0.5778	0.5769	0.6350	0.7888	0.8942	0.7642
0.050	0.6355	0.5777	0.5782	0.8217	0.7888	0.8958	0.5756
0.075	0.6541	0.5775	0.5782	0.8072	0.7634	0.8832	0.6223
0.100	0.6234	0.5769	0.5776	0.7083	0.7605	0.8711	0.6581
0.200	0.5826	0.6267	0.6268	0.5784	0.7602	0.8703	0.6012
0.500	0.5717	0.6194	0.6200	0.5783	0.7611	0.8588	0.5685

6.2.2.2 PGD

Tabla 6.7. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6736	0.6092	0.5789	0.5784	0.9437	0.8052	0.9484
0.030	0.7356	0.5764	0.5790	0.5834	0.7617	0.5243	0.7623
0.050	0.7620	0.5771	0.5786	0.6219	0.2217	0.2314	0.6059
0.075	0.7619	0.5766	0.5786	0.6182	0.2203	0.1808	0.5887
0.100	0.7793	0.5768	0.5789	0.6119	0.2197	0.1786	0.6225
0.200	0.5790	0.6088	0.6097	0.5916	0.2155	0.1782	0.5888
0.500	0.5781	0.6085	0.6096	0.5916	0.2155	0.1782	0.5829

Tabla 6.8. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9996	0.9987	0.9998	1.0000	0.9324	0.9941	0.9941
0.010	0.6318	0.6104	0.5769	0.5786	0.8853	0.8944	0.9346
0.030	0.6167	0.5778	0.5769	0.6350	0.7888	0.8942	0.7642
0.050	0.6355	0.5777	0.5782	0.8217	0.7888	0.8958	0.5756
0.075	0.6541	0.5775	0.5782	0.8072	0.7634	0.8832	0.6223
0.100	0.6234	0.5769	0.5776	0.7083	0.7605	0.8711	0.6581
0.200	0.5826	0.6267	0.6268	0.5784	0.7602	0.8703	0.6012
0.500	0.5717	0.6194	0.6200	0.5783	0.7611	0.8588	0.5685

6.2.3 Resultados TON-IOT

6.2.3.1 FGSM

Tabla 6.9. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.6799	0.5647	0.6577	0.3210	0.4029	0.3369	0.8454
0.030	0.6425	0.5620	0.6391	0.4003	0.3612	0.2857	0.6518
0.050	0.6690	0.5517	0.5496	0.4257	0.2852	0.2861	0.6237
0.075	0.6807	0.5510	0.6003	0.4358	0.2329	0.2867	0.6239
0.100	0.6630	0.5453	0.6004	0.4570	0.2034	0.3372	0.5768
0.200	0.5255	0.5568	0.5666	0.3917	0.2041	0.3531	0.7194
0.500	0.5082	0.5327	0.6206	0.3025	0.2044	0.2107	0.6785

Tabla 6.10. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.5548	0.6436	0.8253	0.3025	0.3217	0.4965	0.7411
0.030	0.4435	0.6053	0.7713	0.3184	0.3209	0.4768	0.6186
0.050	0.4459	0.6052	0.7218	0.3238	0.3066	0.4412	0.5818
0.075	0.4469	0.6118	0.7506	0.3308	0.2954	0.4406	0.5817
0.100	0.4510	0.6099	0.7504	0.3363	0.2888	0.4637	0.5781
0.200	0.4571	0.6002	0.6939	0.3458	0.2204	0.4591	0.5447
0.500	0.4643	0.6905	0.6761	0.3386	0.2178	0.3854	0.6591

6.2.3.2 PGD

Tabla 6.11. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo MLP

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.7322	0.4618	0.6258	0.4997	0.7548	0.3946	0.9192
0.030	0.5675	0.4086	0.5527	0.2887	0.7922	0.4104	0.6605
0.050	0.5725	0.3994	0.5433	0.3501	0.7608	0.3850	0.6488
0.075	0.5221	0.3537	0.6107	0.3290	0.6546	0.3831	0.6317
0.100	0.4931	0.3531	0.5877	0.3367	0.5737	0.3830	0.6114
0.200	0.4590	0.3066	0.5526	0.2307	0.4415	0.3826	0.4588
0.500	0.3671	0.3278	0.4360	0.1261	0.3610	0.3826	0.0817

Tabla 6.12. F1-score de la clase ataque ante perturbaciones generadas sobre el modelo CNN

Límite de perturbación	RF	XGB	LGBM	CatBoost	SVM	MLP	CNN
0	0.9946	0.9927	0.9955	0.9949	0.8434	0.9423	0.9361
0.010	0.5529	0.5838	0.7368	0.3758	0.3273	0.5400	0.7410
0.030	0.5078	0.6789	0.7228	0.3322	0.3208	0.4126	0.6080
0.050	0.5977	0.4666	0.6414	0.4006	0.4115	0.4366	0.5804
0.075	0.4611	0.4123	0.5110	0.3748	0.4057	0.4201	0.5501
0.100	0.5492	0.4199	0.4765	0.4390	0.4901	0.4025	0.5360
0.200	0.4576	0.4317	0.4569	0.3485	0.4224	0.3734	0.5026
0.500	0.4910	0.3701	0.4295	0.4782	0.6064	0.3919	0.4413

6.2.4 Conclusiones del análisis del límite de perturbación

A partir de los resultados obtenidos, se observa que el incremento del límite de perturbación no produce siempre una degradación estrictamente monótona del F1-score. Aunque cabría esperar que valores mayores de ϵ provocasen una caída progresiva del rendimiento, en varios modelos aparecen comportamientos irregulares, con recuperaciones parciales del F1-score para perturbaciones superiores. Este fenómeno es especialmente relevante en datos tabulares, donde perturbaciones elevadas pueden desplazar las muestras fuera de la distribución original del dataset o producir efectos de saturación tras el recorte de valores.

En el caso de UNSW-NB15, la degradación es moderada. Algunos modelos mantienen valores de F1-score superiores a 0.5 incluso para perturbaciones elevadas, lo que indica una cierta estabilidad frente a los ataques evaluados.

En IoT-23, los resultados muestran una caída muy acusada respecto al escenario limpio, especialmente porque los modelos parten de valores de F1-score cercanos a 1. En este dataset, algunos modelos como SVM y MLP sufren degradaciones importantes con determinados ataques, mientras que otros

mantienen valores más elevados. Este comportamiento sugiere que, aunque los modelos alcanzan un rendimiento muy alto en condiciones normales, pueden ser sensibles a pequeñas perturbaciones adversarias.

Por último, TON-IoT es el dataset donde se observa una degradación más severa y generalizada. En varios modelos, el F1-score cae por debajo del umbral de 0.5 incluso con valores bajos de ϵ , especialmente bajo ataques PGD. Esto indica que las muestras de este dataset son más sensibles a perturbaciones adversarias.

En conjunto, este análisis permite concluir que la robustez adversaria no puede evaluarse únicamente a partir del rendimiento en test limpio. Modelos con valores elevados de F1-score en condiciones normales pueden sufrir caídas importantes cuando se aplican perturbaciones adversarias. Además, el impacto del ataque depende del dataset, del tipo de ataque, del límite de perturbación y del modelo generador utilizado para construir las muestras adversarias.

7

Conclusiones y Líneas Futuras

7.1 Conclusiones

7.2 Líneas Futuras

Bibliografía

- [1] Takuya Akiba et al. "Optuna: A Next-Generation Hyperparameter Optimization Framework". In: *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631.
- [2] Ayesha Alharthi, Meera Alaryani, and Sanaa Kaddoura. "A Comparative Study of Machine Learning and Deep Learning Models in Binary and Multiclass Classification for Intrusion Detection Systems". In: *Array* 26 (2025), p. 100406. DOI: [10.1016/j.array.2025.100406](https://doi.org/10.1016/j.array.2025.100406).
- [3] Fatima Jobran ALzaher and Asma AlJarullah. "Intrusion Detection Using Machine Learning and Deep Learning". In: *International Journal of Advanced Computer Science and Applications* 16.8 (2025), pp. 440–454.
- [4] Kelly Anderson. "Comparative Study of White-Box and Black-Box Adversarial Attacks on Network Intrusion Detection Models". In: (2024).
- [5] Tulasi sai Charan sharma Gaddam. "Adversarial Vulnerability Analysis of Deep Neural Network-Based Intrusion Detection Systems Using FGSM and PGD Attacks". In: *2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*. 2026, pp. 1–6. DOI: [10.1109/IITCEE67948.2026.11394707](https://doi.org/10.1109/IITCEE67948.2026.11394707).
- [6] Sydney Mambwe Kasongo. "A Deep Learning Technique for Intrusion Detection System Using a Recurrent Neural Networks Based Framework". In: *Computer Communications* 196 (2023), pp. 57–73. DOI: [10.1016/j.comcom.2022.10.015](https://doi.org/10.1016/j.comcom.2022.10.015).
- [7] Ilana Löwy. "ART with PGD: risky heredity and stratified reproduction". In: *Reproductive Biomedicine And Society Online* 11 (2020). Reprotech in France and the United States: differences and similarities, pp. 48–55. ISSN: 2405-6618. DOI: <https://doi.org/10.1016/j.rbms.2020.09.007>. URL: <https://www.sciencedirect.com/science/article/pii/S240566182030023X>.
- [8] Vyshnavi Manduru et al. "A Light Gradient Boosted Model for Network Intrusion Detection". In: *2024 International Conference on Smart Systems for Electrical, Electronics, Communication and Computer Engineering (ICSSECC)* (2024), pp. 1–6. URL: <https://api.semanticscholar.org/CorpusID:272372361>.
- [9] Microsoft. *Jupyter Notebooks in VS Code*. 2023. URL: <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>.

- [10] Abdulrahman Mohammed, Muhanad Arif, and Ali Ali. "A multilayer perceptron artificial neural network approach for improving the accuracy of intrusion detection systems". In: *IAES International Journal of Artificial Intelligence (IJ-AI)* 9 (Dec. 2020), p. 609. DOI: [10.11591/ijai.v9.i4.pp609-615](https://doi.org/10.11591/ijai.v9.i4.pp609-615).
- [11] Nour Moustafa, Benjamin Turnbull, and Kim-Kwang Raymond Choo. "An Ensemble Intrusion Detection Technique Based on Proposed Statistical Flow Features for Protecting Network Traffic of Internet of Things". In: *IEEE Internet of Things Journal* 6.3 (2019), pp. 4815–4830. DOI: [10.1109/JIOT.2018.2871719](https://doi.org/10.1109/JIOT.2018.2871719).
- [12] Haedar Ahmed Mukhef. "Adversarial Robustness of Network IDS (Structured Data)". In: *Journal of Al-Qadisiyah for Computer Science and Mathematics* 17.4 (2025), Comp 209–224. DOI: [10.29304/jqcsms.2025.17.42554](https://doi.org/10.29304/jqcsms.2025.17.42554). URL: <https://jqcsms.qu.edu.iq/index.php/journalcm/article/view/2554>.
- [13] Nishank Parekh et al. "Network Intrusion Detection System Using Optuna". In: *2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS)*. 2024, pp. 312–318. DOI: [10.1109/ICICNIS64247.2024.10823298](https://doi.org/10.1109/ICICNIS64247.2024.10823298).
- [14] Anshika Sharma and Himanshi Babbar. "Understanding IoT-23 Dataset: A Benchmark for IoT Security Analysis". In: *2024 4th International Conference on Intelligent Technologies (CONIT)*. 2024, pp. 1–5. DOI: [10.1109/CONIT61985.2024.10627334](https://doi.org/10.1109/CONIT61985.2024.10627334).
- [15] Gurbakhsis Singh and Meenakshi Bansal. "An Analytical Review of Deep Learning Models and Datasets for Anomaly-Based Intrusion Detection Systems". In: *Cuestiones de Fisioterapia* 52.3 (2023). DOI: [10.48047/qdgj7n28](https://doi.org/10.48047/qdgj7n28). URL: <https://doi.org/10.48047/qdgj7n28>.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga